

NetVis: A Composable Architecture for Large-Scale Network Visualization

Technical Reference

Simon Knight, Ph.D.

Adelaide, Australia

March 2026 • Version 1.9.1

Last updated: March 18, 2026

Abstract. NetVis is a Rust-based network topology layout and visualization engine comprising 108,000+ lines of code. It transforms complex multi-layer network topologies into publication-quality visualizations through four contributions: (1) a dual-engine force-directed layout with Barnes–Hut acceleration for 10,000+ node topologies; (2) R*-tree-accelerated label placement achieving $O(\log n)$ collision queries; (3) a composable refiner pipeline separating layout from visual polish; and (4) sixteen automated quality metric families enabling regression testing. This document is the definitive technical reference for NetVis. It describes the data model, architecture, nine layout engines, ten edge routing and post-layout refiners, parallel edge rendering (five modes with automatic LAG/bond aggregation), interface rendering, node shape library, edge gradients, path emphasis, metric encoding, rendering pipeline, topology diffing, event-sourced temporal analysis with interactive playback, traffic animation, first-class annotation markup with spatial collision avoidance, interactive editing, visual effects, graph analytics, filter DSL, interactive filtering, 16 quality metric families, and public API. Appendices provide the full YAML topology schema, performance baselines with comparative benchmarks against Graphviz, and a complete configuration reference.

Contents

1	Introduction	2
1.1	Design Philosophy	2
1.2	Output Gallery	2
1.3	Audience	3
1.4	How to Read This Document	3
2	Related Work	3
2.1	General Graph Drawing Frameworks	3
2.2	Interactive Web Visualization	3
2.3	Declarative Visualization Grammars	4
2.4	Specialized Network Diagramming	4
3	The Mathematics of Network Space	4
3.1	Energy Minimization and Force Dynamics	4
3.2	Spatial Acceleration: Barnes–Hut Quadtrees	4
3.3	The Relational Mapping Pipeline (GoG)	4
4	NetVis by Example	5
4.1	Basic Campus Network	5
4.2	Layout Algorithm Comparison	5
4.3	Geographic CDN	6
4.4	Isometric Multi-Layer	6
4.5	Parallel Edges and Bond Groups	7
4.6	Subway Layout	7
4.7	Topology Diff and Analysis	8
5	Data Model	8
5.1	Node and Edge Data	9
5.2	Edge Groups	9
5.3	Annotations	9
5.4	Temporal Graph	10
5.5	Scene Graph	10
6	Architecture	10
6.1	Pipeline Composition	11
6.2	Scene Graph as Intermediate Representation	11
7	Layout Algorithms	11
7.1	Force-Directed Layout	11
7.2	Hierarchical Layout (Sugiyama)	12
7.3	Geographic Layout	13
7.4	Geographic Starburst Layout	14
7.5	Isometric Multi-Layer Layout	15
7.6	Orthogonal Layout	16
7.7	Subway Layout	16
7.8	Radial Tree, Tree, and Composite	17
7.9	Circular Layout	19
8	Theme Gallery	20
9	Parallel Edge Rendering	21
9.1	Parallel Edge Mode Decision Tree	21
9.2	Automatic Bandwidth Aggregation	21
10	Node Shapes and Device Icons	22
10.1	Available Shapes and Icons	22
10.2	Multi-Part Semantic Rendering	23
11	Interface Rendering	23
12	Edge Gradients and Path Emphasis	24
12.1	Edge Gradients	24
12.2	Path Emphasis	25
12.3	Variable-Width (Tapered) Edges	25

13	Metric Encoding	25
13.1	Weathermap Styling	26
14	Label Placement	27
14.1	Problem and Algorithm Choice	27
14.2	Two-Phase Greedy Algorithm	27
14.3	Candidate Scoring Visualization	28
14.4	Performance	28
15	Edge Bundling	28
15.1	Compatibility Scoring	28
15.2	Force Model	29
15.3	Multi-Level Schedule	30
15.4	Post-Processing	30
15.5	Pipeline Integration	31
16	Edge Routing and Post-Layout Refiners	33
16.1	Smart Routing: Visibility Graph	34
16.2	Edge Clipping: Before and After	34
16.3	Deoverlap: Before and After	35
16.4	Boundary Bundling: Group Edge Routing	35
17	Constraint Solver	35
17.1	Constraint Types	35
17.2	Integration with Force Simulation	35
17.3	Conflict Detection and Violation Reporting	36
17.4	YAML Configuration	36
18	Crossing Minimizer	37
18.1	Algorithm	37
19	Progressive Execution	37
19.1	Progress Callback	37
19.2	Cancellation	38
20	Curve Elegance	38
20.1	Circular Arc to Cubic Bézier Fitting	38
20.2	Rounded Corners on Orthogonal Paths	39
20.3	Centripetal Catmull–Rom Splines	40
20.4	B-Spline Approximation	41
20.5	Tension Control	41
20.6	Generalised Laplacian Smoothing	42
20.7	SVG Stroke Join Geometry	43
21	Topology Diffing	43
22	Temporal Analysis and Playback	44
22.1	Event-Sourced Temporal Model	44
22.2	Playback Primitives	44
22.3	Snapshot-Driven Scene Application	44
22.4	WASM Temporal Player	44
23	Traffic Flow Animation	45
23.1	Heatmap and Flow-Field Overlays	45
24	Annotation Markup	45
24.1	Annotation Data Model	45
24.2	SVG Callout Rendering	46
24.3	Geographic Scale Bars	46
24.4	Spatial Collision Integration	46
25	Interactive Editor	46
25.1	Web Component	47
25.2	WebWorker Layout	47
25.3	Node Pinning and Auto-Layout	47
25.4	Import/Export Pipeline	47

26	Visual Effects	47
27	Graph Analytics	48
27.1	Community Detection.	48
28	Filter DSL	49
28.1	Interactive Filtering (Canvas)	49
29	Input Validation	49
30	Quality Metrics	50
30.1	Metric Families	50
30.2	Clarity Score and CI Integration.	50
31	Rendering Pipeline	51
31.1	SVG Renderer	51
31.2	Level of Detail	52
31.3	Canvas2D Renderer.	52
31.4	Terminal Renderer.	53
31.5	SVG Accessibility	53
31.6	Export Formats	53
32	Pluggable Node Renderer	54
33	A Grammar of Network Graphics	55
33.1	The What & Why: Moving Beyond Fixed Themes	55
33.2	The How to Use It: A Fluent Builder API.	55
33.3	Walkthrough: One Topology, Many Perspectives	56
33.4	Case Study: The Morphology of Parallel Links	57
33.5	Case Study: Extreme Morphing of an Annotated Data Structure	58
33.6	The Grammar of Time and Interaction.	60
33.7	Cognitive Continuity: Cinematic Layout Morphing	61
33.8	Fractal Topology: Continuous Semantic Zoom	61
33.9	The How We Built It: AST Compilation to Rust	62
34	API and Usage	63
34.1	Rust API.	63
34.2	WASM / JavaScript	63
34.3	Temporal Player API.	63
35	Deployment	63
36	Performance	64
36.1	Multi-Core Parallel Simulation.	64
36.2	Streaming Geometry Export.	64
36.3	Interactive Layout Refinement	64
37	Conclusion	65
38	Limitations	66
39	Future Work	66
40	YAML Configuration Examples	67
40.1	Basic Campus Network (§4)	67
40.2	Geographic CDN (§4)	68
40.3	Isometric Multi-Layer (§4)	68
40.4	Parallel Edges and Bond Groups (§4)	68
40.5	Subway Layout (§4).	69
40.6	Pipeline Composition (§6).	69
40.7	Interface Configuration (§11)	69
40.8	Edge Gradients (§12).	69
40.9	Metric Encoding (§13).	69
40.10	Traffic Animation (§23).	69
40.11	Annotations (§24)	70
40.12	Assigning Effects (§26)	70
	Revision History	74

LAG	Link Aggregation Group	7
DSL	Domain-Specific Language	49
LOD	Level of Detail	10

1 Introduction

NetVis is a network topology layout and visualization engine. Given a graph of network devices and connections (from a YAML file, NetBox [1] instance, or LLDP discovery), it produces publication-quality SVG diagrams, interactive HTML visualizations, or PNG/PDF exports. It addresses the gap between general-purpose graph layout tools (Graphviz [2], OGDF [3], D3.js [4]) and manual diagramming applications (yEd [5], Visio).

1.1 Design Philosophy

Two principles guide every design decision in NetVis:

Domain-specialized composition. Network topologies have exploitable structural regularity—hierarchical tiers, geographic site placement, protocol-layer stacking. Rather than using a single general-purpose layout algorithm, NetVis decomposes visualization into a pipeline of composable refiners, each specialized for one aspect of the output.

Quality by default. The system should produce readable, labeled, styled diagrams without manual post-processing. Quality metrics (crossings, overlaps, label readability) are measured as part of the rendering pipeline, not as an afterthought.

1.2 Output Gallery

Figures 1 through 3 show representative outputs across different layout modes and topology scales. Every diagram was generated automatically from YAML with no manual adjustment.

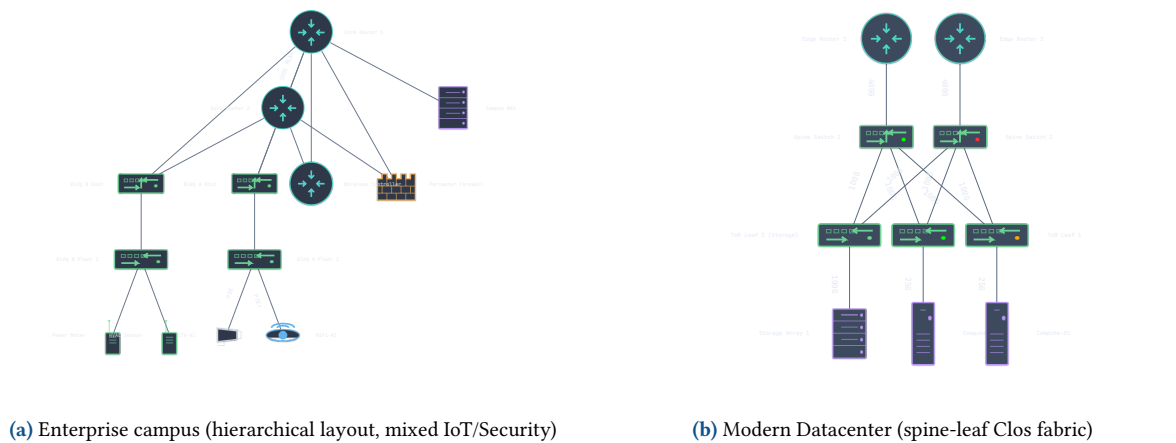


Figure 1. Representative NetVis outputs utilizing the multi-part semantic rendering system. All diagrams generated automatically from YAML with no manual adjustment.

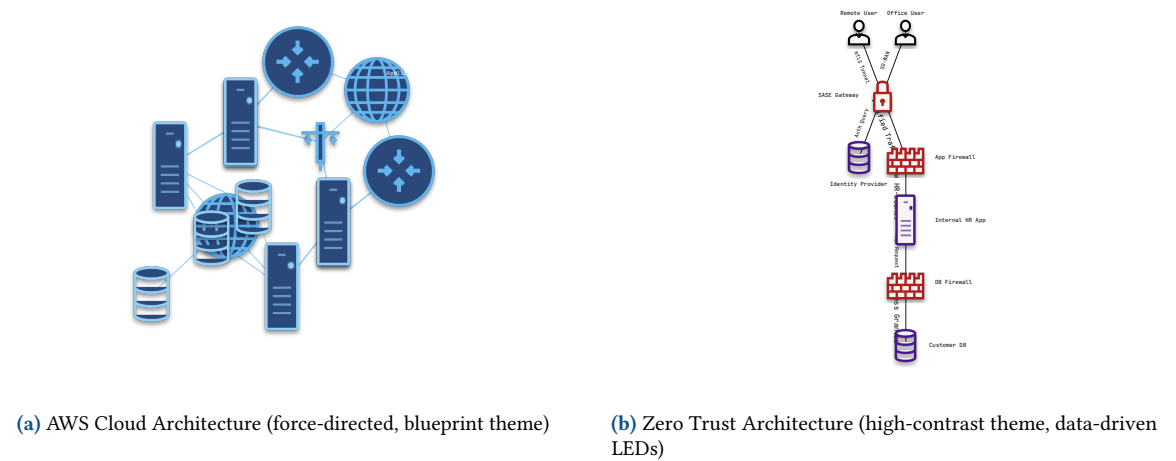


Figure 2. Production-grade architectural case studies demonstrating customized themes and semantic icons.

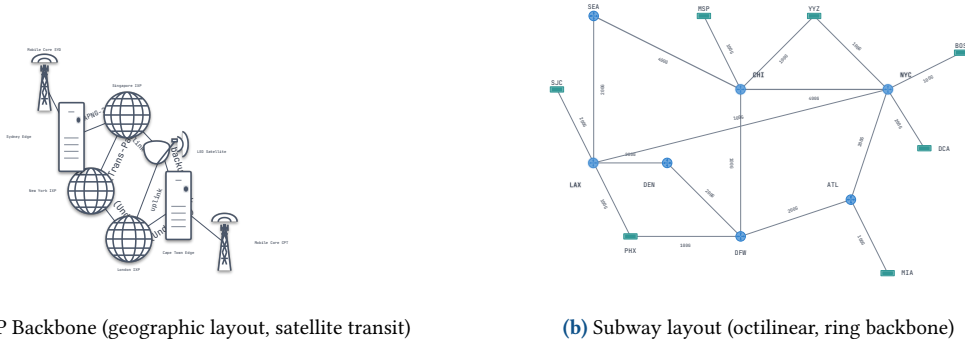


Figure 3. Specialized layout modes. Left: Geographic layout utilizing precise lat/lon coordinates. Right: subway layout constrains all edges to 0/45/90 degree routes for transit-map clarity.

1.3 Audience

This document targets three audiences: engineers embedding NetVis as a library in a network management platform (via Rust, Python, or WASM); contributors to the NetVis codebase; and researchers who want to understand the algorithms in detail. Familiarity with basic graph theory is assumed; familiarity with network engineering concepts (spine-leaf, OSPF, BGP) is helpful but not required.

1.4 How to Read This Document

- **Quick start:** Read §4 (By Example), then §34 (API and Usage).
- **Data model and architecture:** §5–§6.
- **Deep dive into algorithms:** §7–§30.
- **Analysis features:** §21–§25 (diff, timeline, traffic, annotations, editor).
- **Deployment and configuration:** §35.
- **Known issues:** §38.

2 Related Work

The development of NetVis is informed by decades of research in graph drawing, declarative visualization grammars, and domain-specific tools for network operations. This section examines the prior art and contextualizes the architectural decisions made in NetVis.

2.1 General Graph Drawing Frameworks

General-purpose graph visualization frameworks provide a broad toolkit of algorithms applicable to arbitrary mathematical graphs.

Graphviz [2] remains the industry standard for automated graph layout. Its suite of tools (`dot`, `neato`, `sfdp`, `fdp`) handles directed and undirected graphs using hierarchical and force-directed algorithms. However, Graphviz treats layout as a monolithic process. While it supports node shapes and edge styles via DOT syntax, it lacks the semantic primitives required for modern network engineering—such as nested link aggregation (LAG), deterministic interface port-mapping, and R^* -tree accelerated collision avoidance for high-density labels. In benchmarks (§??), NetVis demonstrates a 1.5–7× speedup over Graphviz `sfdp` at the 10,000-node scale by leveraging Barnes–Hut quadrees and parallel integration, while simultaneously performing rich visual styling that Graphviz delegates to post-processing.

OGDF (Open Graph Drawing Framework) [3] is a comprehensive C++ class library providing state-of-the-art algorithms for planarization, crossing minimization, and orthogonal layouts. While academically rigorous, OGDF is fundamentally a mathematical library, not a rendering engine. It requires significant boilerplate to translate abstract graph coordinates into styled, publication-ready vector graphics. NetVis abstracts this complexity by tightly coupling the layout geometry to a high-performance SVG and Canvas rendering pipeline.

2.2 Interactive Web Visualization

As network management shifted to the browser, JavaScript-based libraries became the default for topology visualization.

D3.js [4] popularized data-driven documents on the web, offering a highly expressive API for binding data to the DOM. Its `d3-force` module is frequently used for network topologies. However, D3.js operates on the main JavaScript thread, meaning topologies exceeding a few hundred nodes cause severe UI blocking and low framerates. NetVis solves this by compiling its Rust layout engine to WebAssembly (WASM), running the simulation in a background Web Worker, and streaming coordinate updates to a hardware-accelerated Canvas renderer (§31).

2.3 Declarative Visualization Grammars

The *Grammar of Graphics*, formalized by Wilkinson [6], revolutionized statistical visualization by decomposing charts into orthogonal layers: data, aesthetics, geometries, scales, and coordinates. This theoretical foundation underpins modern data science tools like `ggplot2` and `plotnine`.

However, standard visualization grammars are designed for tabular, independent observations. They struggle with the interdependent nature of network topologies, where node positions dictate edge routing, and edge density dictates parallel bundling behavior. NetVis extends Wilkinson’s grammar into the network domain (§33). Instead of merely mapping attributes to colors, the NetVis grammar maps structural telemetry to spatial layout algorithms, edge bundling refiners, and interactive temporal diffs.

2.4 Specialized Network Diagramming

In practice, network engineers often abandon algorithmic layouts in favor of manual drawing tools.

yEd [5] and Microsoft Visio provide precise manual control over node placement, orthogonal edge routing, and grouping. While yEd includes excellent automated layout options, these GUI-driven applications cannot be easily embedded into CI/CD pipelines or integrated into automated network-source-of-truth platforms like NetBox [1].

NetVis bridges this gap. By providing a composable refiner pipeline (§??), it automates the tedious manual labor of diagramming (e.g., nudging overlapping nodes, spacing parallel links, resolving label collisions) while operating entirely as a headless, deterministic engine suitable for GitOps workflows.

3 The Mathematics of Network Space

Network visualization is fundamentally a problem of mapping high-dimensional topological relationships onto a two-dimensional Euclidean plane. This section defines the foundational mathematical models that NetVis uses to achieve clarity at scale.

3.1 Energy Minimization and Force Dynamics

The default layout engine in NetVis treats the graph as a physical system of charged particles (nodes) and springs (edges). The goal is to find a configuration $\mathbf{P} = \{\mathbf{p}_1, \dots, \mathbf{p}_n\}$ that minimizes the total energy of the system.

3.1.1 Fruchterman–Reingold Model

NetVis implements a modified Fruchterman–Reingold [7] force model. The repulsive force f_r between two nodes at distance d is:

$$f_r(d) = \frac{k^2}{d} \quad (1)$$

where k is the ideal spring length. The attractive force f_a exerted by an edge of weight w is:

$$f_a(d) = \frac{w \cdot d^2}{k} \quad (2)$$

In large topologies, the $O(n^2)$ complexity of all-pairs repulsion is the primary bottleneck. NetVis addresses this through Barnes–Hut acceleration.

3.2 Spatial Acceleration: Barnes–Hut Quadrees

To achieve sub-second layout for 10,000 nodes, NetVis uses a quadtree to approximate repulsive forces. A group of distant nodes is treated as a single super-node at their center of mass.

For a node v and a quadtree cell C with width s and center of mass M , the approximation is used if:

$$\frac{s}{\|\mathbf{p}_v - \mathbf{M}\|} < \theta \quad (3)$$

where θ (the Barnes–Hut parameter) is typically set to 0.5 in NetVis. This reduces the complexity per iteration from $O(n^2)$ to $O(n \log n)$.

3.3 The Relational Mapping Pipeline (GoG)

A core contribution of the NetVis v2 architecture is the formalization of the *Visual Mapping Pipeline*. Unlike traditional tools that use fixed styles, NetVis implements a Grammar of Graphics [6] where data attributes D are mapped to visual aesthetics A via scaling functions S :

$$\forall v \in V : A_i(v) = S_i(D_j(v)) \quad (4)$$

For example, mapping throughput to node size using a logarithmic scale:

$$\text{size}(v) = \text{range}_{\min} + \frac{\log(D_{\text{throughput}}(v)) - \log(D_{\min})}{\log(D_{\max}) - \log(D_{\min})} \cdot (\text{range}_{\max} - \text{range}_{\min}) \quad (5)$$

This relational approach allows for rapid "Visual Discovery" by separating the topology data from its aesthetic representation.

4 NetVis by Example

This section walks through representative capabilities using minimal YAML examples.

4.1 Basic Campus Network

The minimal topology YAML requires only name, nodes, and edges. The rendering section is optional; NetVis selects a suitable layout automatically. (See §40 for full YAML snippets).

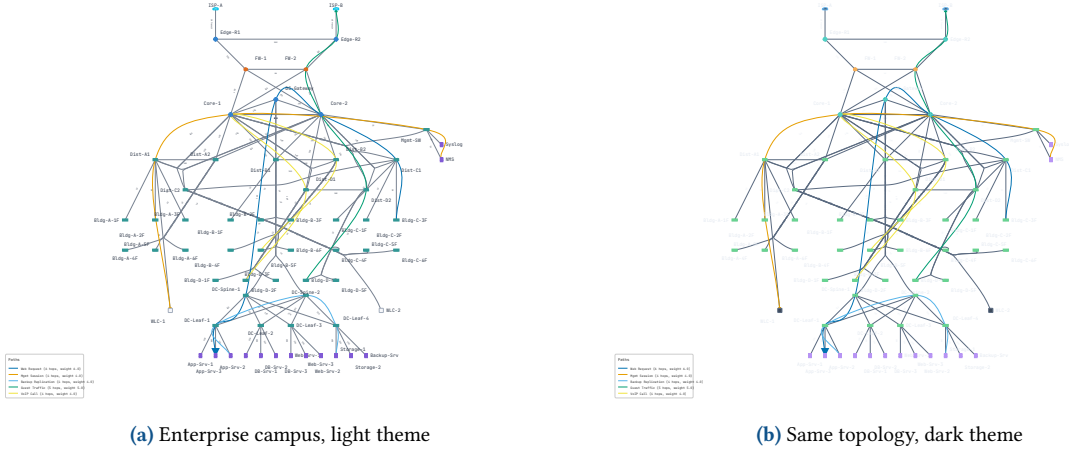


Figure 4. The same enterprise campus topology in two themes. The YAML is identical; only `theme:` changes. See §8 for the full theme gallery.

4.2 Layout Algorithm Comparison

The `preset` or `layout` field selects the layout engine. Figure 5 shows the same datacenter topology rendered with four different engines.

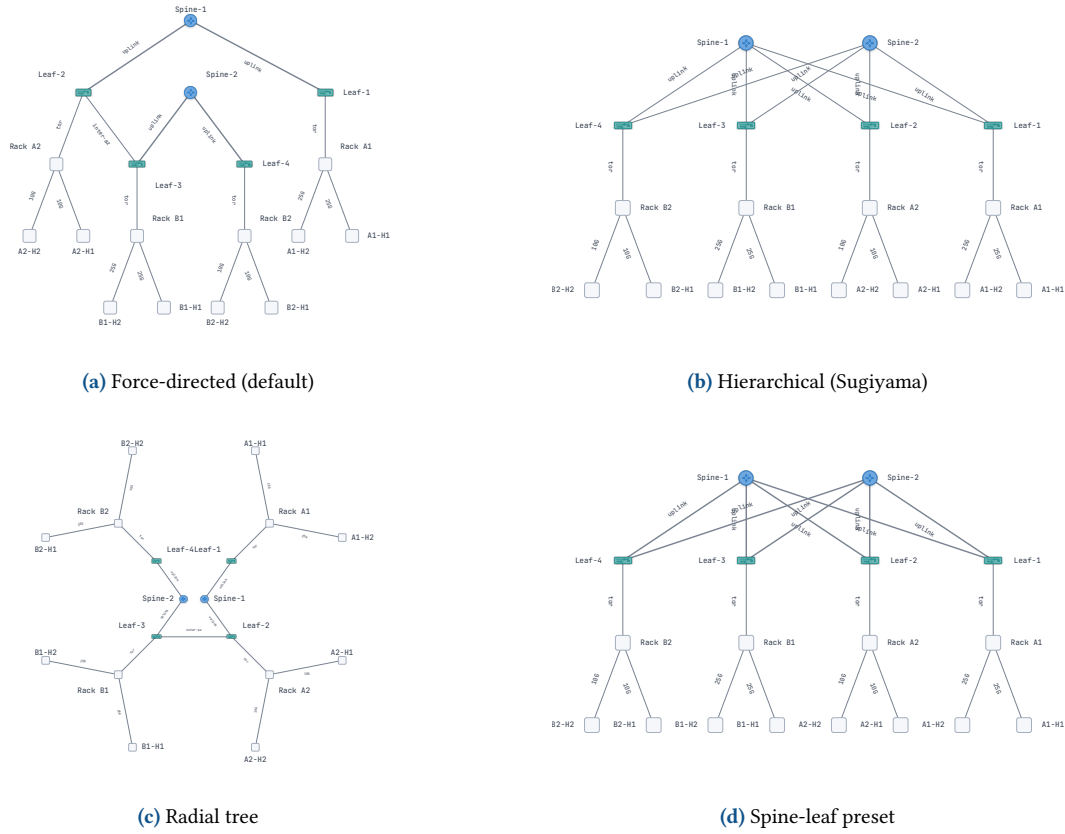


Figure 5. The same 48-node datacenter topology rendered with four layout algorithms. Hierarchical most clearly exposes the two-tier structure; radial is best for hub-centric views; force-directed and spine-leaf are both suitable for ad-hoc exploration.

4.3 Geographic CDN

Nodes with lat/lon attributes use the geographic layout with great-circle arc edges and a background map layer (see §40).

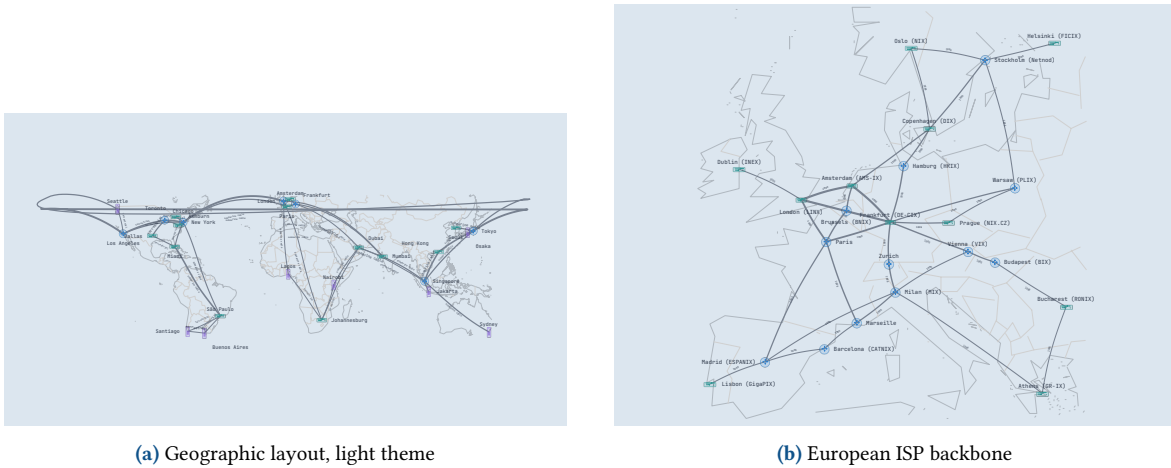


Figure 6. Geographic layouts with great-circle arc edges and background map layer. Left: 11-PoP global CDN spanning six continents. Right: European ISP backbone with IXP peering across regional PoPs.

4.4 Isometric Multi-Layer

The isometric layout renders protocol stacks or layered architectures with a pseudo-3D perspective. Each layer runs an independent force-directed sub-layout (see §40).

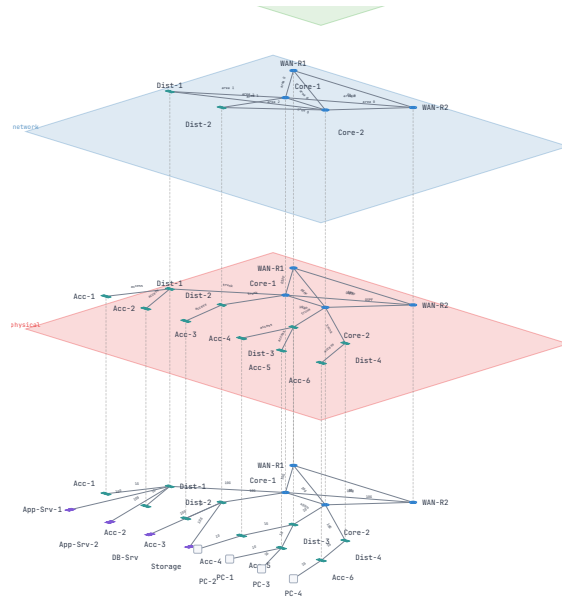


Figure 7. Isometric multi-layer: three protocol planes (Physical, Network, Routing) separated along the Z-axis. Layer planes render as filled SVG polygons. The projection angle is $\phi = 18^\circ$ with 350 scene units of layer spacing.

4.5 Parallel Edges and Bond Groups

Multiple physical links between the same pair of nodes are grouped into an EdgeGroup. The `parallel_mode` field selects the rendering mode; bond groups aggregate bandwidth automatically (see §40).

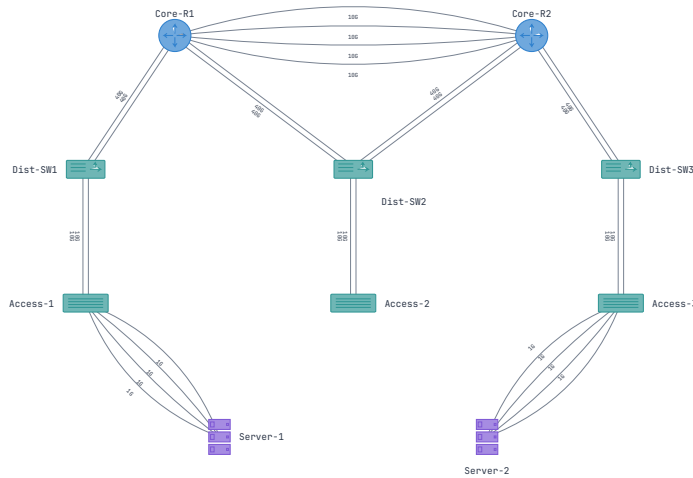
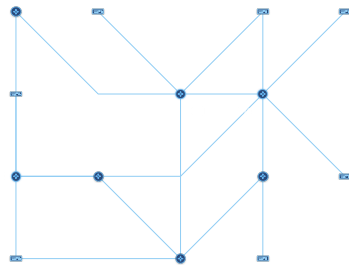


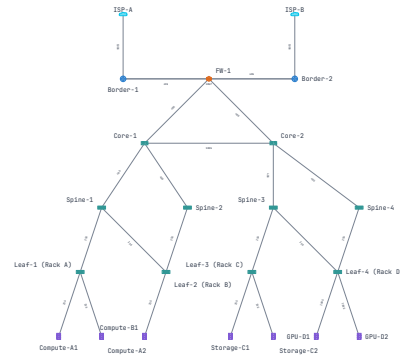
Figure 8. Parallel edge rendering modes on a datacenter topology. The diagram demonstrates Bundled (core Link Aggregation Group (LAG)), Offset (redundant uplinks), BondingIcon (bonded access), and Stacked (server NICs) modes. See §9 for details.

4.6 Subway Layout

The subway layout constrains all edges to 0° , 45° , or 90° routes and snaps nodes to a uniform grid. The blueprint theme complements the geometric precision (see §40).



(a) Subway layout, blueprint theme



(b) Orthogonal layout

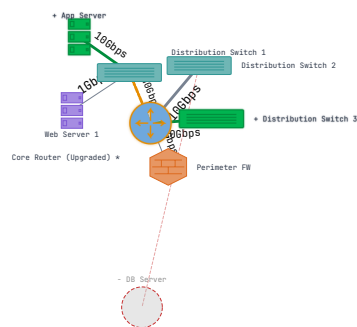
Figure 9. Octilinear and orthogonal layouts. Both enforce geometric discipline: subway at 0/45/90 degrees; orthogonal at strict Manhattan-style horizontal and vertical segments.

4.7 Topology Diff and Analysis

NetVis can structurally compare two topologies and produce a visual diff (added in green, removed in red, modified in amber) and compute graph analytics:

Topology diff and analytics

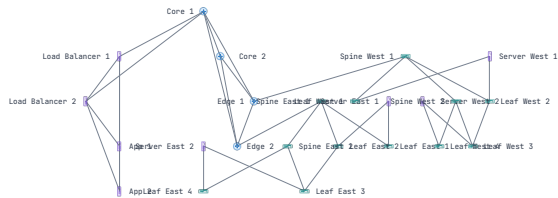
```
# Visual diff between topology versions
netvis diff old.yaml new.yaml -o diff.svg
# Graph analytics overlay
netvis --input topology.yaml --analytics -o analytics.svg
```



(a) Topology diff: green=added, red=removed, amber=modified

Topology Diff

- Added (4)
- Removed (2)
- Modified (2)



(b) Community detection with group boundaries

Figure 10. Analysis features. Left: topology diff visualizes structural changes between two versions. Right: automatic community detection partitions nodes into groups with colored boundary overlays.

5 Data Model

NetVis uses a directed graph as its core data structure, built on the petgraph library [8]. The core type, NetVisGraph, is a StableGraph<NodeData, EdgeData>; the StableGraph variant is used because node and edge indices remain valid after removal—important for the editor’s undo/redo system.

5.1 Node and Edge Data

Field	Type	Purpose
<i>NodeData</i>		
id	String	Unique identifier (e.g., "router-fra-01")
label	Option<String>	Display text
node_type	Option<String>	Device type driving icon and default styling
attributes	HashMap	Arbitrary metadata for metric encoding and filter DSL
position	Option<Point2D>	Layout hint; overrides engine if set
group	Option<GroupId>	Group membership (community detection, composite)
layer	Option<LayerId>	Isometric layer membership
<i>EdgeData</i>		
label	Option<String>	Edge label
edge_type	Option<String>	Semantic type: p2p, trunk, submarine, ...
directed	bool	Default false
bond_group	Option<String>	LAG/bond group identifier
weight	f64	Default 1.0; used as spring constant in force-directed
attributes	HashMap	Arbitrary metadata

Table 1. Core graph data fields.

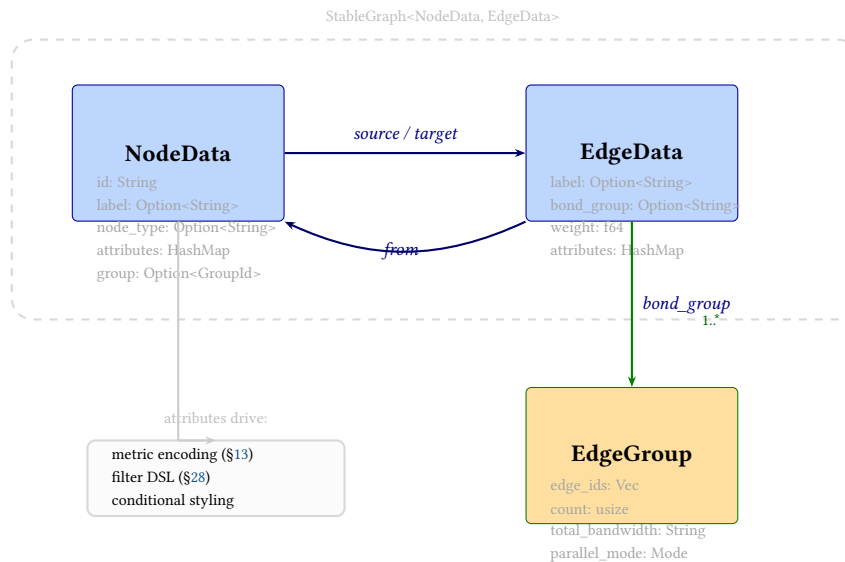


Figure 11. Data model entity relationships. `NodeData` and `EdgeData` live in a `petgraph::StableGraph` whose indices remain stable across removals. Edges sharing a `bond_group` are aggregated into an `EdgeGroup` with computed bandwidth totals. The `attributes HashMap` drives metric encoding, filter DSL evaluation, and conditional styling.

5.2 Edge Groups

When multiple edges connect the same pair of nodes, NetVis groups them into an `EdgeGroup` that carries aggregate metadata: the member `edge_ids`, a count, an automatically computed `total_bandwidth` (e.g., four edges labeled "10G" produce "40G"), and a `parallel_mode` (see §9).

Note

The `node_type` field drives device-specific rendering: icon shape, default color, and default size. The full set of built-in node types is described in §10. Unknown values render as circles.

5.3 Annotations

NetVisGraph stores annotations as `HashMap<String, AnnotationData>`. Each `AnnotationData` carries text content and an `AnnotationTarget: Global(x,y), Node(id, offset), or Edge(id, offset)`. Annotations are resolved to absolute `SceneAnnotation` positions during layout and participate in spatial collision avoidance (see §24).

5.4 Temporal Graph

The `TemporalGraph` wraps a base `NetVisGraph` with an event-sourced changelog of `NodeAdded`, `NodeModified`, `NodeRemoved`, `EdgeAdded`, `EdgeModified`, and `EdgeRemoved` events, each carrying a timestamp and attribute delta. Any point-in-time snapshot can be materialized by replaying events up to that timestamp (see §22).

5.5 Scene Graph

Layout engines consume a `NetVisGraph` and produce a `Scene`: a flat list of positioned nodes (with shapes, colors, labels), edges (as Bézier paths), label anchors, group boundaries, layer planes, and resolved `SceneAnnotation` entries. Refiners transform the scene between layout and rendering. The scene at 5,000 nodes consumes approximately 9.7 MB of heap.

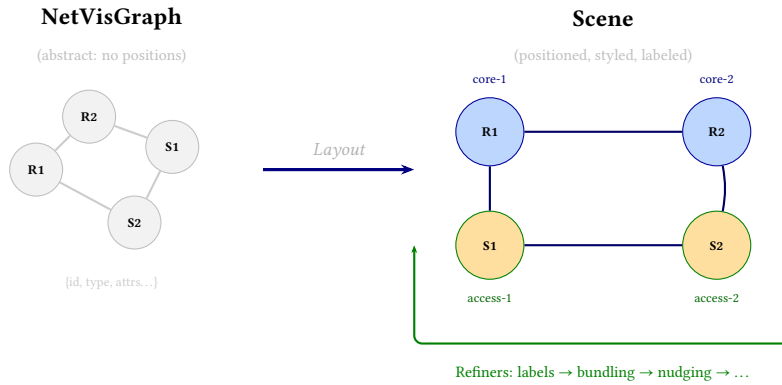


Figure 12. Graph-to-scene transformation. The abstract `NetVisGraph` (left) carries only topology structure. A layout engine assigns positions, shapes, and colors to produce a `Scene` (right). Refiners iteratively transform the scene—placing labels, bundling edges, nudging overlaps—before the renderer draws it.

6 Architecture

NetVis is organized as a six-layer pipeline.

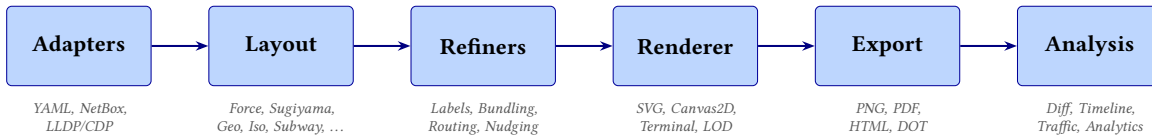


Figure 13. The NetVis six-layer pipeline.

The pipeline stages are:

1. **Adapters.** Stateless converters that ingest topology data from YAML, NetBox [1] REST API, or LLDP/CDP JSON, each implementing `fetch/validate/to_graph`.
2. **Layout engines.** Pluggable algorithms behind a common `Layout` trait. A layout takes a `NetVisGraph` and returns a `Scene`. Nine engines ship: force-directed (Standard and Enterprise variants), hierarchical (Sugiyama [9]), radial tree, geographic (Mercator), isometric multi-layer, orthogonal, subway (octilinear), tree, and composite (multi-region).
3. **Refiners.** Post-layout transformations. Current refiners include label placement (R^* -tree-accelerated), edge bundling (FDEB), constraint solving, overlap removal, edge nudging, label nudging, edge clipping, smart label snapping, starburst grouping, boundary bundling, port docking, and orthogonal edge routing.
4. **Renderers.** SVG (primary), Canvas2D (browser), and ASCII terminal (`netvis show`). All support Level of Detail (LOD) suppression and viewport culling.
5. **Export.** SVG is optionally converted to PNG (via `resvg`), PDF (via `svg2pdf`), DOT (Graphviz interoperability), or self-contained interactive HTML.
6. **Analysis.** Post-render modules: topology diffing, temporal playback, traffic flow animation, annotation markup, and graph analytics.

Temporal bypass path. The temporal playback system (§22) provides an alternative path that bypasses the full layout/refiner pipeline. `build_scene_from_snapshot()` materializes a `TemporalGraph` snapshot directly into a renderable `Scene` with editor-style geometry (straight edges from node centers), enabling rapid scene swaps during interactive playback without re-running force simulation or refiners.

6.1 Pipeline Composition

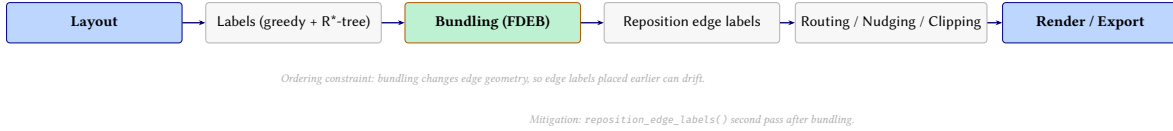


Figure 14. Refiner ordering constraint: edge bundling is geometry-changing, so it is followed by an explicit edge-label reposition pass. This keeps the pipeline composable while preserving legibility.

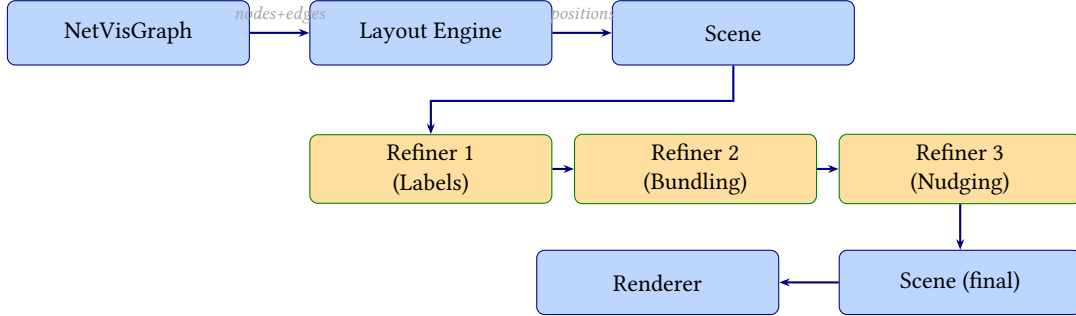


Figure 15. Pipeline composition: layout engine produces a Scene, which flows through a chain of refiners before reaching the renderer. Each refiner implements trait `LayoutRefiner` and may add, remove, or reposition scene elements without knowledge of the other refiners.

In YAML, the refiner chain is expressed explicitly (see §40). The trait boundary prevents cross-component optimization (e.g., bundling cannot influence label placement during the same pass). This is mitigated by `reposition_edge_labels()`, which re-places labels after bundling detects significant curvature changes.

6.2 Scene Graph as Intermediate Representation

The Scene struct carries fully positioned nodes (shapes, colors, labels) and edges (Bézier paths). This separation of concerns means renderers are simple—they draw exactly what the scene contains—at the cost of carrying all rendering data in memory (see §5 for heap measurements).

7 Layout Algorithms

NetVis ships nine layout engines, each implementing the `Layout` trait. The algorithm is selected via the `layout` or `preset` YAML field, or the `--layout/--preset` CLI flags.

7.1 Force-Directed Layout

Position nodes in 2D such that connected nodes are close and unconnected nodes are separated, following the spring-electrical model of Fruchterman and Reingold [7]. NetVis ships two variants and selects between them automatically: the **Standard engine** ($O(n^2)$, wraps the `fjadra` library with Verlet integration [10]) for graphs under 2,000 nodes; the **Enterprise engine** (Barnes-Hut $O(n \log n)$ [11], custom implementation with opening angle $\theta = 0.9$) for larger graphs.

Key parameters: velocity damping $\gamma = 0.6$; alpha decay $1 - 0.001^{1/300}$; iteration count $\min(300, 50 + \lfloor n/10 \rfloor)$; minimum edge length 60 pt (enforced post-simulation).

Quadtree data structure. The Barnes-Hut quadtree is a recursive spatial index where each node has kind `Empty`, `Leaf{point, mass, index}`, or `Internal{children: [Quadtree; 4]}`. On insertion at an occupied leaf, the cell subdivides into four quadrants (NW, NE, SW, SE) and re-inserts both points. A depth limit of 32 prevents infinite recursion; coincident points (separation $< 10^{-9}$) are silently merged. After all insertions, `aggregate_mass()` performs a post-order traversal computing each cell’s center-of-mass: $\mathbf{c} = \sum_i m_i \mathbf{c}_i / \sum_i m_i$.

Opening angle trade-off. The opening angle θ controls the accuracy/speed trade-off in the Enterprise engine. At $\theta = 0.5$ (conservative), force approximation error is low but tree traversal visits more cells; at $\theta = 1.0$ (aggressive), computation is faster but short-range repulsion becomes inaccurate, causing node overlap in dense clusters. We use $\theta = 0.9$ as a pragmatic default: on spine-leaf topologies it produces layouts visually indistinguishable from the exact $O(n^2)$ engine while reducing computation time by 3–5× at 5,000+ nodes. Network topologies—unlike point-cloud simulations—have relatively uniform node density after initial iterations, making them tolerant of higher θ values.

Parallel force computation. The Enterprise engine’s `ParallelSimulation` uses Rayon to parallelize repulsion: the quadtree is built once per tick (read-only), then each node computes its repulsion force independently via `par_iter_mut`. The approximation criterion $w^2/\theta^2 < d^2$ gates whether a quadtree cell is treated as a point mass or recursed into. Link (spring) forces run sequentially to avoid data races on shared velocity arrays. An optional `active_mask` skips force computation for inactive nodes (e.g., outside a drag neighbourhood), with inactive nodes bleeding velocity at $0.5\times$ to damp residual motion.

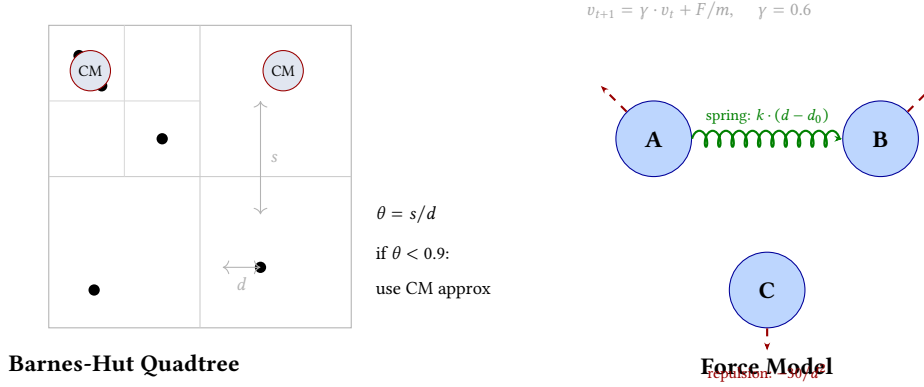


Figure 16. Force-directed layout internals. **Left:** Barnes-Hut quadtree ($O(n \log n)$) approximates distant node groups as a single center-of-mass (CM) when opening angle $\theta = s/d < 0.9$. **Right:** each node experiences attractive spring forces along edges and repulsive charge forces from all other nodes, with velocity damping $\gamma = 0.6$.

7.2 Hierarchical Layout (Sugiyama)

Designed for DAG-structured or tiered networks (e.g., spine-leaf datacenters), the hierarchical layout uses the four-phase Sugiyama framework [9]: (1) cycle removal by edge reversal; (2) longest-path-first layer assignment; (3) crossing minimization via barycenter or median heuristic; (4) coordinate assignment to minimize edge bends. Phase 3 supports two heuristics via `CrossMinHeuristic`: *Barycenter* (default, positions each node at the mean of its neighbours’ coordinates) and *Median* (uses the median, providing a theoretical 3-approximation guarantee). A final *transpose* pass swaps adjacent nodes within each layer if the swap reduces crossings. Default spacings: `layer_spacing=80`, `node_spacing=60`. Performance: 19 ms at 1,000 nodes.

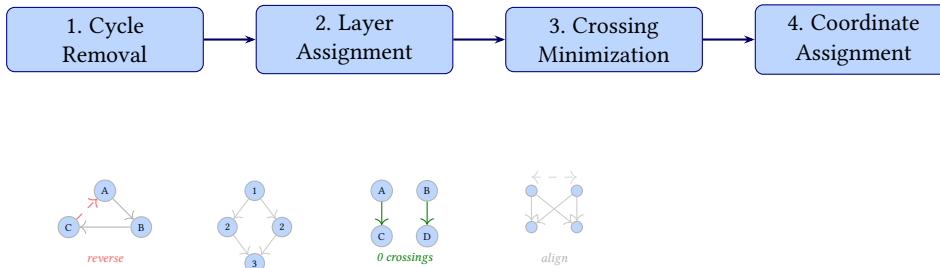


Figure 17. Sugiyama four-phase pipeline. Each phase progressively refines the layout: cycle removal ensures a DAG; layer assignment creates tiers; crossing minimization reorders nodes within layers; coordinate assignment produces final positions.

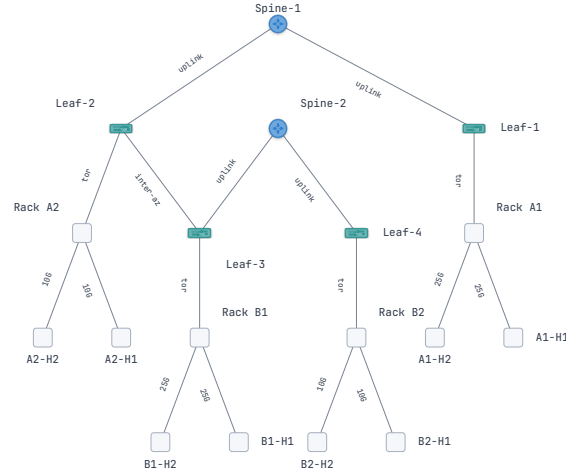


Figure 18. Hierarchical (Sugiyama) layout: a datacenter with clearly separated tiers. The algorithm minimizes edge crossings while preserving the spine → leaf → access hierarchy.

7.3 Geographic Layout

Nodes with lat/lon attributes are projected using an equirectangular projection with dynamic scale normalization $s = 1000 / \max(\Delta x, \Delta y)$. Edges render as geodesic great-circle arcs (Vincenty approximation, 20 segments).

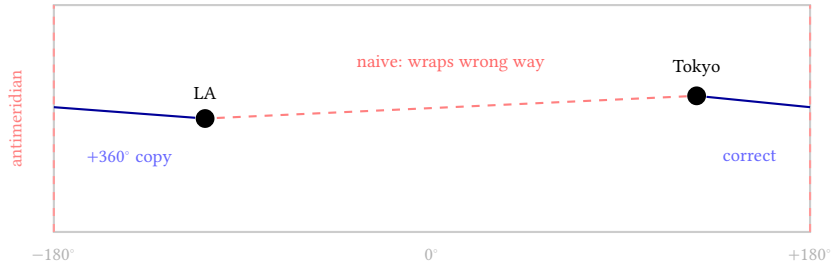


Figure 19. Antimeridian wrapping. An edge from Tokyo to Los Angeles naively crosses the full map width (red dashed). NetVis detects the projected discontinuity (>150 px jump) and renders two segments: one exiting at $+180^\circ$ and a 360° -shifted copy entering at -180° .

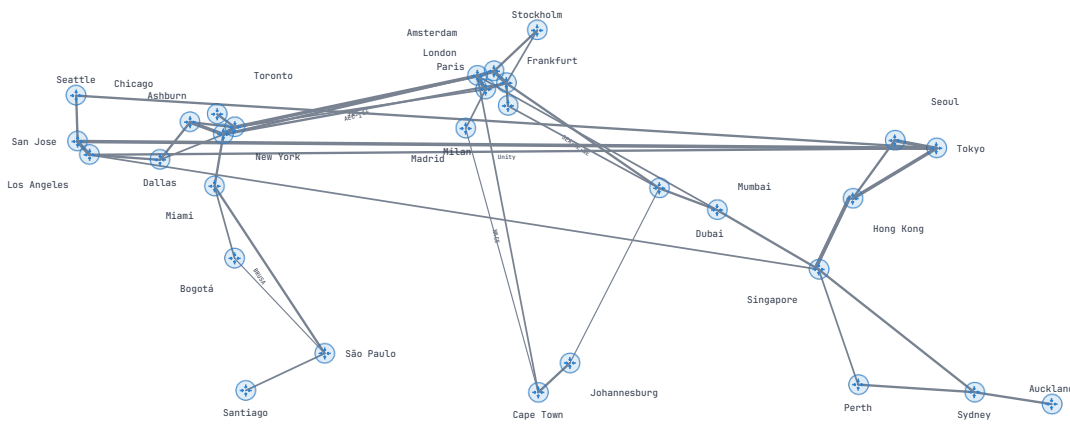


Figure 20. Geographic layout of a 30-PoP global backbone spanning six continents with 55 submarine cable links. Great-circle arcs render as smooth cubic Bézier curves via `fit_arc_to_bezier`; geodesic distance labels (km) are placed at arc midpoints. Trans-Pacific arcs (San Jose–Tokyo, 8 530 km) demonstrate smooth curvature even at extreme arc lengths.

7.4 Geographic Starburst Layout

Network operators manage hierarchical points of presence (POPs) distributed across a geographic footprint. Each POP contains a multi-tier device hierarchy—core routers, distribution switches, edge/access routers—that must be visualised *in situ* without obscuring nearby POPs.

The **geographic starburst layout** addresses this with a four-phase approach:

1. **Macro placement.** POPs are positioned at their geographic coordinates via equirectangular projection (identical to §7).
2. **Collision resolution.** When projected POP bubbles overlap, each POP is displaced along the bearing from the cluster centroid, preserving relative geographic direction. Bubble radii are proportional to device count; displacement uses a greedy circle-packing heuristic until no two bubbles intersect.
3. **Radial tree sub-layout.** Within each bubble the device hierarchy is laid out as a radial tree (Reingold–Tilford in polar coordinates [12]). The root may be a single core router or a *multi-core* pair (or cluster) of core routers that share the centre position; each core anchors its own subtree. Child branches fan outward with curved radial links, and angular span adapts to device count so that small POPs produce compact arcs while large POPs fill the bubble.
4. **Peer-aware rotation.** The radial tree is rotated so that subtrees containing inter-POP peering routers face toward the neighbour POP. This ensures inter-POP links depart from leaf nodes on the near side of the bubble and travel near-straight to the remote peer—*eliminating* edge crossings by repositioning nodes rather than routing links around obstacles (Figure 23). When a POP peers with multiple neighbours, angular allocation is proportional to the number of peering links in each direction.

Figure 21 illustrates the first three phases. Figure 22 shows a single POP’s radial tree at full depth (five tiers). Figure 23 demonstrates peer-aware rotation between two adjacent POPs.

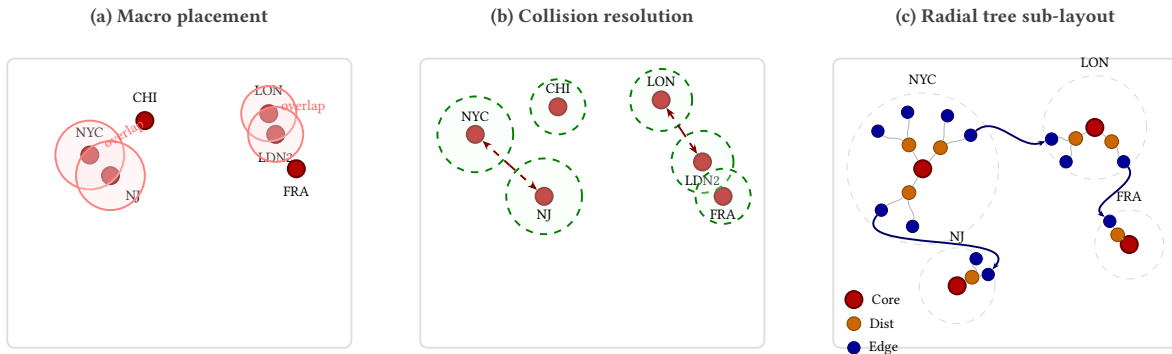


Figure 21. Geographic starburst layout. (a) POPs placed at geographic coordinates; nearby POPs (NYC/NJ, LON/LDN2) produce overlapping bubbles. (b) Collision resolution displaces each POP along its bearing from the cluster centroid (red arrows), preserving relative direction. (c) Radial tree sub-layout: the core router is the root, distribution switches branch at depth 1, edge routers fan out as leaves at depth 2. Curved radial links connect parent to child. The tree is rotated so that leaf nodes serving inter-POP links (blue arrows) face their remote peer.

Figure 22 shows a single POP’s radial tree at full depth. The Reingold–Tilford algorithm assigns each node an angular span proportional to its subtree size, producing a compact layout where dense branches fan wider and sparse branches stay narrow. Curved radial links (`d3.linkRadial`-style) connect each parent to its children. The outermost leaves—edge and access routers—are the attachment points for inter-POP links.

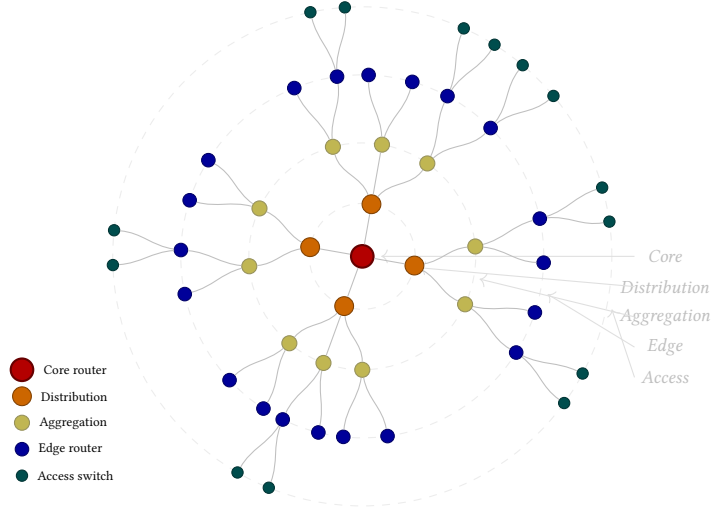


Figure 22. Detailed radial tree within a single POP bubble. The core router (red) is the root; distribution switches (orange) branch at depth 1; aggregation switches (yellow) at depth 2; edge routers (blue) at depth 3; access switches (teal) at depth 4. Angular span is proportional to subtree size—the upper-right branch (3 aggregation nodes) fans wider than the sparser lower-right branch (2 nodes). Depth 4 leaves are shown on selected branches to indicate that the tree extends to arbitrary depth. In the geographic starburst layout, the entire tree rotates so that outward-facing leaves align with inter-POP bearings.

Figure 23 shows two adjacent POPs whose radial trees have been *rotated* so that inter-POP edge routers face their remote peer. The algorithm places peering leaf nodes on the facing side of each tree, so inter-POP links travel near-straight across the gap without crossing any interior branches. The left POP illustrates a *multi-core* topology: two core routers share the root and each anchors its own subtree, with the peering-heavy subtree oriented toward the neighbour.

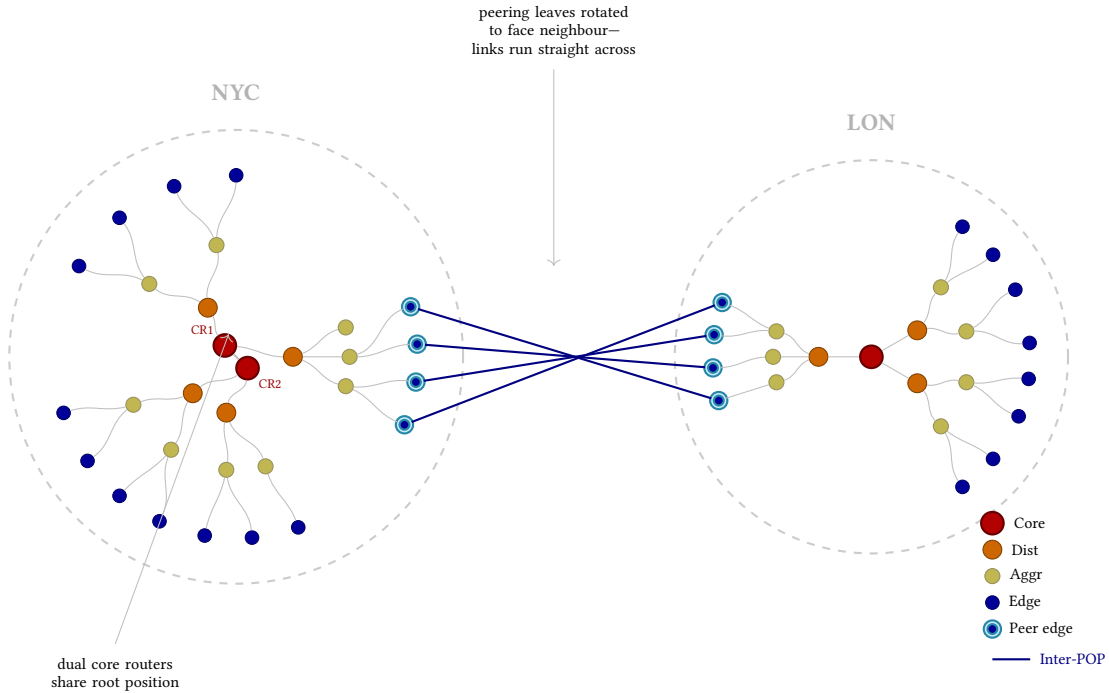


Figure 23. Peer-aware radial tree placement. The NYC POP (left, dual-core) and LON POP (right, single-core) have their radial trees rotated so that subtrees containing peering edge routers (cyan highlight) face the neighbour POP. The four inter-POP links run near-straight across the gap with no edge crossings—the algorithm repositions the nodes themselves rather than routing links around obstacles. Non-peering branches fill the remaining angular span on the far side of each bubble.

7.5 Isometric Multi-Layer Layout

Each layer runs an independent force-directed sub-layout. Results are projected isometrically with rotation angle $\phi = \pi/10$ (18°) and configurable layer spacing (default: 350 scene units). Layer planes render as filled SVG polygons. Edge labels are suppressed when edge length falls below 40 scene units.

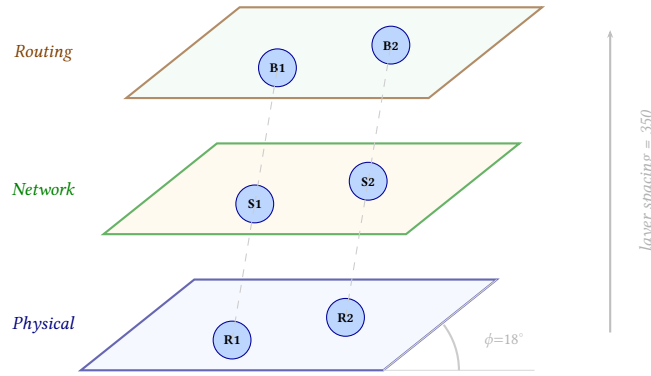


Figure 24. Isometric projection geometry. Each protocol layer runs an independent 2D layout, then all layers are projected with rotation angle $\phi = 18^\circ$. Inter-layer edges (dashed) connect corresponding entities across planes. Layer planes render as filled SVG polygons with configurable opacity.

7.6 Orthogonal Layout

Assigns node positions to a grid and routes all edges as sequences of horizontal and vertical segments with configurable corner radius. A sweep-line algorithm minimizes total edge length while maintaining minimum channel spacing between parallel routes. Suitable for standards-compliant engineering documentation.

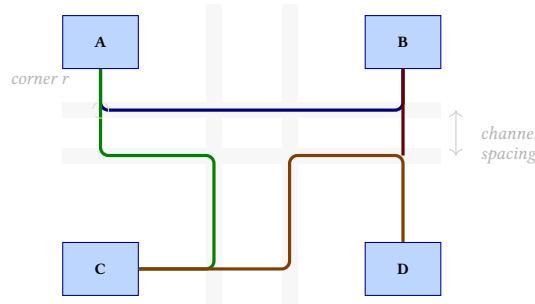


Figure 25. Orthogonal layout with channel routing. Nodes are placed on a grid; all edges are routed as horizontal/vertical segments through dedicated routing channels. A sweep-line algorithm assigns channels to minimize total edge length while enforcing minimum channel spacing. Configurable corner radius rounds the bends.

7.7 Subway Layout

Transit-map style: edges constrained to 0° , 45° , and 90° ; nodes snapped to a regular grid. The engine uses a three-phase pipeline:

1. **Force-directed seed.** A standard force-directed layout produces initial node positions that preserve the graph's topology.
2. **Grid snapping with collision avoidance.** Each node position is rounded to the nearest grid point: $\hat{x} = \lfloor x/g + 0.5 \rfloor \cdot g$, where g is the `grid_spacing` parameter (default 120 px). When two nodes snap to the same cell, a spiral search displaces the later node to the nearest unoccupied cell.
3. **Octilinear routing.** Each edge is routed using at most two segments. Given source (x_1, y_1) and target (x_2, y_2) , let $d = \min(|\Delta x|, |\Delta y|)$. The algorithm first traces a 45° diagonal of length d , then a straight segment to the target, producing at most two bends. If the edge is already axis-aligned or a perfect diagonal, it routes as a single segment.

Eight transit-line colours are assigned cyclically to edges: red #E32636, blue #003399, green #00985F, yellow #FFC82E, purple #951169, orange #F39200, cyan #00A1DE, grey #898D8D. Edges are styled with stroke width 8 px to evoke physical transit maps.

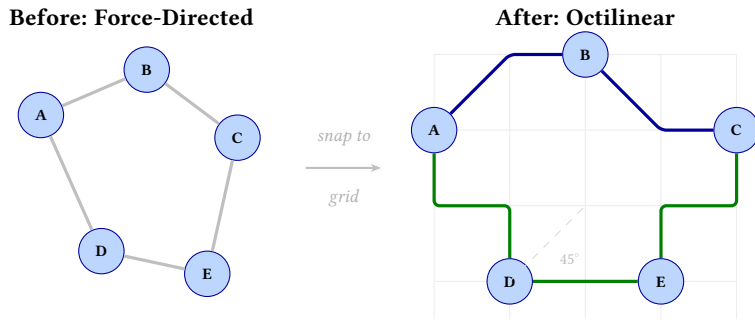


Figure 26. Subway layout: force-directed positions (left) are snapped to an octilinear grid (right). All edges use only 0°, 45°, and 90° segments with at most two bends per edge.

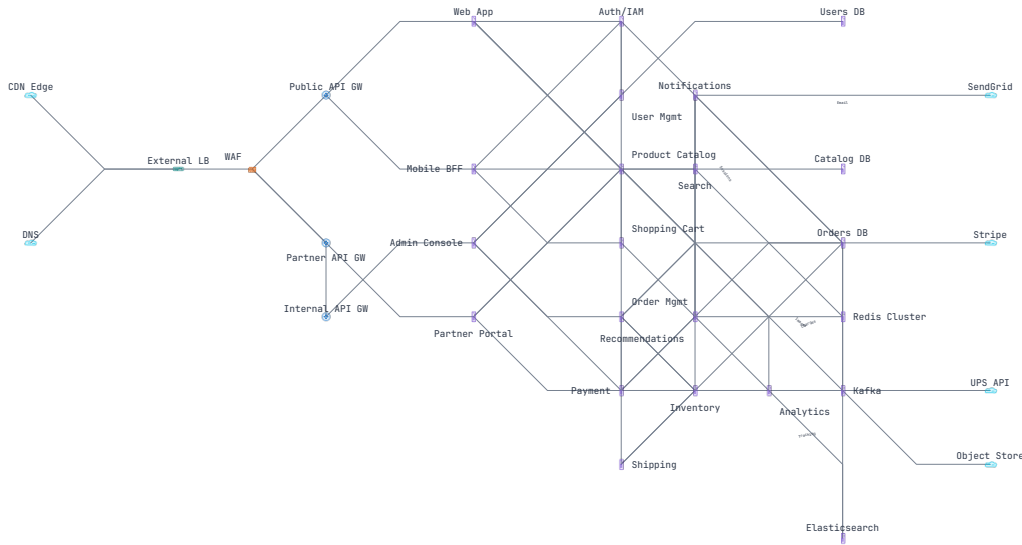


Figure 27. Subway layout applied to a 34-node microservices platform architecture. Service chains flow left-to-right through ingress, API gateways, business logic, and data tiers. Octilinear routing with transit-line colouring produces a readable schematic despite 55+ edges.

7.8 Radial Tree, Tree, and Composite

Radial tree places a root node at the center with children radiating outward in concentric rings; ring radii scale with subtree size to prevent overlap.

Tree layout produces classic hierarchical diagrams via a four-step algorithm:

1. **Root identification.** Nodes with in-degree 0 become roots. If none exist (the graph has a cycle), the first node is chosen as a fallback. Disconnected components yield separate roots.
2. **BFS tree building.** A breadth-first traversal from each root assigns every reachable node a depth and parent, producing a forest of `TreeNode` structures. Back-edges are ignored.
3. **Bottom-up width calculation.** A reverse pass accumulates subtree widths: leaves have width `sibling_spacing` (default 80 px); internal nodes sum the widths of their children.
4. **Top-down coordinate assignment.** Each root is allocated horizontal space proportional to its subtree width. Children are distributed within their parent's allocation, and the depth dimension is scaled by `level_spacing` (default 150 px).

The four orientation modes apply a final coordinate transform: *TopDown* (x, d), *BottomUp* ($x, -d$), *Left-Right* (d, x), *RightLeft* ($-d, x$), where x is the cross-axis position and $d = \text{depth} \times \text{level_spacing}$.

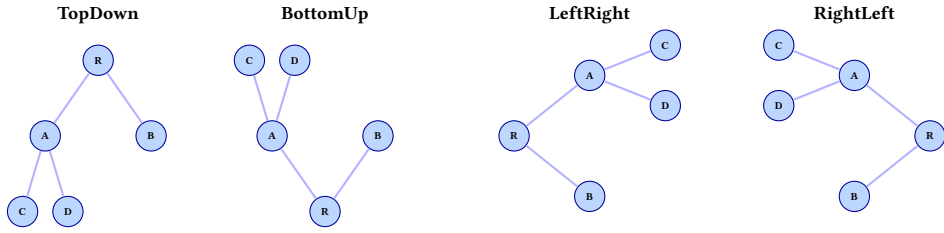


Figure 28. Tree layout orientation modes. The same tree rendered in all four orientations: *TopDown* (root at top), *BottomUp* (root at bottom), *LeftRight* (root at left), and *RightLeft* (root at right). Parameters: `level_spacing=150`, `sibling_spacing=80`.

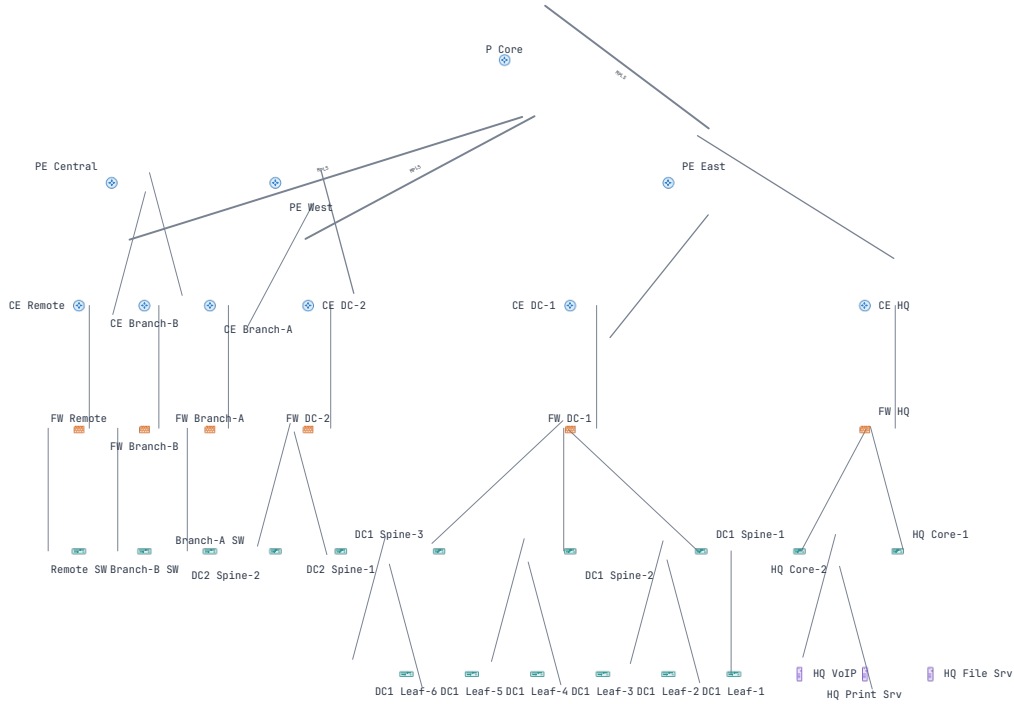


Figure 29. Tree layout rendering a 36-node MPLS VPN service hierarchy spanning six levels: P-core → PE routers → CE routers → firewalls → spine switches → leaf switches. Asymmetric subtrees (DC1 has six leaves; Branch-B has one switch) demonstrate the algorithm’s proportional width allocation.

Composite layout assigns different layout algorithms to different sub-graphs within a single canvas, using a parent-level layout to position groups relative to each other. Each sub-graph is abstracted into a *macro-node* whose radius is sized logarithmically:

$$r = \min(\text{BASE} + \ln(n + 1) \cdot \text{SCALE}, \text{MAX}) \quad (6)$$

with constants `BASE = 40`, `SCALE = 15`, `MAX = 200` (all in scene units). The logarithmic growth provides diminishing visual returns for larger groups: adding 90 nodes to a 10-node group increases the radius by ~35 px, while adding 90 nodes to a 1000-node group adds only ~1.3 px.

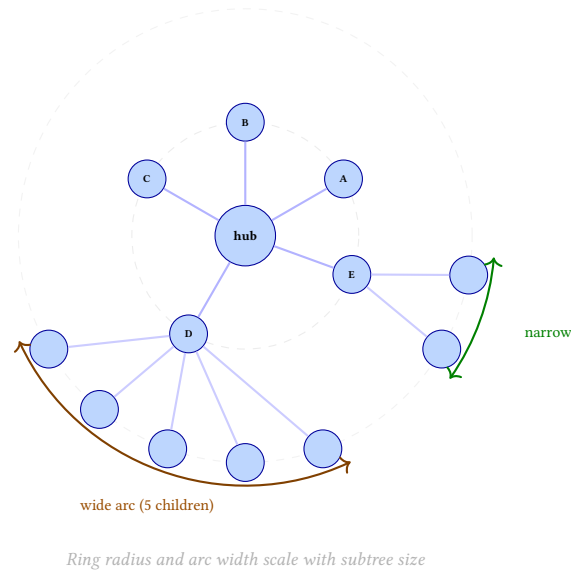


Figure 30. Radial tree layout. The hub sits at center; children radiate outward in concentric rings. Each subtree’s angular allocation is proportional to its descendant count: D’s 5-child subtree gets a wide arc; E’s 2-child subtree gets a narrow arc. This prevents overlap without wasting space.

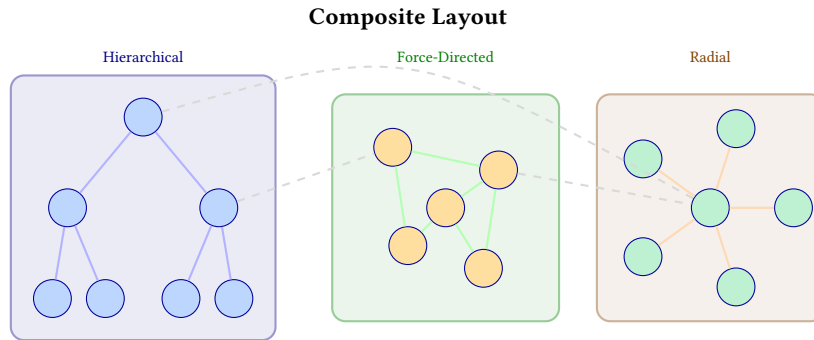


Figure 31. Composite layout: three sub-graphs use different algorithms (hierarchical, force-directed, radial) within a single canvas. A parent-level layout positions the groups relative to each other; inter-group edges (dashed) connect across boundaries.

7.9 Circular Layout

Nodes are placed at equal angular intervals on a circle of configurable radius. Edges render as *chord* curves through the circle interior using one of three modes: **Straight** (direct lines through center), **Quadratic** (default; quadratic Bézier with a single control point offset toward the center by a curvature factor), or **Cubic** (cubic Bézier for smoother long-distance arcs).

Group-aware *sector distribution* partitions the circle into angular sectors—one per group—with configurable gap degrees between sectors. Three distribution strategies are available: **Equal** (uniform arc per group), **Proportional** ($\text{arc} \propto \text{member count}$), and **ProportionalWithMin** (proportional with a minimum arc guarantee to prevent small groups from collapsing to a sliver). Within each sector, nodes are sorted by adjacency using a BFS traversal to minimize chord crossings.

Labels are placed *radially*: text is rotated to align with the radius at each node’s angular position, with left-hemisphere labels flipped 180° to prevent upside-down text.

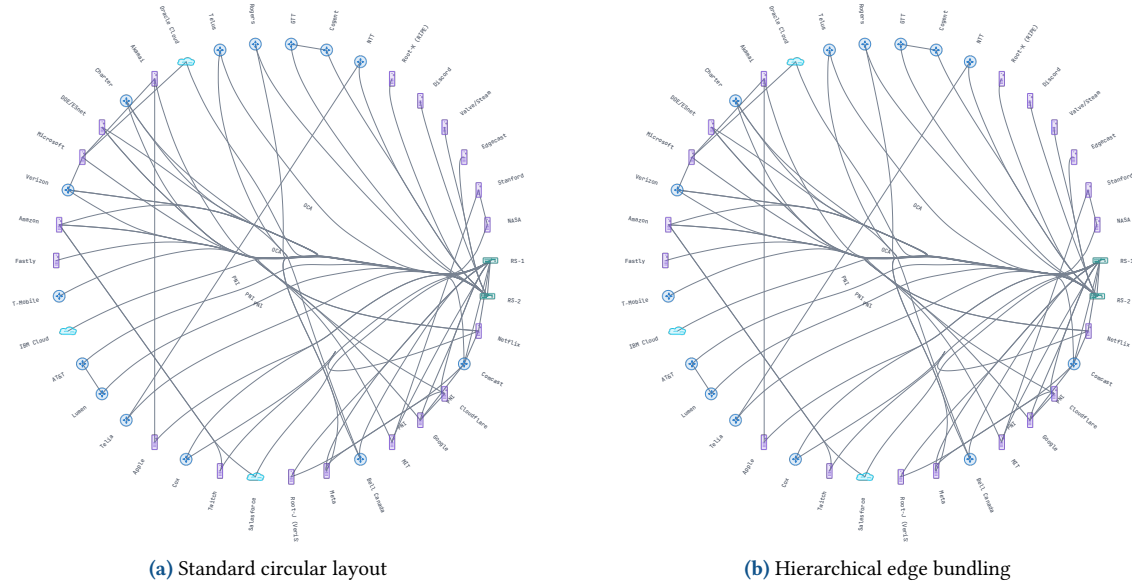


Figure 32. Circular layout of a 40-member internet exchange point. Nine sectors group participants by role. **(a)** Standard edge routing with straight-line edges. **(b)** Hierarchical edge bundling (Holten 2006): edges route through sector centroids via Catmull-Rom splines, coalescing cross-sector peering into visible corridors while intra-sector connections remain distinct.

8 Theme Gallery

NetVis ships seven built-in themes. Each theme defines a complete visual vocabulary: node fill and stroke colors, edge stroke and gradient defaults, label color and font family, background fill, glow and shadow intensity, and group boundary style.

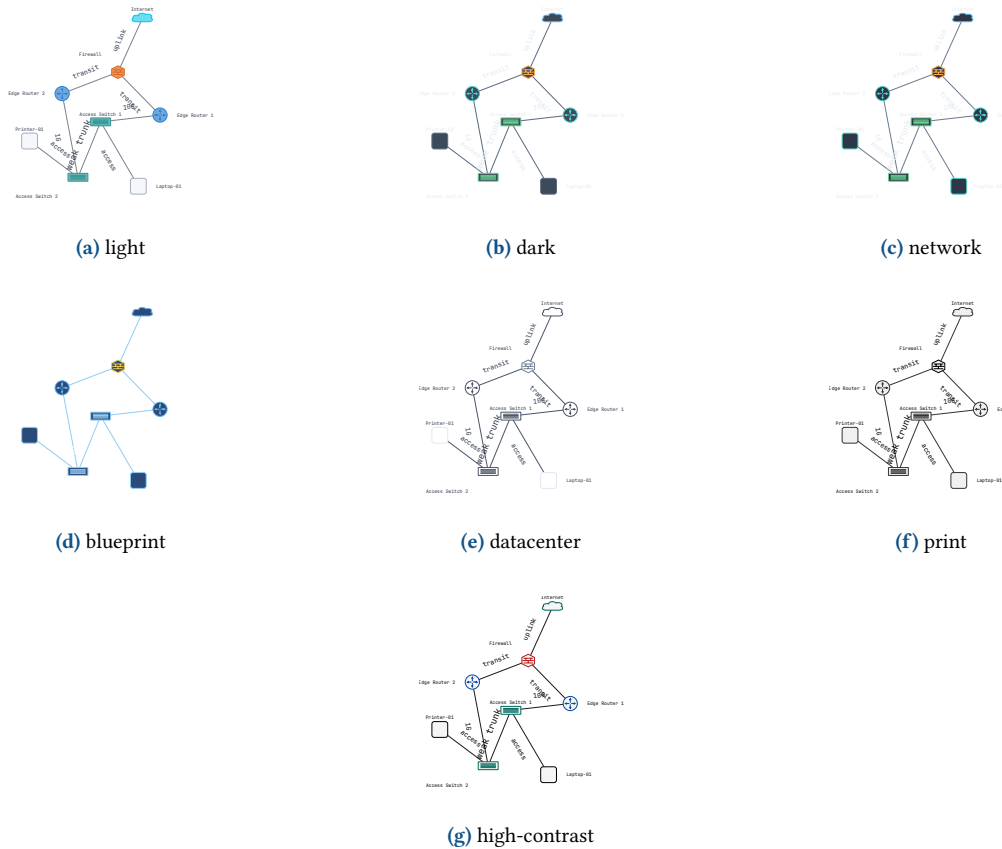


Figure 33. All 7 built-in themes applied to the same 8-node campus topology. Theme is the only YAML difference between these diagrams. The blueprint theme uses a technical-ink style optimized for geometric layouts; high-contrast meets WCAG AA contrast ratios throughout.

9 Parallel Edge Rendering

Network topologies frequently contain multiple physical links between the same pair of devices—redundant uplinks, **LAG** bundles, and NIC-teaming configurations. General-purpose graph tools either render these as overlapping lines (unreadable) or collapse them into a single edge (losing information).

NetVis provides five rendering modes, selectable via `parallel_mode`:

Mode	Behavior
offset (default)	Each link drawn as a visually separate parallel line with perpendicular offset. Four edges produce offsets of $-6, -2, +2, +6$ pixels.
bundled	All links collapsed into one thick line (width scales with $\sqrt{\text{count}}$) with aggregate multiplicity label (e.g., “4x 10G (40G)”).
bonding_icon	Single line with a bracket or chain-link icon at the midpoint.
stacked	Lines drawn very close together (1–2 px offset), creating a thick-line effect without merging strokes.
angled	Curves with different angles/curvatures; distinguishes many parallel links without excessive width.

Table 2. Parallel edge rendering modes.

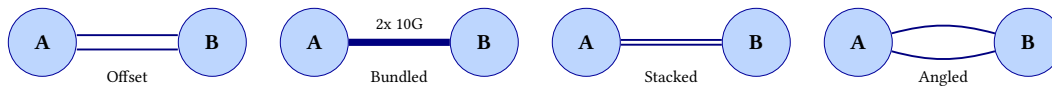


Figure 34. Four parallel edge rendering modes (BondingIcon omitted). Each mode trades visual fidelity for compactness. Offset is clearest for redundant-path diagrams; Bundled is most compact for **LAG** groups.

9.1 Parallel Edge Mode Decision Tree

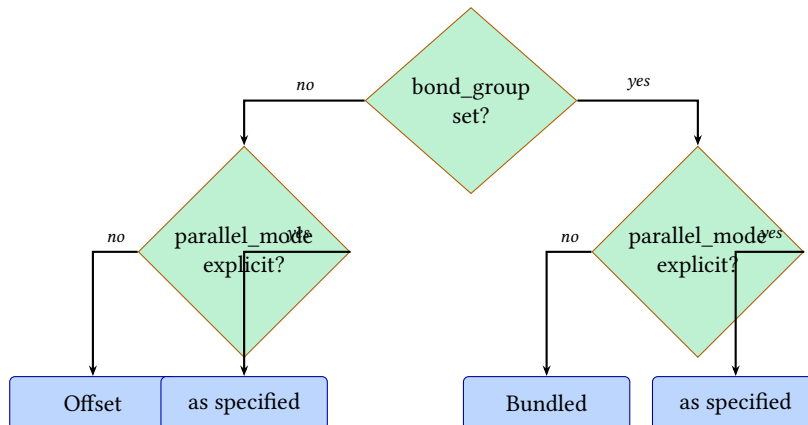


Figure 35. Parallel edge mode selection logic. Bond groups default to Bundled; non-bonded parallel edges default to Offset. An explicit `parallel_mode` always overrides the default.

9.2 Automatic Bandwidth Aggregation

When edges share a `bond_group` identifier, the system automatically parses per-link bandwidth labels and computes aggregate totals using `parse_bandwidth_gbps()`. In Bundled mode, only the total is shown (e.g., “40G”); in other bonded modes, the detail is shown (e.g., “4x 10G (40G)”).

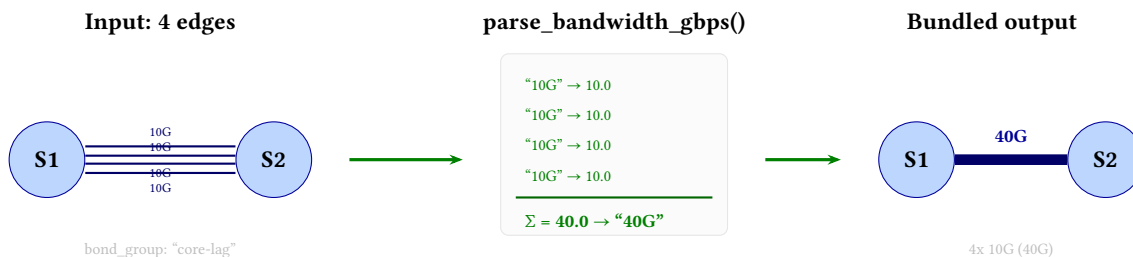


Figure 36. Automatic bandwidth aggregation. Four edges sharing `bond_group`: “core-lag” have their bandwidth labels parsed by `parse_bandwidth_gbps()`, which recognizes SI suffixes (M/G/T). The totals are summed and formatted: Bundled mode shows only “40G”; other modes show “4x 10G (40G)”.

10 Node Shapes and Device Icons

NetVis supports a rich library of geometric shapes and network device icons for node rendering.

10.1 Available Shapes and Icons

Name	Category	Description
circle	Geometric	Default for generic nodes
rectangle	Geometric	Axis-aligned; useful for hosts and servers
ellipse	Geometric	Horizontal ellipse for cloud/WAN nodes
rounded_rectangle	Geometric	Rectangle with configurable corner radius
diamond	Geometric	Rotated square; used for firewalls
triangle	Geometric	Upward-pointing triangle
hexagon	Geometric	Six-sided; useful for AS/BGP speakers

Table 3. Built-in geometric node shapes. Unknown node_type values render as circles.

Name	Category	Description
router	Icon	Modern circular icon with inward routing arrows
switch	Icon	Flat rectangular icon with horizontal routing arrows and RJ45 ports
l3_switch	Icon	Switch icon with additional vertical routing arrows
firewall	Icon	Shield or brick-wall icon with offset brick texture
server	Icon	Vertical rack-blade rectangle with drive bay vents
cloud	Icon	Cumulus cloud outline with internal volume lines
load_balancer	Icon	Funnel or balance icon with distribution arrows
database	Icon	Stacked-cylinder icon indicating data tiers
endpoint	Icon	Laptop/workstation or monitor icon
wireless_ap	Icon	Ceiling-mounted access point with signal waves
satellite	Icon	Parabolic dish with receiver arm
mobile_tower	Icon	Lattice cellular tower with fanned antenna arrays
iot_device	Icon	Small sensor enclosure with antenna and vents
vpn	Icon	Padlock symbol representing secure tunnel gateways
internet	Icon	Globe with detailed meridians and latitudes
storage	Icon	Stacked 3D disk drawers
user	Icon	Human silhouette
industrial	Icon	Factory outline with sawtooth roof and smoke
security_camera	Icon	Modern bullet-style CCTV unit with mount

Table 4. Built-in device icons. All icons support multi-part semantic rendering (e.g., dynamic lighting, glowing LEDs).

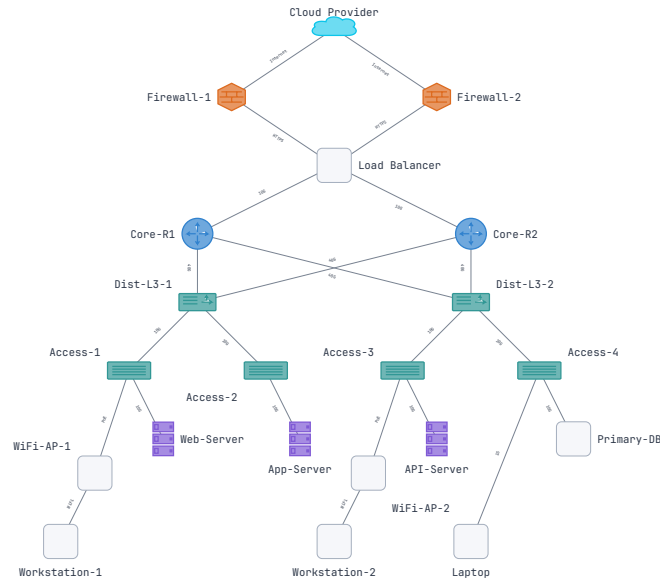


Figure 37. Full device icon library. Each icon shape adapts its size, fill, and stroke to the active theme.

10.2 Multi-Part Semantic Rendering

To bridge the gap between simple generative output and professional, commercial-grade architectural diagrams (e.g., Visio or Lucidchart stencils), NetVis implements a multi-part semantic rendering system for device icons. Rather than rendering a flat SVG shape, each icon is decomposed into semantic layers:

- **Body:** The primary structural outline, utilizing the main theme fill and stroke.
- **BodyTop and BodySide:** For 3D isometric icons (e.g., servers, storage, switches), the rendering engine intercepts these facets and applies simulated lighting. A semi-transparent white overlay (`rgba(255,255,255,0.15)`) is applied to top faces for highlights, while a semi-transparent black overlay (`rgba(0,0,0,0.2)`) is applied to side faces for shadows. This true isometric lighting adapts flawlessly to any underlying theme color.
- **Detail:** Internal features such as RJ45 ports, drive bays, or brick patterns are drawn with a subtle, semi-transparent outline (e.g., 60% opacity), ensuring they provide texture without visual clutter.
- **Accent:** Emphasized lines such as routing arrows or signal waves that receive thicker, bolder strokes.
- **Led:** Status indicators mapped directly to node states. The engine parses the status or state attribute of a node and emits data-driven glowing LEDs (e.g., bright red for critical, green for active, or yellow for warning).

Coupled with injected premium SVG filters—such as `icon-shadow` for a subtle drop shadow granting physical depth, and `icon-glow` utilizing `feGaussianBlur` for light bloom on LEDs—NetVis yields exceptionally polished, data-driven visual components.

11 Interface Rendering

NetVis treats network interfaces as first-class visual objects. Each node can expose its interfaces with configurable visual modes and status indicators.

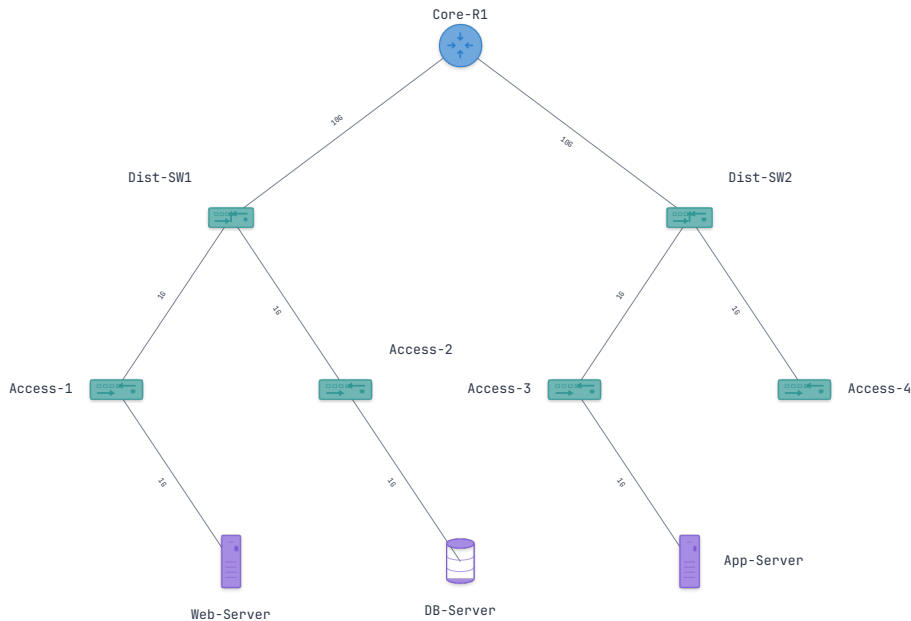


Figure 38. Interface rendering modes: Dot (compact), Port (stub with label), and Detailed (full interface card with status). Status colors are Up (active), Down (gray), AdminDown (strikethrough), and Errors (amber).

Interfaces are configured per-node in YAML (see §40).

The status field accepts up, down, admin_down, or errors. The mode field controls visual detail: dot is compact (small circle on node boundary); port adds a rectangular stub with label; detailed shows the full interface card with speed and operational state. The marker field (none, dot, square, diamond) controls the endpoint symbol on the edge.

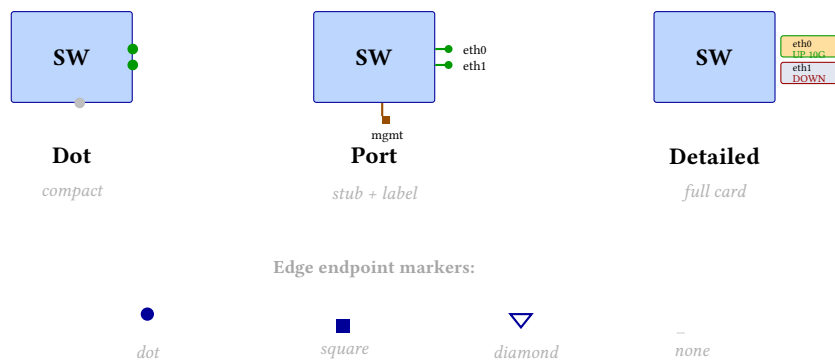


Figure 39. Interface rendering modes. **Dot:** compact circles on the node boundary. **Port:** rectangular stubs with labels. **Detailed:** full interface cards with status and speed. Four marker types control the edge endpoint symbol.

12 Edge Gradients and Path Emphasis

12.1 Edge Gradients

Directional stroke gradients encode flow direction or metric values along an edge. A gradient edge transitions smoothly from a *start_color* at the source to an *end_color* at the target, implemented as SVG `<linearGradient>` elements aligned to the edge direction:

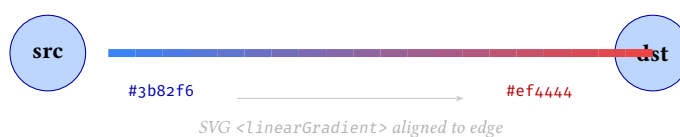


Figure 40. Edge gradient rendering. A directional gradient transitions from *start_color* at the source to *end_color* at the target, implemented as an SVG linear gradient rotated to match the edge angle.

12.2 Path Emphasis

Path emphasis highlights a named subset of edges as a forwarding path. Two emphasis levels (**Primary** for dominant, **Secondary** for backup) combine with three fanout modes (**Fixed**, **Scaled**, **Hybrid**) that control how emphasis interacts with parallel edge groups.

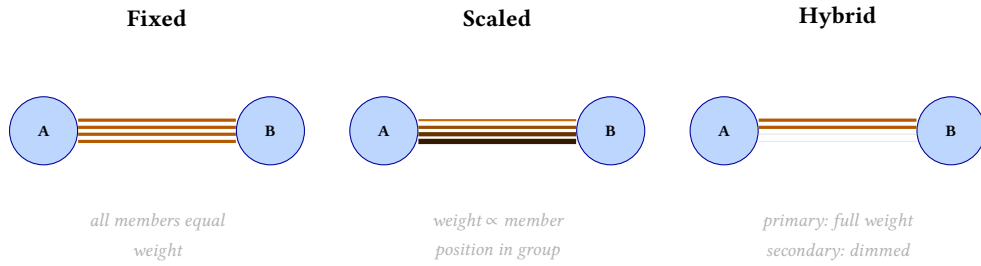


Figure 41. Path emphasis fanout modes on a 4-member LAG. **Fixed:** all members receive equal emphasis weight. **Scaled:** emphasis weight is distributed proportionally across members. **Hybrid:** primary members get full emphasis; secondary members are dimmed.

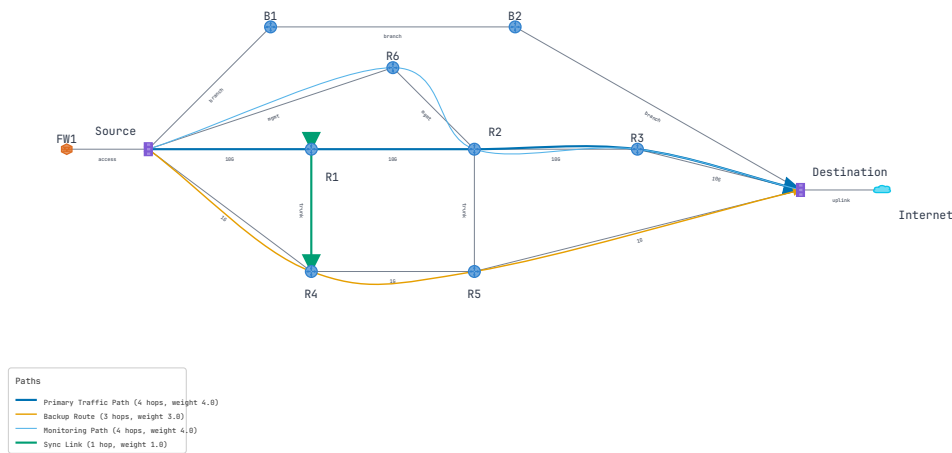


Figure 42. Path emphasis: primary paths in full-weight highlight; secondary in reduced emphasis. Fanout mode *Scaled* is shown on a LAG bundle where emphasis weight scales with member count.

12.3 Variable-Width (Tapered) Edges

Edges can be rendered with linearly varying width to indicate flow direction or asymmetric capacity. The `variable_width_outline` function samples 20 evenly-spaced arc-length positions along the edge path, computing the tangent and normal at each sample. At sample i , the half-width is $w_i/2$ where $w_i = w_{\text{start}} + (w_{\text{end}} - w_{\text{start}}) \cdot i/20$. The algorithm offsets each sample by $\pm w_i/2$ along the normal to produce a closed polygon (43 path elements: forward pass, reverse pass, close), which is filled rather than stroked.

13 Metric Encoding

NetVis maps arbitrary node attributes to visual channels (color, size, opacity), transforming static topology diagrams into live network dashboards. Six sequential palettes are built in: blues, greens, oranges, reds, purples, greys. All are perceptually uniform and work on both light and dark themes.

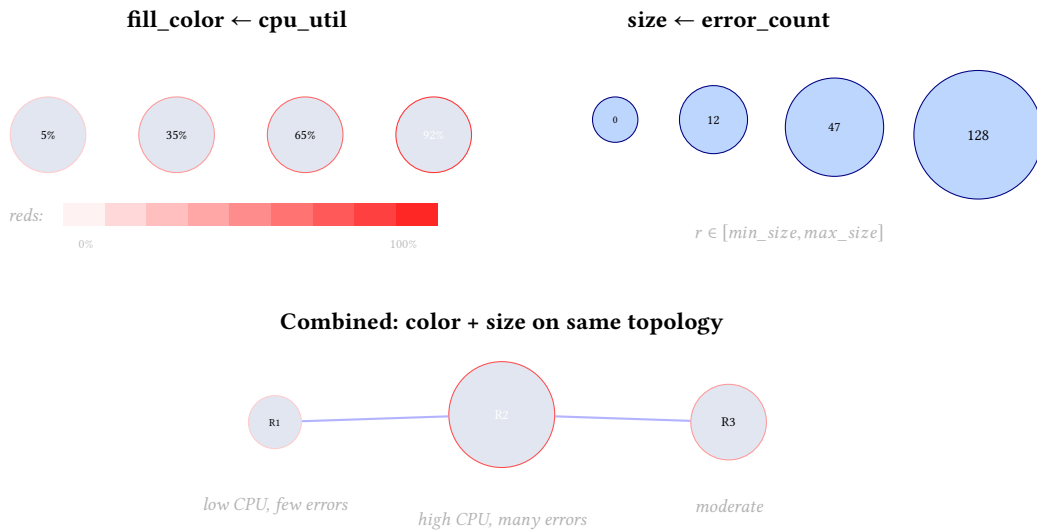


Figure 43. Metric encoding maps data to visual channels. **Top left:** CPU utilization drives node fill color through the “reds” palette. **Top right:** error count drives node radius. **Bottom:** both channels applied simultaneously—at a glance, R2 is the hotspot (large, dark red) while R1 is healthy (small, light).

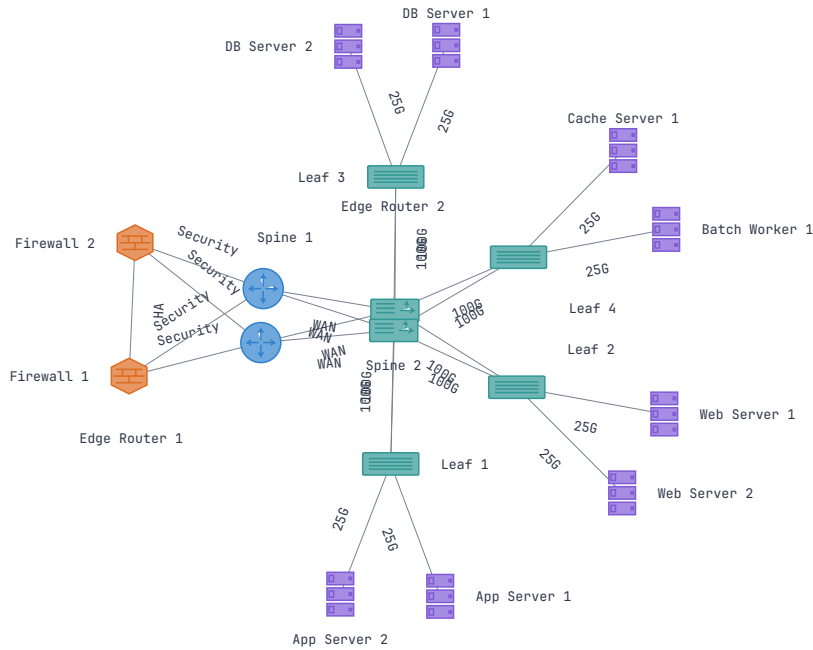


Figure 44. Metric encoding: node fill encodes CPU utilization (reds); node size encodes interface error count; edge stroke encodes link utilization (oranges).

13.1 Weathermap Styling

For NOC-style dashboards, NetVis provides a *weathermap* mode that encodes link utilization through both color and width simultaneously. A 5-stop continuous color ramp maps utilization $u \in [0, 1]$:

Utilization	Color	Hex
0%	green	#22C55E
25%	yellow	#EAB308
50%	orange	#F97316
75%	red	#EF4444
100%	dark red	#991B1B

Intermediate values are linearly interpolated in sRGB between adjacent stops.

Edge width is scaled through a configurable transfer function (`MetricScaleMode`): *Linear* (direct mapping), *Sqrt* (default; emphasizes low utilization), or *Log* ($\ln(1 + 9u)/\ln 10$, compresses high values). Width is interpolated between `min_width` (default 1.5 px) and `max_width` (default 8 px).

Labels are formatted using a template string with placeholders `{throughput}`, `{capacity}`, `{utilization}`, `{latency}`, `{loss}`. The default format is "`{throughput}/{capacity} ({utilization}%)`". Throughput and capacity values use SI prefixes (K/M/G/T).

14 Label Placement

14.1 Problem and Algorithm Choice

Placing text labels near their owner without overlapping other labels, nodes, or edges is a variant of the NP-hard point-feature label placement (PFLP) problem [13]. Three approaches were considered: (a) no collision avoidance (Graphviz approach); (b) $O(n^2)$ greedy; (c) R^* -tree-accelerated greedy with $O(\log n)$ collision queries.

NetVis uses option (c). The R^* -tree [14] build cost (~ 5 ms) is recouped after approximately 20 collision queries, which occurs by ~ 50 labels.

14.2 Two-Phase Greedy Algorithm

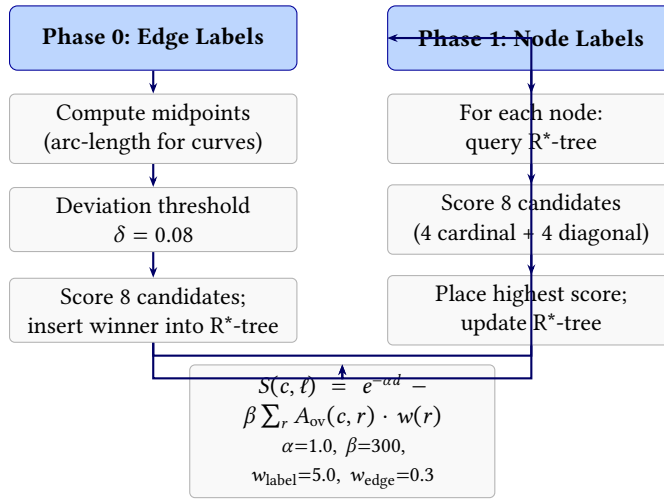


Figure 45. Label placement algorithm. Phase 0 places edge labels first (more constrained by midpoint proximity). Phase 1 places node labels around already-occupied regions. The scoring function penalizes both distance from the owner and overlap with existing geometry.

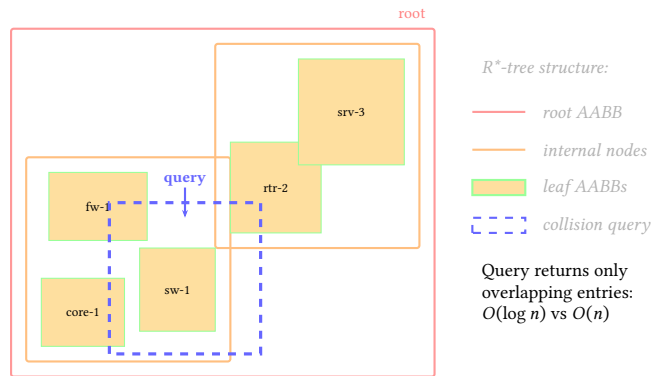


Figure 46. R^* -tree spatial index for label collision detection. Occupied regions (node shapes, existing labels, edge exclusion zones) are inserted as axis-aligned bounding boxes. A candidate label position is tested by querying the tree for intersections— $O(\log n)$ per query vs. $O(n)$ brute force.

Edge exclusion zones: Curved/bundled edges can produce 47,000+ segment AABBs. Per-segment exclusion zone registration is skipped for non-straight edges; only a single bounding-box exclusion is registered per curved path.

Arc-length label sampling. For curved edges (geographic great-circle arcs, bundled edges), the label midpoint is computed by `arc_length_sample`: the `BezPath` is flattened into a polyline whose cumulative segment lengths are accumulated, then a binary search locates the point at the target arc-length fraction (typically

0.5). A *curve deviation* test determines whether the curve midpoint should replace the chord midpoint: if $\delta/\ell_{\text{chord}} > 0.08$ (where δ is the Euclidean distance between curve and chord midpoints), the curve midpoint is used. This prevents bundled edges—whose midpoints converge to a shared point—from clustering labels at the convergence, while still using arc-length midpoints for geographic arcs whose curvature is genuine.

Post-bundling repositioning. After edge bundling modifies edge geometry, `reposition_edge_labels()` selectively re-places only those edge labels whose curvature changed significantly (deviation exceeds the 0.08 threshold). All other labels keep their original positions from the initial placement pass.

Smart corner labels. For Manhattan-routed (orthogonal) edges, `find_orthogonal_elbows` detects 90-degree bends in the edge path and adds the elbow points as high-priority label candidate positions. This places labels at corners where they are naturally close to the edge without obscuring straight segments, improving readability for orthogonally routed topologies.

Font size optimization. Edge labels are capped at 70% of the edge length to avoid overlapping neighbouring nodes. If the estimated label width exceeds $0.7 \times \ell_{\text{edge}}$, the font size is scaled proportionally, with a floor of 9 px to maintain readability. Edges shorter than 40 scene units suppress labels entirely.

14.3 Candidate Scoring Visualization

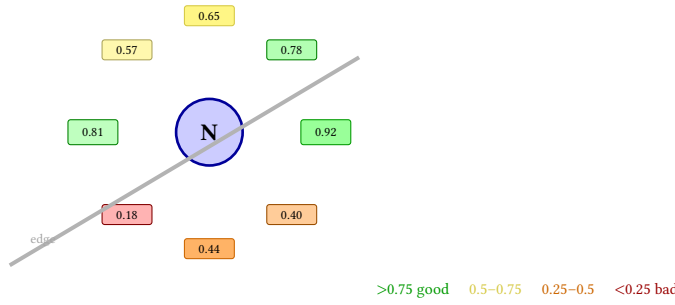


Figure 47. Scoring function applied to 8 candidate label positions around node N. Position scores (0–1) are color-coded from green (good) to red (poor). The right position wins (0.92); the lower-left loses because a diagonal edge passes through it (score 0.18).

14.4 Performance

Nodes	Time	Overlaps	Avg. Distance
100	<1 ms	0	8.2 pt
500	12 ms	0	9.1 pt
1000	48 ms	2	10.3 pt
5000	210 ms	18	12.7 pt

Table 5. Label placement performance on spine-leaf topologies (R*-tree acceleration via `rstar` [15]).

15 Edge Bundling

Dense graphs produce visual clutter from crossing edges. Edge bundling groups spatially compatible edges into bundles, reducing clutter. NetVis implements a variant of Holten and Van Wijk’s Force-Directed Edge Bundling (FDEB) [16] with modifications to the force model and post-processing pipeline. This section gives the full mathematical treatment.

15.1 Compatibility Scoring

Two edges should attract only if they are geometrically similar. FDEB defines four compatibility metrics, each normalised to $[0, 1]$.

Angle compatibility. Let $\vec{v}_P$ and $\vec{v}_Q$ be the direction vectors of edges P and Q . Angle compatibility is the absolute cosine similarity:

$$C_a(P, Q) = |\cos \theta| = \frac{|\vec{v}_P \cdot \vec{v}_Q|}{|\vec{v}_P| |\vec{v}_Q|} \quad (7)$$

The absolute value makes compatibility direction-agnostic: anti-parallel edges are as compatible as parallel ones.

Scale compatibility. Let ℓ_P, ℓ_Q be edge lengths, $\bar{\ell} = (\ell_P + \ell_Q)/2$, $\ell_{\min} = \min(\ell_P, \ell_Q)$, and $\ell_{\max} = \max(\ell_P, \ell_Q)$:

$$C_s(P, Q) = \frac{2}{\frac{\bar{\ell}}{\ell_{\min}} + \frac{\ell_{\max}}{\bar{\ell}}} \quad (8)$$

This harmonic-mean formulation penalises large length ratios: two edges of equal length score $C_s = 1$; a 10:1 ratio scores $C_s \approx 0.33$.

Position compatibility. Let m_P, m_Q be the midpoints of P and Q :

$$C_p(P, Q) = \frac{\bar{\ell}}{\bar{\ell} + \|m_P - m_Q\|} \quad (9)$$

Edges whose midpoints are far apart relative to their length receive low position compatibility.

Visibility compatibility. Let $V(P \rightarrow Q)$ be the fraction of edge P 's length that “sees” edge Q without occlusion (computed by projecting Q 's endpoints onto P 's line):

$$C_v(P, Q) = \min(V(P \rightarrow Q), V(Q \rightarrow P)) \quad (10)$$

The overall compatibility is the product:

$$C(P, Q) = C_a \cdot C_s \cdot C_p \cdot C_v \quad (11)$$

Edge pairs with $C(P, Q) < \tau$ (default $\tau = 0.6$) are excluded from the compatibility graph. Because each factor is at most 1 and the product amplifies differences, only genuinely similar edges survive the gate.

15.2 Force Model

Each edge e is represented by a polyline of interior *subdivision points* $p_1, \dots, p_m$; the endpoints $p_0 = s_e$ and $p_{m+1} = t_e$ remain fixed. Two forces act on each subdivision point.

Attraction. Subdivision point $p_k^{(i)}$ of edge e_i is attracted toward the corresponding point $p_k^{(j)}$ of each compatible edge e_j :

$$F_{\text{attr}}(k) = \frac{C(e_i, e_j) \cdot \kappa}{d_k / \bar{\ell} + 1} \quad (12)$$

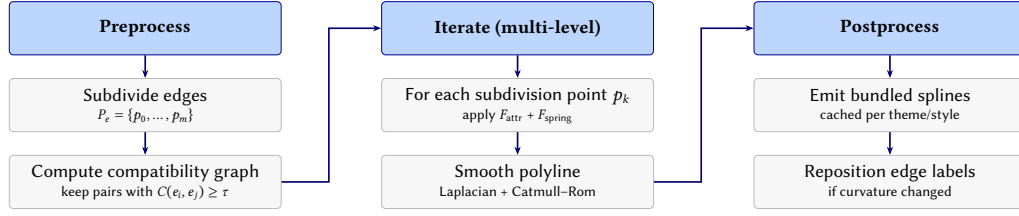
where $d_k = \|p_k^{(i)} - p_k^{(j)}\|$ is the inter-edge distance, $\bar{\ell}$ is the mean edge length across the graph, and $\kappa \in [0, 1]$ is the bundling strength parameter.

The $+1$ term in the denominator is a *singularity guard*. The original Holten & Van Wijk formulation uses a $1/d$ attraction that diverges as edges converge. An earlier implementation used $+0.1$, which bounded the force at $10C\kappa$ —still large enough to pull already-converged edges past each other, producing “hairball” artifacts. With $+1$, the maximum attraction at $d = 0$ is bounded by $C\kappa$, which is dominated by the spring force (see below).

Spring. A spring force restores each point toward the midpoint of its polyline neighbours, preserving the edge's identity:

$$F_{\text{spring}}(k) = 1.5 \kappa \cdot \left(\frac{p_{k-1} + p_{k+1}}{2} - p_k \right) \quad (13)$$

The coefficient 1.5 ensures that springs dominate attraction at convergence. When $d_k \rightarrow 0$, attraction is bounded by $C\kappa$ (Equation 12 with $d_k / \bar{\ell} \rightarrow 0$). Since $C < 1$ for all non-identical edges, the spring magnitude 1.5κ exceeds attraction, preventing edge collapse.



Key idea: move only subdivision points; endpoints stay fixed. Multi-level schedule: increase m each cycle; step size is δ ; bundling happens after initial label placement; edge labels may need a second pass.

Figure 48. FDEB solver pipeline. Compatibility graph construction gates which edges interact; the iterative solver updates only subdivision points; smoothing converts a polyline into a visually stable spline.

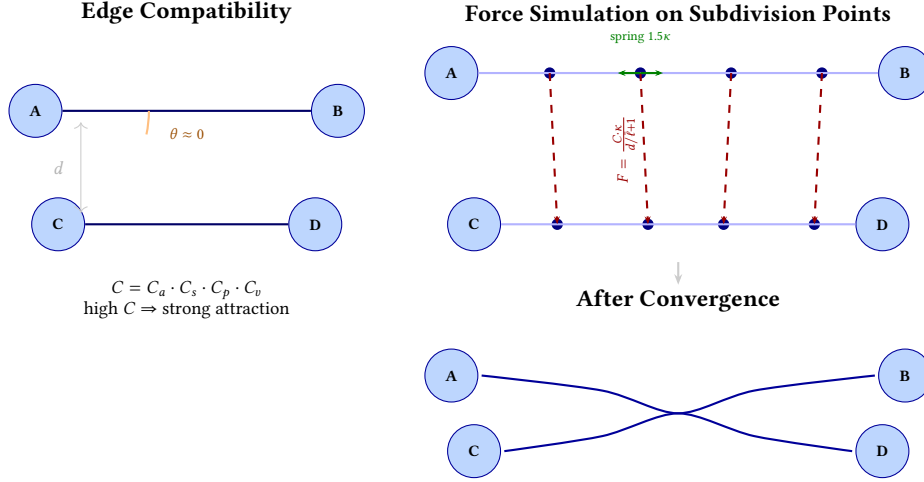


Figure 49. FDEB force-directed edge bundling. **Top left:** edge compatibility scoring—nearly parallel, co-located edges receive high compatibility C . **Top right:** subdivision points along each edge are subject to attraction forces (red, $F = C\kappa/(d/\bar{\ell}+1)$) toward corresponding points on compatible edges, and spring forces (green, 1.5κ) that preserve edge shape. **Bottom:** after convergence, compatible edges merge into a shared bundle through the midpoint, fanning out to their respective endpoints.

15.3 Multi-Level Schedule

The solver uses a multi-level scheme that progressively increases resolution. Starting with a single interior point per edge, each *cycle* runs I force iterations (default $I = 60$), then doubles the point count by inserting midpoints between consecutive points. After cycle c (0-indexed), there are 2^c interior points: 1, 2, 4, 8, 16, 32 for six cycles.

The step size for Euler integration is

$$\delta = \frac{s \cdot \bar{\ell}}{100} \quad (14)$$

where s is the configured step size (default $s = 0.1$) and $\bar{\ell}$ is the mean edge length. The step size is constant across cycles.

The final cycle does *not* subdivide after its force iterations. Subdivision inserts unsettled midpoints that would distort the converged polyline; skipping this avoids high-frequency noise in the output.

15.4 Post-Processing

Laplacian smoothing. After the force simulation completes, three passes of Laplacian smoothing remove residual high-frequency zigzag artifacts from discrete Euler integration:

$$p_k \leftarrow (1 - \alpha) p_k + \alpha \frac{p_{k-1} + p_{k+1}}{2} \quad (15)$$

with $\alpha = 0.5$. Three passes suffice: each pass halves the amplitude of the highest-frequency mode, so three passes reduce it to $\sim 12.5\%$ of the original.

Catmull–Rom to cubic Bézier conversion. The smoothed polyline is converted to a cubic Bézier spline using Catmull–Rom interpolation with tension $\tau = 0.4$. For consecutive points $p_{i-1}, p_i, p_{i+1}, p_{i+2}$, the Bézier control points for the segment $p_i \rightarrow p_{i+1}$ are:

$$c_1 = p_i + \frac{\tau}{3}(p_{i+1} - p_{i-1}) \quad (16)$$

$$c_2 = p_{i+1} - \frac{\tau}{3}(p_{i+2} - p_i) \quad (17)$$

Standard Catmull–Rom uses $\tau = 0.5$; the lower value 0.4 produces tighter curves that follow the settled polyline more closely, reducing overshoot at bundle convergence points.

15.5 Pipeline Integration

Note

Bundling runs *after* label placement. A `reposition_edge_labels()` pass re-replaces labels on edges whose curvature changed significantly post-bundling.

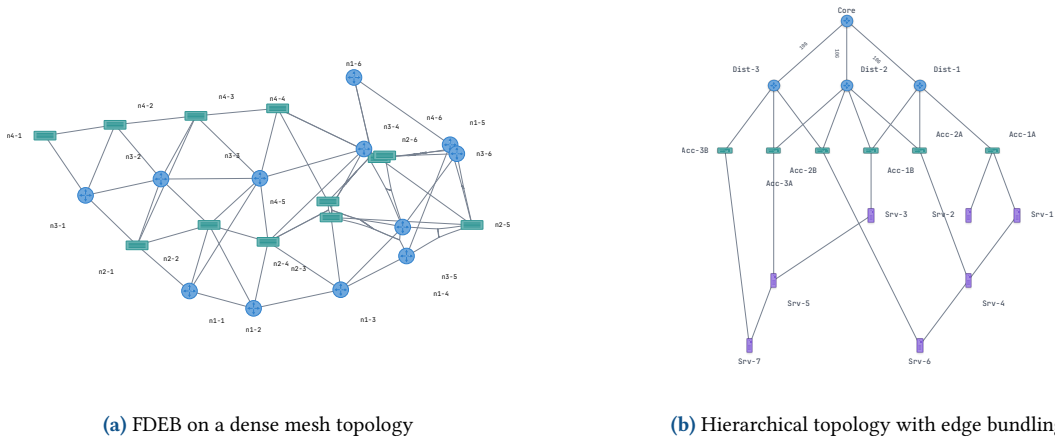


Figure 50. Edge bundling modes. Left: FDEB groups geometrically compatible edges. Right: hierarchical layout combined with edge bundling clarifies cross-level redundancy and server meshes.

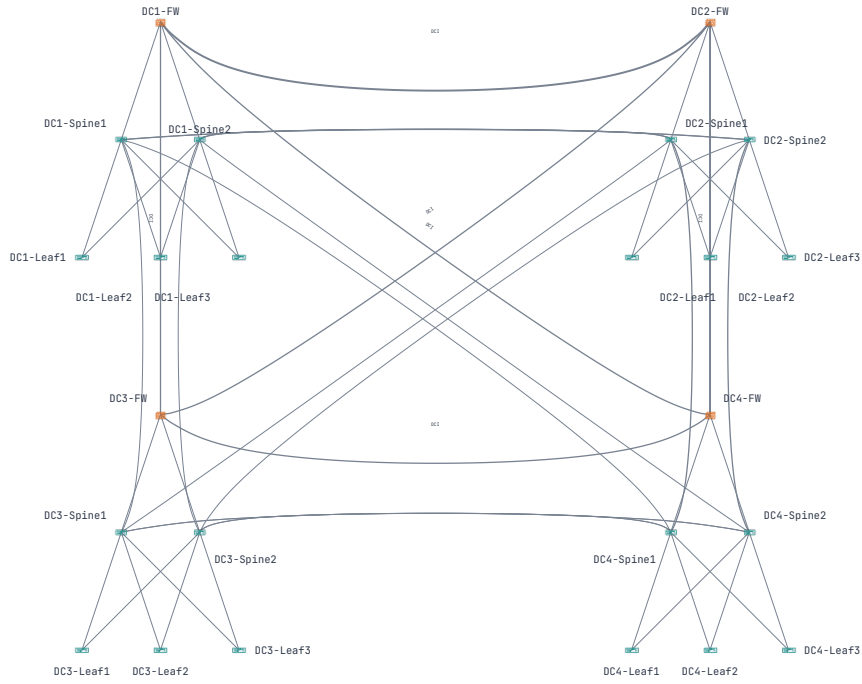


Figure 51. FDEB edge bundling on a 24-node, 64-edge multi-datacenter topology with four spine-leaf fabrics connected by full-mesh VXLAN EVPN peering. Without bundling, 16 cross-DC spine-to-spine links create dense visual clutter; bundled edges coalesce into clear inter-DC corridors while intra-DC spine-leaf connections remain distinct. Catmull-Rom spline conversion produces smooth curves at bundle convergence points.



Figure 52. FDEB bundling on a spine-leaf datacenter fabric. North-south spine-to-leaf links and east-west inter-pod connections coalesce into distinct bundled corridors, revealing the topology’s hierarchical structure more clearly than unbundled straight edges.

16 Edge Routing and Post-Layout Refiners

NetVis provides ten post-layout refiners, each implementing the `LayoutRefiner` trait. Refiners are composable and can be chained in any order (see §6 and Figure 15).

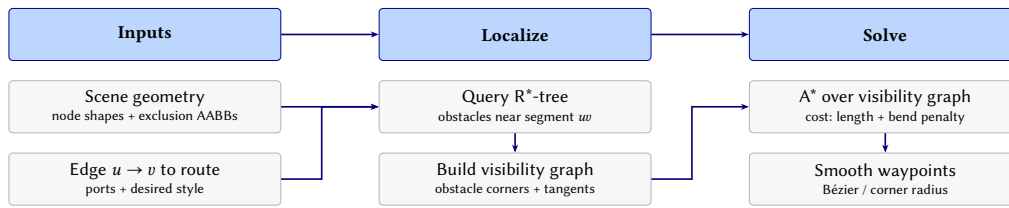
Refiner	Category	Description
SmartRouting	Routing	Bends edges around obstacles using R*-tree visibility graph + A* pathfinding; waypoints smoothed to Bézier splines (see Figure 54)
OrthogonalRouting	Routing	Converts edges to axis-aligned Manhattan paths with configurable corner radius
PortDocking	Routing	Adjusts endpoints to dock at specific interface positions, distributing connections evenly around the perimeter
BoundaryBundling	Routing	Routes edges through a shared crossing point at group boundaries, fanning out inside
Deoverlap	Spacing	Pushes overlapping nodes apart using repulsive spring force; configurable margin (default: 10 units)
EdgeClipping	Spacing	Clips edge endpoints to the exact node shape boundary for correct arrowhead placement

Table 6. Post-layout refiners: routing and spacing.

Refiner	Category	Description
Nudging	Spacing	Detects geometrically coincident straight-line edges via integer-rounded endpoint grouping; applies symmetric perpendicular offsets ($\pm n \cdot 4 \text{ px}$)
PortSpacing	Spacing	Redistributes edge connection points evenly around high-degree node perimeters (≥ 3 edges); sorts by exit angle, then reassigns at $2\pi/n$
SmartLabel	Labels	8-position snapping; second-pass improvement over initial greedy placement
LabelNudging	Labels	Iterative repulsive force between overlapping label bounding boxes; converges in 10–30 iterations
Starburst	Groups	Positions nodes radially within containment groups; best for AS-level diagrams

Table 7. Post-layout refiners: spacing (continued), labels, and groups.

16.1 Smart Routing: Visibility Graph



R-tree keeps per-edge work local; routing doesn't scan the whole diagram; visibility graph is sparse: only corners that can "see" each other become candidates; aesthetics rendering concern: keep the path feasible, then make it legible.*

Figure 53. SmartRoutingRefiner as a three-step algorithm: (1) local obstacle selection via an R*-tree query, (2) visibility-graph construction over obstacle features, (3) shortest-path search (A*) followed by spline smoothing. This separates correctness (obstacle avoidance) from aesthetics (curve shape).

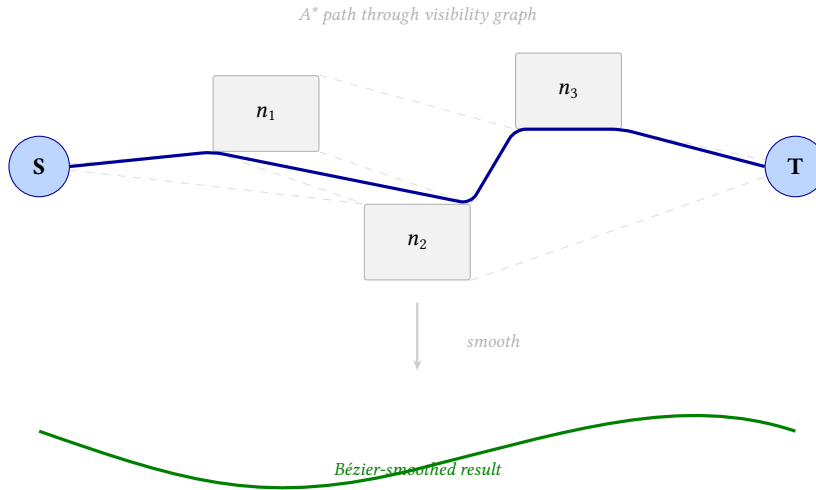


Figure 54. SmartRoutingRefiner. An R*-tree query identifies obstacles near each edge. A visibility graph connects obstacle corners; A* finds the shortest obstacle-free path. Waypoints are smoothed to Bézier splines for natural-looking curves.

16.2 Edge Clipping: Before and After

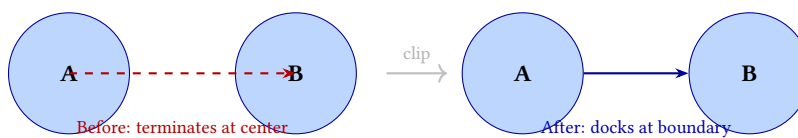


Figure 55. EdgeClippingRefiner: without clipping, edge paths terminate at node centers; with clipping, endpoints dock at the exact node shape boundary, giving correct arrowhead placement regardless of shape.

16.3 Deoverlap: Before and After

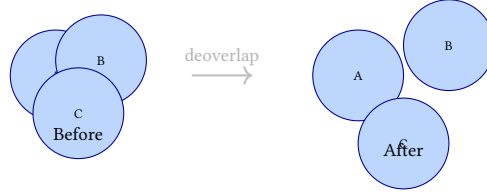


Figure 56. DeoverlapRefiner: applies a repulsive spring force between overlapping node bounding boxes, pushing them apart until the configured minimum margin is satisfied.

16.4 Boundary Bundling: Group Edge Routing

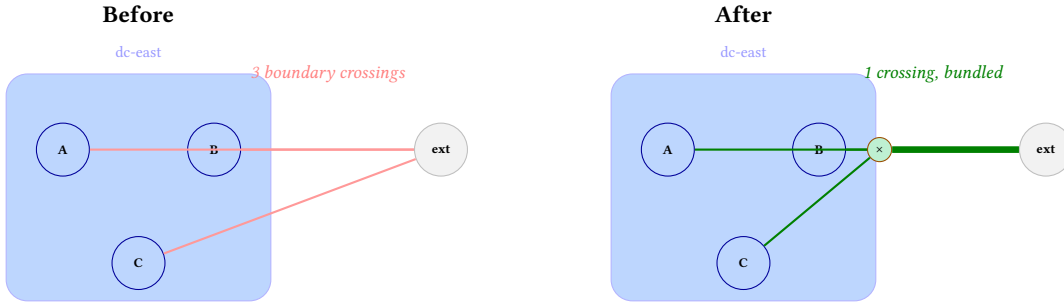


Figure 57. BoundaryBundlingRefiner. **Before:** three edges from internal nodes to an external node each cross the group boundary independently. **After:** all edges fan into a shared crossing point on the boundary and exit as a single bundled edge, reducing visual clutter at group borders.

17 Constraint Solver

NetVis supports *soft layout constraints* that are applied as post-simulation corrections. Unlike hard constraints that must be satisfied exactly, soft constraints apply proportional corrections gated by a priority-derived strength, allowing them to coexist with force-directed dynamics.

17.1 Constraint Types

Five constraint types are supported. All share a common priority field that determines both application order and correction strength via $s = \text{priority}/10$.

Type	Default Priority	Correction
Alignment	4.5	Move each node toward the average coordinate on the alignment axis: $\Delta = (\bar{c} - c_i) \cdot s$
MinDistance	5.5	If $d < d_{\min}$: push each node apart by $(d_{\min} - d)/(2s)$ along the connecting line
MaxDistance	5.5	If $d > d_{\max}$: pull each node together by $(d - d_{\max})/(2s)$ along the connecting line
Proximity	6.5	Pull each node toward the group centroid: $\Delta = (c - p_i) \cdot s$
Region	7.5	Smooth clamping: $p \leftarrow p + (\text{clamp}(p, \text{bounds}) - p) \cdot s$

Table 8. Constraint types with default priorities and correction algorithms. Constraints are applied in descending priority order; higher priority means the correction runs later and can override earlier adjustments. Strength $s = \text{priority}/10$.

Degenerate cases are handled conservatively: distances below 10^{-9} default to a unit displacement vector $(1, 0)$; constraints with fewer than two nodes are skipped.

17.2 Integration with Force Simulation

The constraint solver runs *once per simulation tick*, after all forces have been applied and positions updated. Nodes with fixed positions (pinned nodes) override any constraint corrections.

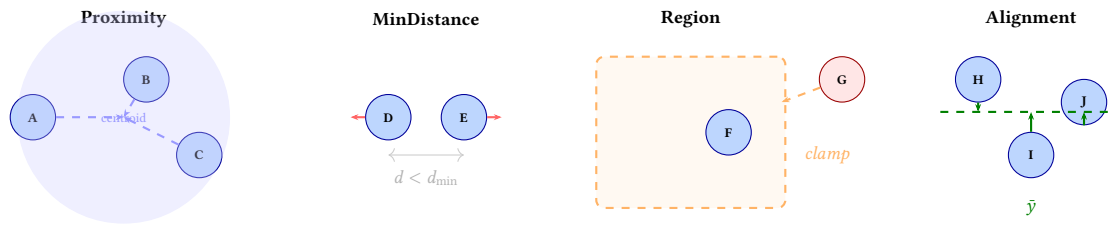


Figure 58. Constraint types visualised. **Proximity:** nodes pulled toward their centroid. **MinDistance:** overlapping nodes pushed apart symmetrically. **Region:** out-of-bounds nodes clamped smoothly toward the boundary. **Alignment:** nodes pulled to the average axis coordinate.

17.3 Conflict Detection and Violation Reporting

Eager conflict detection. Before layout begins, `detect_conflicts()` performs a pre-flight check for logically contradictory constraints:

- **Contradictory distances:** `MinDistance(A, B, x)` and `MaxDistance(A, B, y)` where $x > y$.
- **Proximity vs. min-distance:** A Proximity group containing nodes *A, B* that also have a `MinDistance` constraint (pulling together and pushing apart simultaneously).

If conflicts are detected, the constraint set is rejected with a diagnostic error before any layout work begins.

Post-layout violation reporting. After layout completes, `check_violations()` evaluates each constraint against the final positions. Violation thresholds are intentionally conservative (proximity: 120 px radius; alignment: 10 px deviation) to flag only *noticeably* unsatisfied constraints. The report includes satisfaction percentage, per-violation magnitude, and actionable suggestions.

17.4 YAML Configuration

constraints:

- type: alignment
 - nodes: [spine-1, spine-2, spine-3]
 - axis: horizontal
 - priority: 4.5
- type: min_distance
 - node_a: core-1
 - node_b: core-2
 - distance: 200
 - priority: 5.5
- type: region
 - nodes: [fw-1, fw-2]
 - bounds: { x_min: 0, x_max: 400,

y_min: 0, y_max: 300 }
 - priority: 7.5

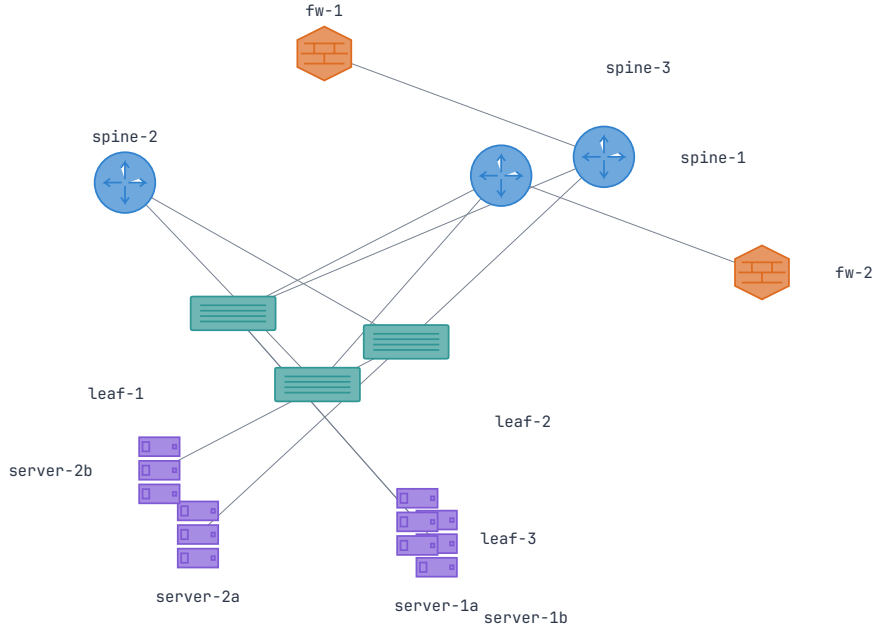


Figure 59. Constraint solver applied to a spine-leaf topology. Alignment constraints keep spine nodes on a horizontal axis; min-distance constraints prevent leaf nodes from overlapping; region constraints confine firewall nodes to a bounded area.

18 Crossing Minimizer

For layouts where edge crossings are the dominant aesthetic defect, NetVis provides a multi-start wrapper (CrossingMinimizer) that runs an underlying layout algorithm N times and selects the result with the fewest edge intersections.

18.1 Algorithm

1. Run the inner layout (typically force-directed) N times.
2. After each run, count edge crossings via pairwise segment intersection testing.
3. If a run produces zero crossings, return immediately (early exit).
4. Otherwise, return the result with the minimum crossing count.

Intersection test. The parametric segment intersection test solves for parameters $\lambda, \gamma \in [0, 1]$ on the two segments. To avoid counting shared endpoints as crossings, the test requires both parameters to lie in the strict interior $(0.01, 0.99)$. Parallel segments ($|\det| < 10^{-6}$) are excluded.

Curves (quadratic and cubic Bézier) are approximated by their endpoints for the crossing test in the minimizer's inner loop; the separate quality-metrics module (§30) uses finer 8-subdivision approximations for precise crossing counts.

19 Progressive Execution

Interactive front-ends (notably the WASM/Canvas renderer) require progress feedback and cancellation support during long-running layouts. The ProgressiveLayout wrapper provides this without modifying the core Layout trait.

19.1 Progress Callback

A ProgressCallback receives a ProgressInfo struct on each reporting tick:

- fraction: completion ratio $\in [0, 1]$, clamped.
- phase: human-readable phase name (e.g., "force-directed simulation").
- iteration / total_iterations: current and expected iteration counts.

For force-directed layouts, progress is reported every 10 iterations. For non-iterative layouts (tree, radial, hierarchical), the callback is invoked once with fraction = 1.0 after the layout completes.

19.2 Cancellation

The callback returns a boolean: `true` to continue, `false` to cancel. On cancellation, the force simulation breaks immediately and returns the partially-converged scene. All node positions are guaranteed to be finite, so the partial result is always renderable—the layout simply has not fully converged.

20 Curve Elegance

Producing smooth, professional-looking curves requires attention at every stage of the pipeline: arc generation, spline interpolation, corner rounding, tension control, post-processing, and SVG stroke rendering. This section documents the mathematical foundations underlying NetVis’s curve elegance system.

20.1 Circular Arc to Cubic Bézier Fitting

Geographic great-circle arcs, once projected to screen space, are smooth curves that must be rendered without visible facets. Rather than emitting polylines of many short `L` (`LineTo`) commands, NetVis fits cubic Bézier segments that produce identical visual results with 1–2 curve commands.

The kappa formula. For a circular arc of total angle a centered at the origin spanning from $-a/2$ to $+a/2$, a single cubic Bézier approximation places its control points at distance d along the tangent at each endpoint [17]:

$$d = \frac{4}{3} \tan\left(\frac{a}{4}\right) \quad (18)$$

For the quarter-circle ($a = \pi/2$), the half-angle identity gives the well-known constant $\kappa = \frac{4}{3}(\sqrt{2} - 1) \approx 0.5523$.

Control point placement. Let P_0 and P_3 be the arc endpoints on the unit circle, with outward tangent vectors $\mathbf{t}_0$ and $\mathbf{t}_3$. The Bézier control points are:

$$P_1 = P_0 + d \mathbf{t}_0 \quad (19)$$

$$P_2 = P_3 - d \mathbf{t}_3 \quad (20)$$

Error bounds. The maximum radial error for a single cubic Bézier approximating an arc of angle a has sixth-order accuracy:

$$\varepsilon \sim \frac{1}{54} \left(\frac{a}{4}\right)^6 \quad (21)$$

For a quarter-circle, $\varepsilon \approx 2.7 \times 10^{-4}$ —well below one pixel at any practical zoom level. Arcs exceeding $\pi/2$ are split into multiple segments:

Arc angle	Segments	Max error
$\leq \pi/2$ (90°)	1	2.7×10^{-4}
$\leq \pi$ (180°)	2	2.7×10^{-4}
2π (full circle)	4	2.7×10^{-4}

Table 9. Number of cubic Bézier segments needed per arc angle to maintain sub-pixel accuracy. Splitting at $\pi/2$ boundaries keeps per-segment error constant at the quarter-circle value.

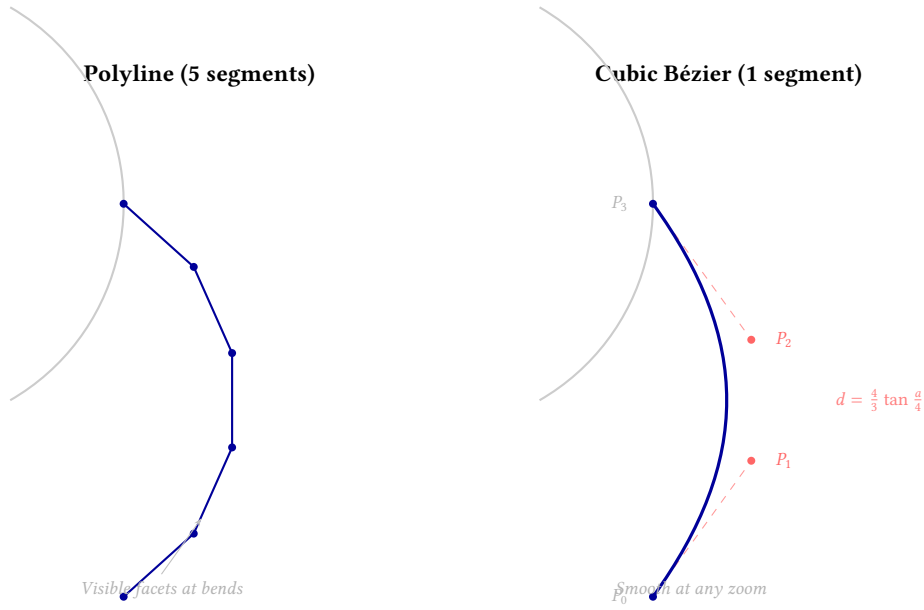


Figure 60. Geographic arc rendering comparison. **Left:** polyline approximation with 5 segments shows visible facets at bend points. **Right:** a single cubic Bézier with control points placed by Equation 18 is indistinguishable from the true arc. Red dashed lines show tangent handles; control points P_1, P_2 are at distance d along the tangent.

20.2 Rounded Corners on Orthogonal Paths

Orthogonal and hierarchical layouts produce Manhattan-style paths with right-angle bends. Replacing sharp corners with circular arcs or quadratic Béziers transforms mechanical-looking paths into polished diagrams.

Arc insertion. Given three consecutive waypoints A, B, C forming a bend at B with exterior angle θ , a circular arc of radius r is inscribed by trimming the segments at tangent distance t :

$$t = r \tan\left(\frac{\theta}{2}\right) \quad (22)$$

The arc starts at $T_1 = B + t \hat{u}_{BA}$ and ends at $T_2 = B + t \hat{u}_{BC}$, where $\hat{u}$ denotes the unit direction vector. For orthogonal paths ($\theta = \pi/2$), this simplifies to $t = r$ and the arc is a quarter-circle.

Segment clamping. The radius is clamped to prevent overlapping on short segments:

$$r_{\text{eff}} = \min(r, |AB|/2, |BC|/2) \quad (23)$$

Quadratic Bézier alternative. A quadratic Bézier with control points (T_1, B, T_2) produces a parabolic approximation tangent to both segments. The maximum deviation from the true circular arc at the midpoint is:

$$\delta = \frac{r(1 - \cos(\theta/2))}{3} \quad (24)$$

For 90° bends, $\delta \approx 0.098 r$ —visually indistinguishable from a true arc at typical stroke widths. A cubic Bézier using the kappa formula (Equation 18) with $a = \pi - \theta$ provides higher fidelity when needed.

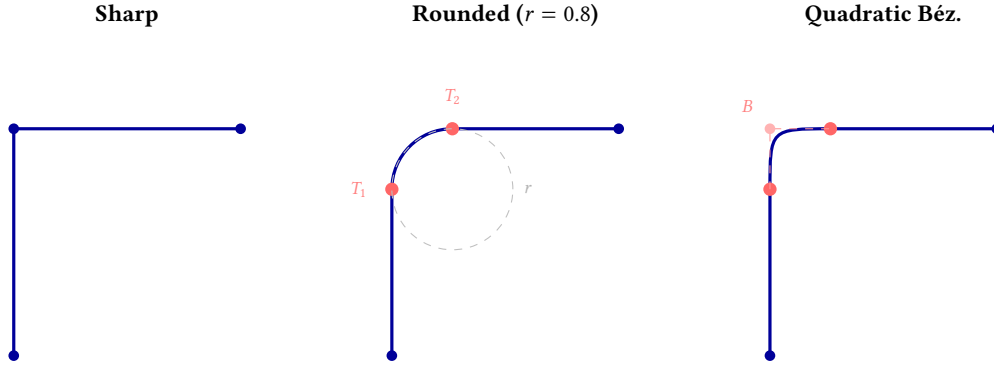


Figure 61. Corner rounding on orthogonal paths. **Left:** raw Manhattan routing with a sharp miter join. **Centre:** circular arc of radius r inscribed at tangent points T_1, T_2 . **Right:** quadratic Bézier using the corner point B as the sole control point; the parabolic curve is tangent to both segments and visually equivalent to the arc at typical stroke widths.

20.3 Centripetal Catmull–Rom Splines

NetVis’s FDEB bundler, spline routing, and flow path rendering all use Catmull–Rom interpolation (§15.4). The uniform parameterisation described in Equations 16–17 assumes equal spacing between control points—an assumption that breaks down when bundling produces unevenly-spaced subdivision points. Centripetal parameterisation [18] resolves this.

Knot sequence. Given control points P_0, P_1, P_2, P_3 , the knot values are:

$$t_i = t_{i-1} + \|P_i - P_{i-1}\|^\alpha \quad (25)$$

with $t_0 = 0$ and α selecting the parameterisation:

Name	α	Behaviour
Uniform	0	All intervals equal; cusps possible on uneven points
Centripetal	0.5	Intervals $\propto \sqrt{\text{chord length}}$; no cusps guaranteed
Chordal	1.0	Intervals $\propto \text{chord length}$; can produce flat spots

Table 10. Catmull–Rom parameterisation families. Only centripetal ($\alpha = 0.5$) guarantees cusp-free curves for all control point configurations [18].

Tangent vectors. At knot P_i with knot values t_{i-1}, t_i, t_{i+1} , the tangent is a weighted blend of the two adjacent chords:

$$\dot{P}_i = \frac{(t_{i+1} - t_i) \frac{P_i - P_{i-1}}{t_i - t_{i-1}} + (t_i - t_{i-1}) \frac{P_{i+1} - P_i}{t_{i+1} - t_i}}{t_{i+1} - t_{i-1}} \quad (26)$$

Conversion to cubic Bézier. For the segment $P_1 \rightarrow P_2$, with tangents $\dot{P}_1$ and $\dot{P}_2$ from Equation 26, the Bézier control points follow from the standard Hermite-to-Bézier mapping:

$$B_1 = P_1 + \frac{\dot{P}_1 (t_2 - t_1)}{3} \quad (27)$$

$$B_2 = P_2 - \frac{\dot{P}_2 (t_2 - t_1)}{3} \quad (28)$$

Note that these generalise Equations 16–17: when $\alpha = 0$ (uniform), the knot intervals cancel and the tangent reduces to $\dot{P}_i = \frac{1}{2}(P_{i+1} - P_{i-1})$, recovering the classical form.

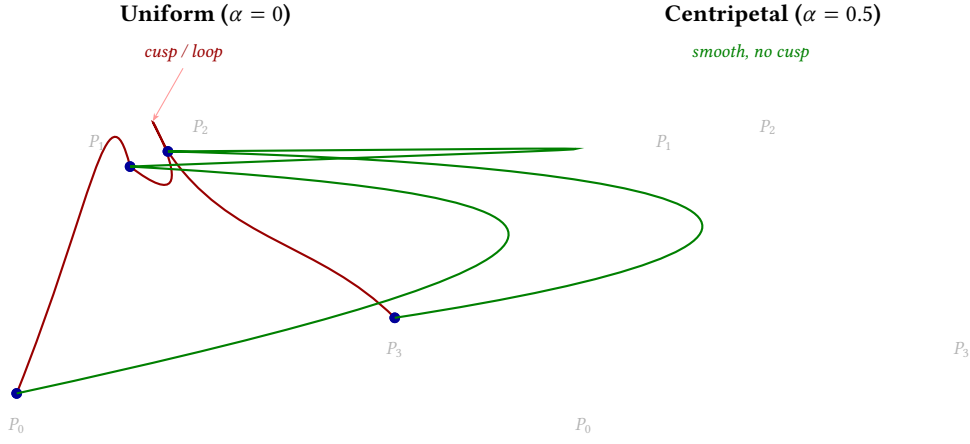


Figure 62. Uniform vs. centripetal Catmull–Rom on unevenly-spaced points. When P_1 and P_2 are close together, uniform parameterisation ($\alpha = 0$) produces a cusp or self-intersection (left, red). Centripetal parameterisation ($\alpha = 0.5$) scales the knot interval by $\sqrt{\|P_i - P_{i-1}\|}$, preventing the overshoot (right, green).

20.4 B-Spline Approximation

As an alternative to interpolating Catmull–Rom splines, NetVis offers uniform cubic B-spline approximation for edge bundling. B-splines produce visually softer curves because they *approximate* rather than interpolate the control polygon, achieving C^2 continuity (vs. C^1 for Catmull–Rom) [19].

Basis functions. For a uniform cubic B-spline segment over $u \in [0, 1]$ with control points $b_{i-1}, b_i, b_{i+1}, b_{i+2}$:

$$C(u) = \frac{1}{6} \begin{bmatrix} 1 & u & u^2 & u^3 \end{bmatrix} \begin{bmatrix} 1 & 4 & 1 & 0 \\ -3 & 0 & 3 & 0 \\ 3 & -3 & 3 & 0 \\ -1 & 3 & -3 & 1 \end{bmatrix} \begin{bmatrix} b_{i-1} \\ b_i \\ b_{i+1} \\ b_{i+2} \end{bmatrix} \quad (29)$$

Approximation property. At $u = 0$, the curve evaluates to $C(0) = (b_{i-1} + 4b_i + b_{i+1})/6$, a weighted average of three control points. This averaging is the source of both the smoothness (the curve cannot have sharp features) and the non-interpolation (the curve does not pass through b_i).

Conversion to cubic Bézier. Each B-spline segment converts to a cubic Bézier via knot insertion [20]:

$$Q_0 = (b_{i-1} + 4b_i + b_{i+1})/6 \quad (30)$$

$$Q_1 = (2b_i + b_{i+1})/3 \quad (31)$$

$$Q_2 = (b_i + 2b_{i+1})/3 \quad (32)$$

$$Q_3 = (b_i + 4b_{i+1} + b_{i+2})/6 \quad (33)$$

This preserves the C^2 joins while producing standard SVG-compatible C commands.

Spline type	Continuity	Interpolates?	Best for
Catmull–Rom (uniform)	C^1	Yes	Paths through waypoints
Catmull–Rom (centripetal)	C^1	Yes	Uneven point spacing
Cubic B-spline	C^2	No	Soft, flowing bundles

Table 11. Comparison of spline families used in NetVis. B-splines trade interpolation for an extra order of continuity, producing visually softer curves.

20.5 Tension Control

The Kochanek–Bartels formulation [21] introduces a *tension* parameter $T \in [-1, 1]$ that controls how tightly curves follow the control polygon. With bias $B = 0$ and continuity $C = 0$, the tangent at control point P_i simplifies to:

$$\mathbf{d}_i = \frac{1-T}{2} (P_{i+1} - P_{i-1}) \quad (34)$$

- $T = 0$: standard Catmull–Rom (tangent = $\frac{1}{2}$ chord).

- $T \rightarrow 1$: tangent approaches zero; curve degenerates to straight-line segments.
- $T < 0$: tangent exceeds half-chord; curve overshoots, producing loose, exaggerated arcs.

NetVis exposes this as a `-curve-tension` CLI parameter (default 0, i.e. standard Catmull–Rom). The FDEB bundler uses an internal tension equivalent to $T \approx 0.2$ (Catmull–Rom $\tau = 0.4$, mapping to $\tau = (1 - T)/2$).

The Bézier control points for a segment $P_i \rightarrow P_{i+1}$ with tangents $\mathbf{d}_i, \mathbf{d}_{i+1}$ are:

$$B_1 = P_i + \mathbf{d}_i/3 \quad (35)$$

$$B_2 = P_{i+1} - \mathbf{d}_{i+1}/3 \quad (36)$$

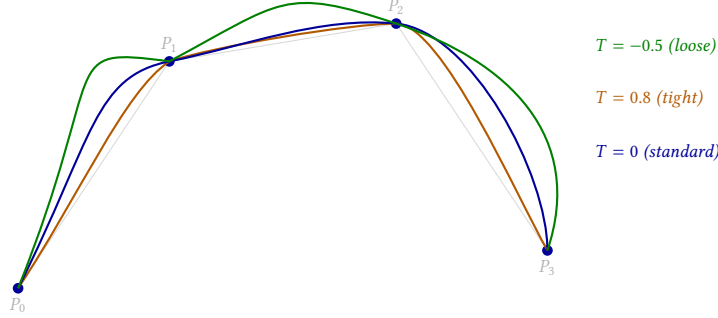


Figure 63. Effect of the Kochanek–Bartels tension parameter T on curve shape. At $T = 0.8$ (orange), the curve nearly follows the control polygon. At $T = 0$ (blue), standard Catmull–Rom smoothness. At $T = -0.5$ (green), the curve overshoots, producing exaggerated arcs. All three interpolate the same control points.

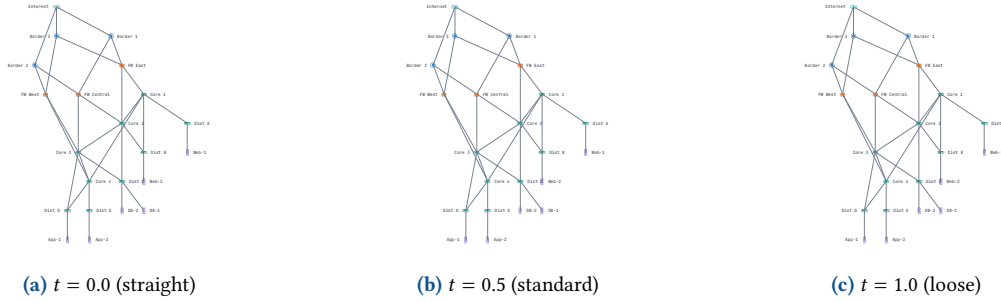


Figure 64. Effect of the global `-curve-tension` parameter on a hierarchical layout. At $t = 0$, orthogonal bends are sharp right angles. At $t = 0.5$ (default), corners are rounded with quadratic Bézier arcs using corner radius $= 30 \times t = 15$ px. At $t = 1.0$, maximum rounding produces flowing curves through the hierarchy. The same parameter simultaneously controls geographic arc curvature and FDEB spline smoothness.

20.6 Generalised Laplacian Smoothing

The Laplacian smoothing pass described in §15.4 (Equation 15) is extended to a general-purpose post-processing tool applicable to any multi-segment path: geographic arcs, routed edges, and spline waypoints.

Spectral interpretation. The discrete Laplacian on an n -point polyline (with fixed endpoints) has eigenvalues [22]:

$$\lambda_k = -2 \left(1 - \cos(k\pi/(n+1)) \right), \quad k = 1, \dots, n \quad (37)$$

After m passes with step size α , the k -th frequency mode is attenuated by:

$$A_k^{(m)} = \left(1 - \alpha \left(1 - \cos(k\pi/(n+1)) \right) \right)^m \cdot A_k^{(0)} \quad (38)$$

With $\alpha = 0.5$ and $m = 3$, the highest-frequency mode ($k = n$) is attenuated by a factor of $(1 - 2\alpha)^3 = 0$ for even n , effectively eliminated. Modes below $k \approx n/3$ are preserved with less than 10% attenuation, maintaining the overall edge shape while removing discretisation artifacts.

Stability. The explicit scheme is stable for $0 < \alpha \leq 1$. Iterating Equation 15 is equivalent to forward-Euler integration of the 1D heat equation $\partial P / \partial t = \kappa \partial^2 P / \partial x^2$, where each pass corresponds to one diffusion time step.

Target	α	Passes	Effect
Light denoising	0.3	1–2	Removes highest-frequency zigzag only
Moderate smoothing	0.5	3	Removes zigzag, preserves broad curves
Heavy smoothing	0.5	5–10	Significant shape simplification
Near-straight	0.5	20+	Converges toward straight line

Table 12. Laplacian smoothing presets. NetVis defaults to $\alpha = 0.5$, $m = 3$ for FDEB post-processing. Geographic arcs and routed edges use $\alpha = 0.3$, $m = 1$ –2 for light denoising.

20.7 SVG Stroke Join Geometry

The SVG stroke-`linejoin` attribute controls how the renderer fills the gap at path vertices. NetVis defaults to round joins on all curved paths to eliminate miter spikes.

Miter geometry. At a bend of angle θ between consecutive segments, the miter point extends to a distance:

$$\ell_{\text{miter}} = \frac{w/2}{\sin(\theta/2)} \quad (39)$$

where w is the stroke width. As $\theta \rightarrow 0$ (near-parallel segments), $\ell_{\text{miter}} \rightarrow \infty$, producing visible spikes. The SVG stroke-`miterlimit` M (default 4) converts to bevel when $1/\sin(\theta/2) > M$, i.e. at $\theta < 2 \arcsin(1/M) \approx 29^\circ$ for $M = 4$.

Round joins. A semicircular arc of radius $w/2$ centred at the path vertex fills the corner region. This produces smooth, spike-free corners regardless of bend angle, making it the appropriate default for bundled and spline-fitted edges.

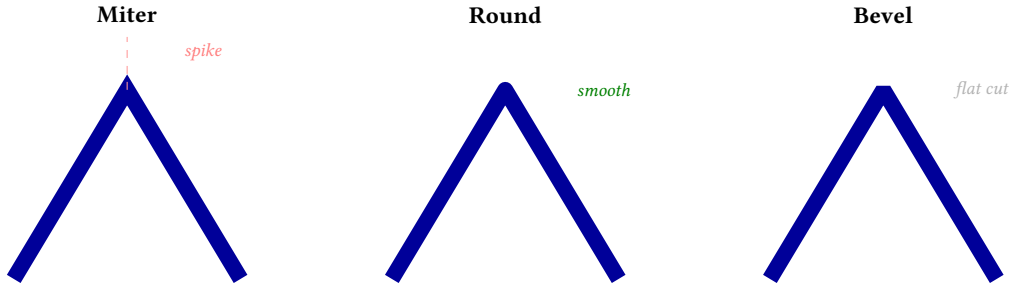


Figure 65. SVG stroke join types at an acute angle. **Miter:** extends both stroke edges to a point, producing a spike proportional to $1/\sin(\theta/2)$. **Round:** fills with a semicircular arc; smooth at all angles. **Bevel:** straight cut across the outer corner; cleaner than miter but faceted. NetVis uses round joins by default on all curved paths.

21 Topology Diffing

The `diff` module computes added, removed, and modified nodes and edges between two `NetVisGraph` instances (by ID and attribute comparison). The `apply_diff_styles()` function color-codes the rendered scene: green for added, red/dimmed for removed, amber for modified. A diff legend is automatically inserted.

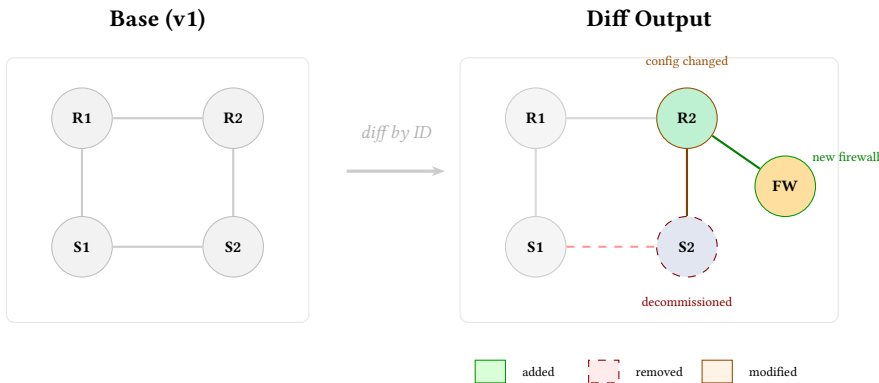


Figure 66. Topology diffing by example. The base topology (v1) is compared to a new version by matching node and edge IDs. The diff output color-codes changes: S2 was decommissioned (red/dashed), R2's config changed (amber), and a new firewall was added (green). Unchanged elements remain gray.

```
netvis diff old.yaml new.yaml -o diff.svg
```

22 Temporal Analysis and Playback

22.1 Event-Sourced Temporal Model

The temporal system uses an event-sourced `TemporalGraph` that records topology evolution as a sequence of discrete events: `NodeAdded`, `NodeModified`, `NodeRemoved`, `EdgeAdded`, `EdgeModified`, and `EdgeRemoved`. Each event carries a timestamp and attribute delta. The `TemporalGraph` materializes any point-in-time snapshot by replaying events up to that timestamp, producing a full `NetVisGraph` instance.

For batch workflows, the legacy directory-scan mode remains available: YAML topology files are scanned from a directory, timestamps extracted from filenames or metadata, and ordered chronologically. Per-entity lifecycle tracking traces each node's and edge's first appearance, all attribute changes, and removal across snapshots.

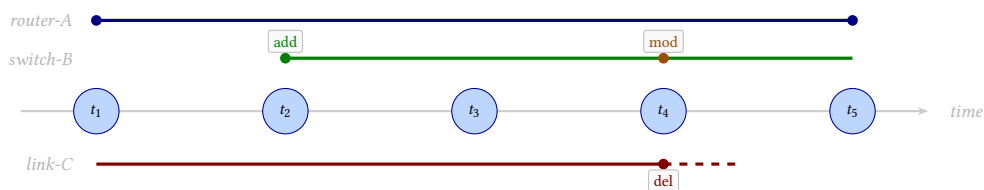


Figure 67. Temporal analysis: entity lifecycle tracking across topology snapshots. Each snapshot (t_1 – t_5) corresponds to a point in time. The timeline module traces first appearance (add), attribute changes (mod), and removal (del) for every node and edge.

22.2 Playback Primitives

The temporal module provides three core primitives for interactive playback:

Timeline An ordered sequence of timestamps with binary-search snapping helpers for mapping continuous time to discrete snapshot indices.

PlaybackController Manages a normalized step-space playhead (floating-point step index) driven by external Δt values (typically from `requestAnimationFrame`). Supports `play()`, `pause()`, `scrub_to()`, and `step_by()` controls. Default step interval is 250 ms; playback stops at the final step (no auto-loop).

Track<T> A keyed sequence of values with hold-last-value semantics and optional linear interpolation for numeric types (`f64`, `Point2D`). Used to animate time-varying attributes (`style.fill`, `style.stroke`, `style.opacity`, etc.) between snapshots.

22.3 Snapshot-Driven Scene Application

The `build_scene_from_snapshot()` function materializes a `TemporalGraph` snapshot into a renderable `Scene` without re-running layout or refiners. It produces editor-style geometry (straight edges from node centers) and applies time-varying style attributes from the snapshot's node and edge metadata. The `CanvasRenderer::set_scene()` method performs a safe scene swap, rebuilding the hit-testing index and ID maps while preserving selection and hover state via external-ID capture/restore helpers.

22.4 WASM Temporal Player

The `TemporalPlayer` provides browser-based temporal playback via the WASM target. It accepts a base topology and a sequence of timestamped graph events (`NodeAdded/Modified/Removed`, `EdgeAdded/Modified/Removed`) as input.

Key features:

- **RAF-driven playback loop:** the player ticks on each `requestAnimationFrame`, advancing the playhead by the elapsed time.
- **Transitions:** short (~150 ms) cancelable tweens between snapshots—node centers interpolate linearly; appearing elements fade in, disappearing elements fade out. Transitions retarget from the current rendered state on interrupt.
- **Keyboard shortcuts:** Space (play/pause), Arrow Left/Right (step ± 1), Shift+Arrow (step ± 10).
- **Filter persistence:** the active `InteractiveFilter` is re-applied on each scene swap, with a `filterHasNoMatches` indicator for UI badges.

- **Selection/hover preservation:** external IDs are captured before and restored after each scene swap, maintaining user context across time changes.

Timeline CLI (batch mode)

```
netvis timeline --dir snapshots/
netvis query --dir snapshots/ --node router-fra-01
```

23 Traffic Flow Animation

The traffic module injects animated flow overlays into rendered SVGs via CSS `@keyframes`. Each edge can carry utilization (0.0–1.0), direction (forward/reverse/bidirectional), and animation style (dot/dash). Utilization maps to color: green (low) → amber → red (congested). Zero-traffic edges receive CSS dimming (`opacity: 0.3`). Play/pause and speed controls are injected as an SVG overlay. For the Canvas2D rendering path (WASM target), traffic animation uses a [LOD](#)-aware rendering strategy that scales to high node counts by reducing particle density and simplifying paths at lower zoom levels.

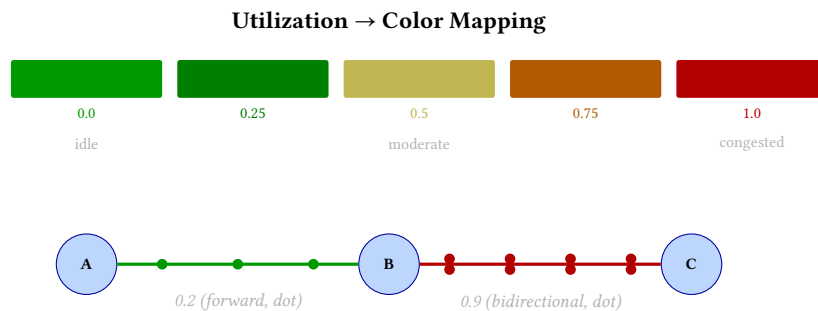


Figure 68. Traffic flow animation. Edge utilization (0.0–1.0) maps to a green–amber–red color ramp. Animated dots travel along the edge path; bidirectional mode shows two opposing dot streams. CSS `@keyframes` drives the animation in SVG output.

23.1 Heatmap and Flow-Field Overlays

Two additional traffic visualization modes complement the animated-dot approach:

Heatmap. The `generate_heatmap` function accumulates edge-metric values onto a 128×128 grid using Gaussian kernels ($\sigma = \text{kernel_cells}/2$, default kernel radius 80 scene units). Grid intensities are normalized to $[0, 1]$, mapped through the `weathermap` color ramp (§13.1), and encoded as a semi-transparent RGBA PNG overlay ($\alpha = 0.6$ default).

Flow field. The `generate_flow_field` creates a regular grid of directional arrows by associating each cell with its nearest edge’s traffic data. Influence attenuates linearly to zero at $5 \times$ the grid spacing. Arrows are equilateral triangles whose length encodes magnitude and whose direction follows the edge angle, rendered as SVG paths with opacity proportional to magnitude.

24 Annotation Markup

24.1 Annotation Data Model

Annotations are first-class elements in `NetVisGraph`, stored as `AnnotationData` instances and declarable directly in YAML via the `annotations` field. Each annotation carries text content and an `AnnotationTarget` specifying its anchor mode:

Global(x, y) Absolute position in scene coordinates; independent of any graph element.

Node(id, offset) Offset from a node’s center; follows the node through layout changes.

Edge(id, offset) Offset from an edge’s midpoint; useful for labeling specific links.

When an anchored node or edge is deleted, the annotation degrades gracefully from anchored to free-floating at its last resolved absolute position. Layout resolution converts all annotations into `SceneCallout` entries with concrete scene coordinates stored in the `Scene`.

24.2 SVG Callout Rendering

The SVG renderer draws annotations as text callouts with leader lines from callout to anchor. Three styles are supported: **balloon** (rounded container), **leader-line** (text with straight line), and **margin** (text in margin with line to target). Annotations support layer filtering for selective visibility.

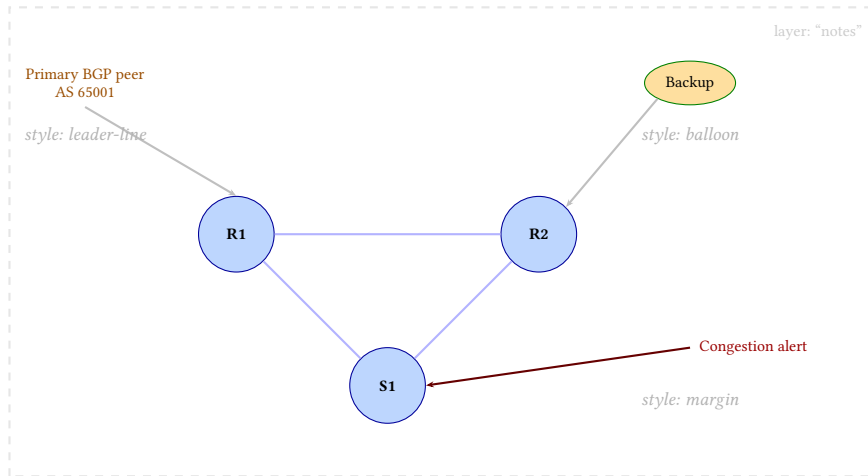


Figure 69. Annotation callout styles. Each annotation anchors to a node or edge with a pointer line. Three styles are supported: **leader-line** (direct line to text), **balloon** (text in a rounded container), and **margin** (text placed in the diagram margin). Annotations are organized into named layers for selective visibility.

24.3 Geographic Scale Bars

For geographic topologies (§7), NetVis generates an automatic **scale bar** that provides a physical distance reference (in kilometers or meters). The `nice_distance()` algorithm picks human-readable intervals (e.g., 10, 50, 100 km) based on the current view's geographic span. The scale bar includes major ticks and unit labels, and its length automatically adjusts during WASM zoom operations to maintain accuracy relative to the projected world-space.

24.4 Spatial Collision Integration

Annotations participate in the R^* -tree-based spatial label placer (§14). During layout, annotation bounding boxes are registered as `RegionKind::Annotation` occupied regions *before* the edge-label and node-label placement passes. This ensures that automatically placed labels avoid overlapping user-authored annotations. Bounding box dimensions are computed using viewport-aware effective font sizes (clamped to 8–16 px for node-label scale).

25 Interactive Editor

The editor module provides interactive topology editing for browser deployments via the WASM target. It implements the command pattern: `MoveNodeCommand`, `TransactionCommand`, etc., storing only deltas (24 bytes for a node move) rather than full snapshots. An `UndoStack` with 50-action capacity provides undo/redo, using $\sim 400,000\times$ less memory than snapshot-based undo at 5,000 nodes.

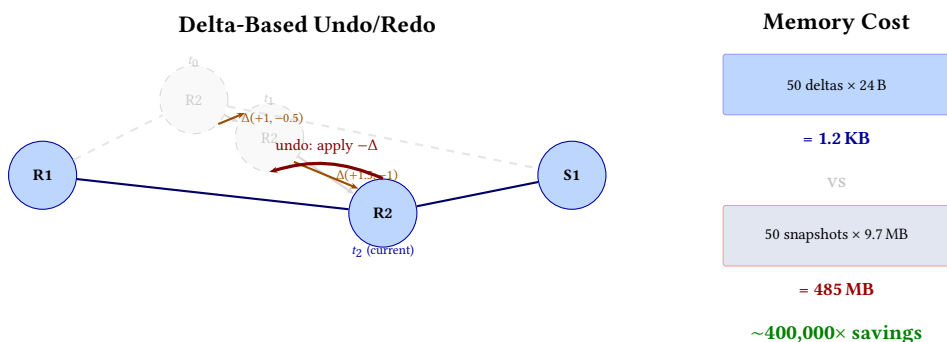


Figure 70. Interactive editor with spatial undo. Node R2 is dragged through three positions; each move stores only a delta ($\Delta x, \Delta y$, 24 bytes) rather than a full scene snapshot. Undo applies $-\Delta$ to restore the previous position. At 5,000 nodes the delta approach uses 1.2 KB vs. 485 MB for snapshot-based undo.

25.1 Web Component

A Lit-based `<netvis-editor>` custom element wraps the WASM editor as a framework-agnostic Web Component with Shadow DOM encapsulation. The element exposes a programmatic API (`loadTopology()`, `exportSVG()`) and attribute-based configuration for declarative usage in any HTML page or framework.

25.2 WebWorker Layout

Layout computation is offloaded to a background Web Worker via a `LayoutClient` with a Promise-based API. This prevents the main thread from blocking during force-directed simulation on large topologies. The worker supports progress callbacks for intermediate positions and an algorithm selector for switching between layout engines at runtime.

25.3 Node Pinning and Auto-Layout

Nodes carry a pinned boolean field. Pinned nodes maintain their user-set positions during force-directed layout while still exerting forces on unpinned neighbors. Pinning is exposed as an undoable command (`SetNodePinnedCommand`) and via WASM bindings for the editor UI. A parameter tuning panel exposes iterations, charge, and `link_distance` sliders with 500 ms debounced real-time preview, allowing users to interactively adjust layout quality.

25.4 Import/Export Pipeline

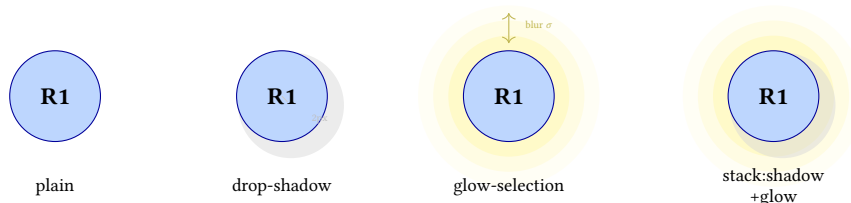
The editor supports round-trip topology import from JSON, YAML, NetBox, and LLDP/CDP adapters via a browser-based UI. Export options include SVG, PNG, and a compressed Share URL (`#nv1: scheme`) that encodes the full topology as a shareable URL fragment. The encoding pipeline is `JSON → DEFLATE → base64url (no padding)`, prefixed with `#nv1:`. A 3,500-character guard prevents exceeding browser and proxy URL limits. A “Copy Embed” action generates an `<iframe>` snippet for embedding in documentation or dashboards. Node positions are preserved across JSON/YAML round-trips for layout fidelity.

26 Visual Effects

NetVis provides a pluggable visual effects system mapping semantic effect names to SVG filter definitions. The `EffectRegistry` deduplicates identical filter definitions via content-addressed hashing: each effect computes a SHA-256 signature over `name || theme || theme || params` (null-byte separated to prevent length-extension ambiguity), and effects sharing the same signature share a single `<defs>` entry. The filter id is the sanitized effect name suffixed with 16 hex characters from the signature prefix. Effects are assigned via the `highlight` field (see §40):

Name	Type	Description
drop-shadow-subtle	Drop shadow	Light offset shadow
drop-shadow-emphasis	Drop shadow	Strong offset shadow
inner-shadow-boundary	Inner shadow	Inset shadow for group boundaries
glow-selection	Glow	Selection highlight
glow-status	Glow	Status indicator
glow-emphasis	Glow	Emphasis marker
stack:<a>+	Composed	Combine any two base effects

Table 13. Built-in visual effects.



SVG `<filter>`: `SourceGraphic → feGaussianBlur → feOffset → feMerge` (identical chains deduplicated by content hash)

Figure 71. Visual effects applied to the same node. **Plain:** no filter. **Drop shadow:** offset blurred copy behind the node. **Glow:** colored Gaussian blur halo radiating outward. **Composed:** `stack:` combines both via SVG `feMerge`. The `EffectRegistry` deduplicates identical filter chains by content hash.

All effects adapt to the active theme. Glow intensity and shadow direction vary by theme family.

Effect Budget

SVG filter performance degrades at ~50+ filtered elements. `effects.max_filtered_elements` (default: 50) sets a budget.

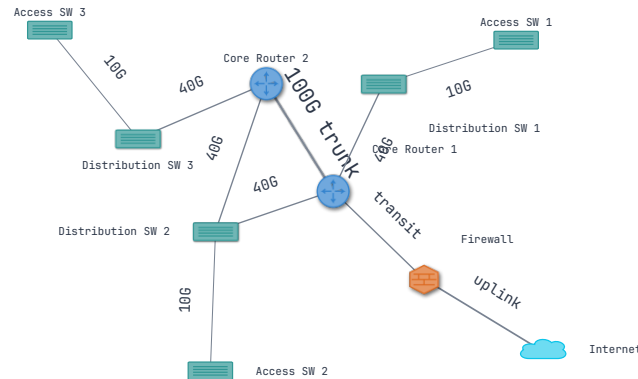


Figure 72. Visual effects showcase. Left to right: drop shadow, glow selection, emphasis glow, and composed stack:drop-shadow+glow. All effects adapt their intensity and color to the active theme.

27 Graph Analytics

The analytics module computes structural graph metrics and injects them as a visual “Network Insights” card. Three metrics are computed: **betweenness centrality** (Brandes’ algorithm, $O(nm)$) identifies critical transit nodes; **graph diameter** (longest shortest path) highlights worst-case latency; **clustering coefficient** measures local redundancy.

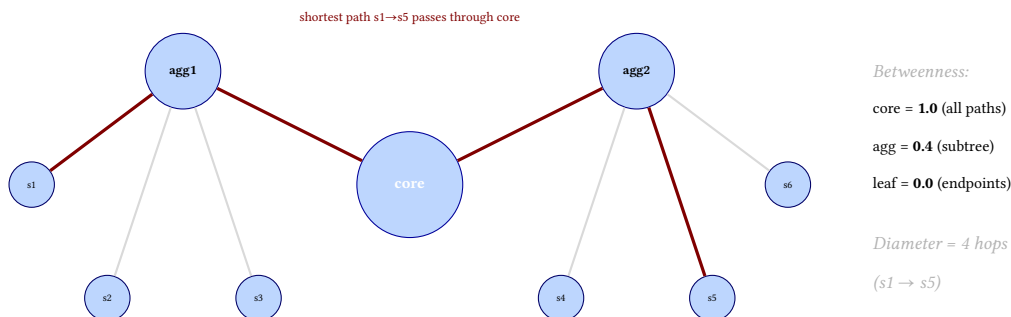


Figure 73. Graph analytics: betweenness centrality visualized. Node size encodes centrality—the core router sits on every shortest path between the two subtrees (centrality 1.0), making it the critical transit point. Aggregation nodes score 0.4 (bridging their subtrees); leaf nodes score 0.0. Graph diameter (longest shortest path) is 4 hops.

Analytics CLI

```
netvis --input topology.yaml --analytics -o analytics.svg
```

27.1 Community Detection

NetVis provides three community detection algorithms for automatic group identification:

Louvain modularity optimization (via `graphrs`). Weighted, resolution-parameterized, convergence threshold 10^{-4} . Modularity is computed post-hoc. Communities are sorted by lexicographic member-name key for deterministic ordering.

Label propagation. Randomized seeded iteration with early-exit convergence and $O(n)$ per-iteration cost. Budget: 100 iterations maximum.

Girvan–Newman edge betweenness. Iteratively removes the highest-betweenness edge (BFS-based computation), splitting the graph into a target number of communities.

Automatic algorithm selection. The `suggest_algorithm` heuristic selects the algorithm based on graph size and density ($\rho = 2|E|/(|V|(|V| - 1))$):

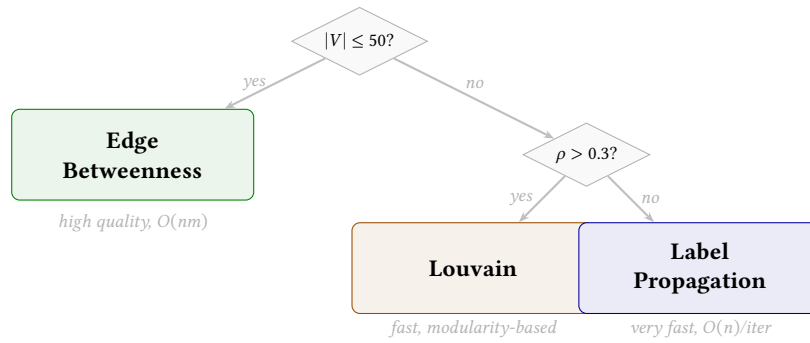


Figure 74. Automatic algorithm selection for community detection. Small graphs (≤ 50 nodes) use the high-quality Girvan–Newman algorithm; large dense graphs use Louvain; large sparse graphs use label propagation for speed.

Group-constrained clustering. When user-defined groups already exist, `cluster_within_constraints` partitions nodes by manual group membership first, runs the selected algorithm independently within each partition, and merges results. This ensures user-authored group boundaries are respected while still discovering sub-communities within each group.

28 Filter DSL

NetVis includes a Domain-Specific Language (DSL) for filtering topology elements before rendering. The system comprises three layers: a **parser** that converts filter expressions into an AST (supporting boolean operators, attribute selectors, glob patterns, and set operations); an **evaluator** that walks the AST per node/edge; and an optional **LLM layer** (feature-gated) that translates English queries into filter AST nodes.

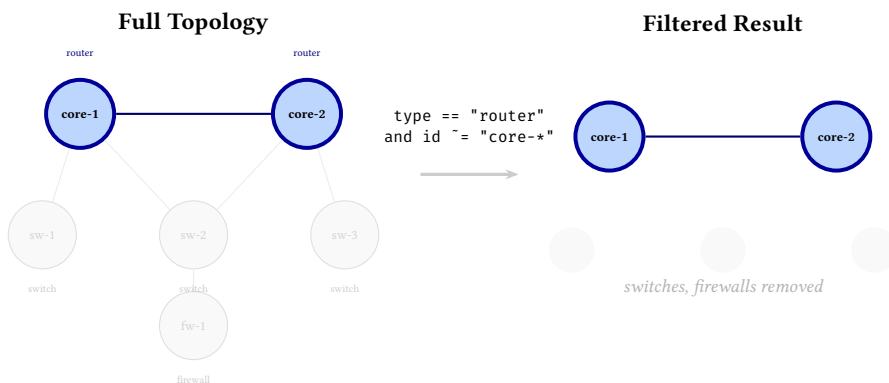


Figure 75. Filter DSL in action. The expression `type == "router" and id ~ "core-"` matches only the two core routers (highlighted). All non-matching nodes and their incident edges are removed from the rendered output. The filter supports boolean operators, attribute comparisons, and glob patterns.

Filter DSL examples

```

netvis --input t.yaml --filter 'type == "router"'
netvis --input t.yaml \
  --filter 'group == "dc-east" and id ~ "core-"'
  
```

28.1 Interactive Filtering (Canvas)

Separate from the batch filter DSL, the WASM canvas renderer provides an `InteractiveFilter` system for live, per-frame visibility masking. The `InteractiveFilter` uses a predicate AST evaluated against node and edge attributes, toggling element visibility without re-running layout. The interactive filter persists across temporal scene swaps (§22), and exposes a `filterHasNoMatches` indicator for UI feedback when no elements match the active filter.

29 Input Validation

Topology and configuration files are validated against a JSON Schema (generated from Rust types via `schemars`). The `validation::fuzzy` module provides “Did you mean...?” suggestions for misspelled YAML field names

using Levenshtein distance (strsim crate). Structured diagnostics are returned as `ValidationDiagnostic` entries with severity, location, and suggested corrections.

Schema export and validation

```
netvis schema --type topology > topology-schema.json
netvis validate --input topology.yaml
```

30 Quality Metrics

NetVis computes quality metrics as part of every render, following the aesthetic metrics framework of Purchase [23] and Dunne et al. [24].

30.1 Metric Families

Metric	Description
Edge crossings	R*-tree segment intersection, weighted by angle
Overlap area	AABB intersection between all label/node pairs; sub-pixel (<2 px) ignored
Label readability	Composite of owner distance, font-size variance, reading order
Angular resolution	Minimum angle between consecutive incident edges
Node alignment	Degree to which nodes align on axes or grid lines
Edge clearance	Minimum distance from non-incident edges to nodes
Coincident edges	Detection of duplicate or overlapping edge paths
Contrast	Label color contrast against background (WCAG-derived)
Edge-length variance	Coefficient of variation across edge lengths
Spatial distribution	Evenness of node distribution (entropy-based)
Path straightness	Curvature penalty for bent edges
Viewport coverage	Elements outside viewport bounds
Node-edge overlap	Non-incident edge segments crossing node bounding boxes
Density heatmap	Spatial density for identifying visual hotspots

Table 14. 14 of the 16 quality metric families computed per render. All are aggregated into a unified `ClarityReport` (0–100).

WCAG contrast checking. The Contrast metric above is backed by a full WCAG 2.1 implementation in the `a11y` module. Relative luminance is computed per the WCAG specification:

$$L = 0.2126 R_{lin} + 0.7152 G_{lin} + 0.0722 B_{lin} \quad (40)$$

where each channel is linearized from sRGB via the standard breakpoint at 0.04045. The contrast ratio between two luminances is $(L_{lighter} + 0.05)/(L_{darker} + 0.05)$, yielding values in $[1, 21]$.

For semi-transparent labels, compositing is performed in linear sRGB (not gamma-encoded) before measuring contrast, since the perceived ratio depends on the composited result. A binary-search solver (`solve_min_alpha_for_contrast`) finds the minimum alpha value that achieves a target contrast ratio (e.g., WCAG AA at 4.5:1), enabling automatic opacity adjustment for overlaid text. The high-contrast theme applies these checks to guarantee WCAG AA compliance for all label/background combinations.

30.2 Clarity Score and CI Integration

The composite *Clarity Score* (0–100) combines four equally-weighted factors: edge crossings, angular resolution, node spacing, and label overlap. It is designed as a single-number summary for CI/CD pipelines.

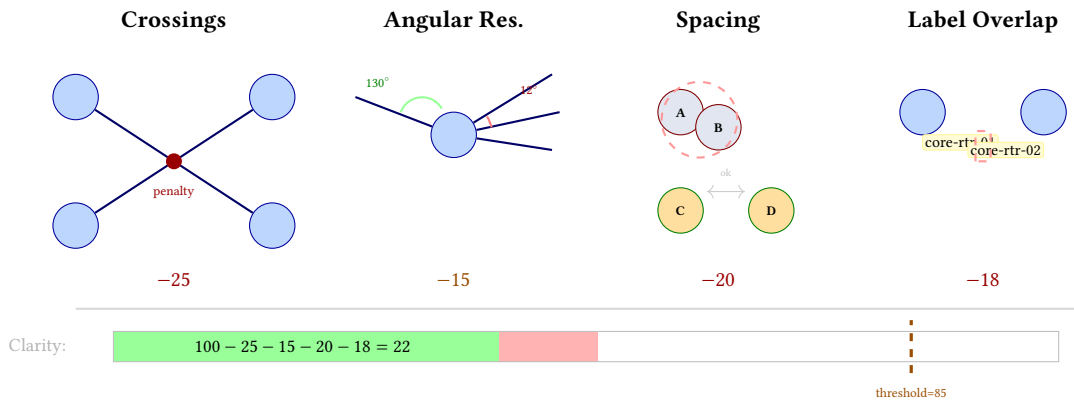


Figure 76. Clarity score: four quality factors illustrated spatially. Each panel shows the visual defect and its penalty. Edge crossings, narrow angular resolution, overlapping nodes, and colliding labels all subtract from 100. CI/CD pipelines gate on a configurable threshold (default: 85; this example scores 22—a clear failure).

Quality baselines are stored in `benches/regression_baselines.json`; CI fails if any metric degrades by more than 20%.

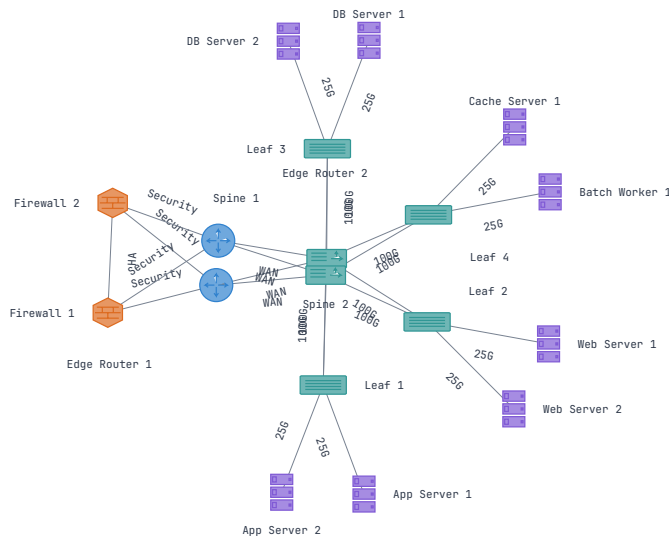


Figure 77. Quality metrics are computed alongside every render and can be overlaid on the output. Each node's betweenness centrality, edge utilization, and label overlap are directly visible.

Separate from aesthetic quality, the validation module performs structural topology checks: isolated nodes, disconnected subgraphs, duplicate edge IDs, and missing labels. Results appear as Warning-severity entries in the `DiagnosticsEnvelope`.

31 Rendering Pipeline

31.1 SVG Renderer

The primary renderer produces SVG using the `svg crate`. Elements render in painter's-model order: background map layers → isometric layer planes → group boundaries → edges → nodes → labels → effects.

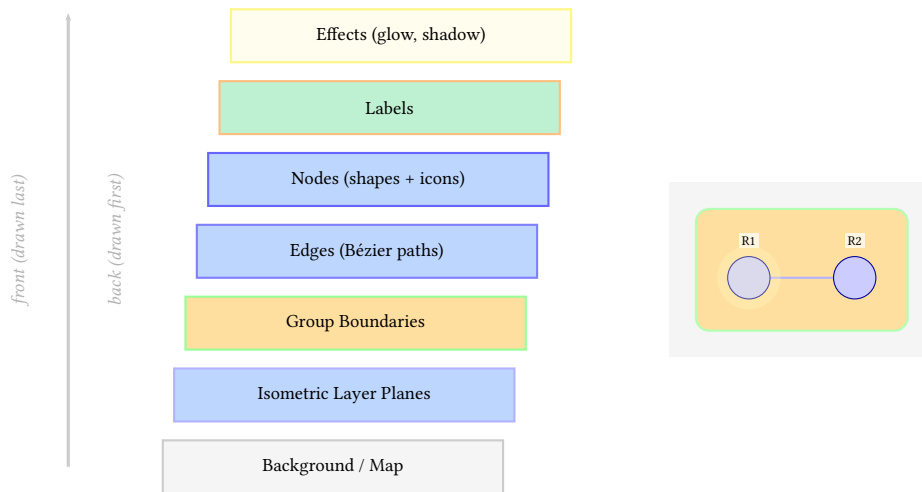


Figure 78. SVG rendering layer order (painter’s model). Elements are drawn back-to-front: background first, effects last. This ensures labels are always readable over edges, nodes always cover edge strokes at endpoints, and effect overlays (glow, shadow) render above all content.

31.2 Level of Detail

LOD suppresses visual details at configurable thresholds, following Abello et al. [25]:

Profile	Labels	Edge Lbl.	Interfaces	Effects
Conservative	2000	1000	1000	500
Balanced	500	300	300	200
Aggressive	200	100	100	50

Table 15. LOD suppression thresholds.

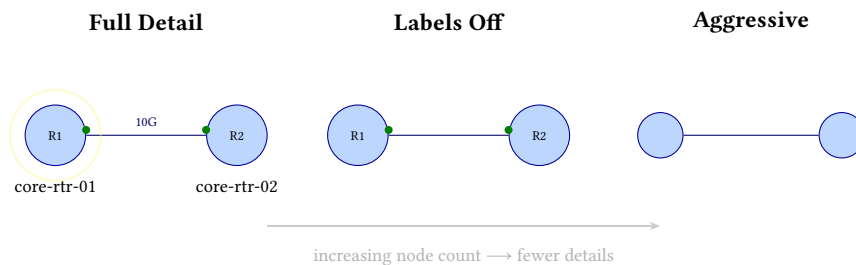


Figure 79. Level of detail (LOD) progressive suppression. As topology size grows past profile thresholds, labels, edge labels, interface markers, and visual effects are progressively hidden to maintain render performance and visual clarity.

31.3 Canvas2D Renderer

The WASM target renders to HTML5 Canvas2D using a three-layer compositing architecture that separates content by update frequency:

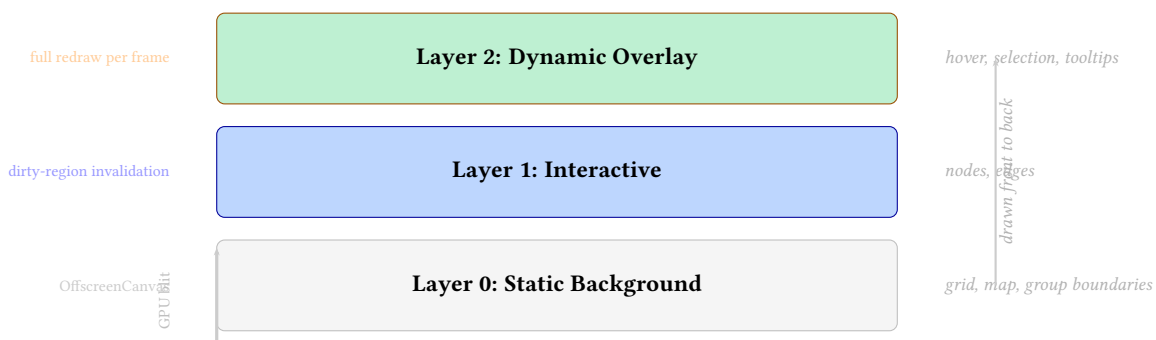


Figure 80. Three-layer canvas compositing. Layer 0 is an OffscreenCanvas rendered once and blitted via drawImage(); Layer 1 uses dirty-region tracking for partial redraws; Layer 2 redraws fully each frame but contains only lightweight overlay elements. This reduces 1,000-node repaints from ~16 ms to 2–5 ms for localized updates.

Inset overview map (mini-map). The WASM renderer includes an **inset overlay** that provides a miniature view of the full topology in a corner of the canvas. This “mini-map” serves three purposes: (1) global context during high-zoom exploration; (2) visual indication of the current viewport via a tracked bounding rectangle; and (3) **click-to-navigate** functionality, where clicking inside the inset instantly centers the main view on the corresponding world coordinates. The inset uses a simplified rendering mode (small dots for nodes, thin lines for edges, no labels) to minimize the performance impact of redundant rendering.

Dirty-region tracking. The `DirtyRegionTracker` maintains a list of invalidated bounding boxes. On each frame, only cells overlapping a dirty region are cleared and re-rendered. `mark_all_dirty()` forces a full repaint (e.g., on viewport resize or scene swap).

Viewport transform. The `ViewportTransform` applies translate-then-scale to the canvas context. Cursor-relative zoom keeps the world point under the cursor stationary: given cursor screen position (s_x, s_y) , the new pan is computed as $\text{pan}_x = s_x - w_x \cdot \text{scale}_{\text{new}}$, where (w_x, w_y) is the cursor’s world coordinate before zoom. Scale is clamped to $[0.1, 10.0]$.

Hit detection. Interactive hit testing reuses the R*-tree spatial index (§14). `CanvasHitIndex` bulk-loads node AABBs and edge AABBs (expanded by a 3 px tolerance for thin strokes) into an R*-tree at $O(n \log n)$. Point queries run in $O(\log n)$; candidates are sorted by bounding-box area (smallest first) so nodes win over edges in overlapping regions.

Progressive rendering. For initial scene display, the `Canvas2D` renderer uses frame-chunked progressive rendering: 200 nodes and 500 edges per `requestAnimationFrame` callback, in three phases (edges → nodes → labels). This prevents the browser’s main thread from blocking during first paint of large topologies. Device-pixel-ratio (DPR) scaling ensures retina-quality output.

31.4 Terminal Renderer

`netvis show` renders topology to Unicode box-drawing and ANSI color codes without a graphical display. `light` produces plain ASCII; `dark` produces ANSI-colored output.

31.5 SVG Accessibility

SVG exports include WCAG 2.1 Level AA compliant semantic structure. Each SVG root carries `role="img"` with a `<title>` and `<desc>` element. Nodes are grouped with `role="graphics-symbol"` and `aria-label` attributes derived from node ID and type. Logical groupings (layers, group boundaries) use `role="group"` with descriptive labels. Tab-order indices (`tabindex`) enable keyboard navigation of interactive SVG output.

WCAG contrast checking. The `atty::contrast` module implements WCAG 2.x relative luminance and contrast ratio computation following the W3C specification. Channel linearization uses the standard sRGB breakpoint (0.04045), and alpha-aware compositing operates in linear sRGB space (not gamma) to avoid perceptual inaccuracies. A `solve_min_alpha_for_contrast` function performs binary search to determine the minimum foreground alpha that achieves a caller-supplied target contrast ratio against a given opaque background—supporting AA (4.5:1), AAA (7.0:1), and large-text (3.0:1) thresholds. The module is API-ready for future enforcement; current consumers include the high-contrast theme validation test suite.

31.6 Export Formats

SVG is the canonical output. Additional formats: PNG via `resvg`; PDF via `svg2pdf`; self-contained interactive HTML (WASM embedded as base64, pan/zoom/drag, openable from `file://`); Graphviz DOT for interoperability; JSON (structured scene with node positions, edge control points, bounds, and metrics); GraphML (standards-compliant XML for import into Gephi, yEd, and Cytoscape, with optional layout coordinates as data attributes); GeoJSON (FeatureCollection with Point nodes and LineString edges for geographic topologies, requires `lat/lon` node attributes); and snapshot (timestamped JSON capturing complete topology, metrics, and positions for temporal archiving).

IEEE publication figures. A specialized **figure export mode** targets academic publication standards. Presets can be selected via the CLI flag `--figure-size preset` or the YAML `render: block`:

YAML figure sizing

```
render:
  figure_size: ieee-single
```

Five presets are provided:

Preset	Width	Min font	Target
ieee-single	3.5 in	7 pt	IEEE TVCG single-col
ieee-double	7.0 in	6 pt	IEEE TVCG double-col
acm-single	3.33 in	7 pt	ACM single-col
acm-double	7.0 in	6 pt	ACM double-col
a4-full	7.5 in	6 pt	A4 full-width

Table 16. Built-in figure-size presets. Custom dimensions may also be specified as WxH (e.g. 5x3).

Font and stroke widths are rescaled to the target canvas: $font_scale = \max(target_px/800, 0.6)$; $stroke_scale = \max(font_scale, 1.0)$. The floor of 0.6 prevents text from becoming illegibly small on narrow single-column targets. Each preset enforces a per-column minimum font size (see Table 16) and the final PDF dimensions match the target publication column width exactly.

PDF QR codes. The `render_pdf_with_qr` pipeline injects a vector-rendered QR code into the PDF’s margin or corner. This provides a physical-to-digital bridge: readers of a printed report can scan the code to instantly access the interactive WASM version of the topology (hosted at a URL encoded in the QR) on their mobile devices. The QR code is rendered as precise PDF paths, maintaining crispness at any zoom level.

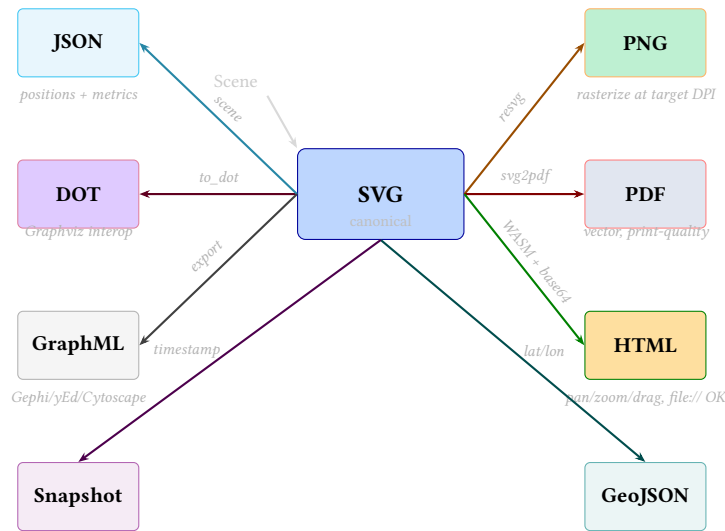


Figure 81. Export format pipeline. SVG is the canonical intermediate representation. `resvg` rasterizes to PNG; `svg2pdf` produces vector PDF; HTML embeds WASM for interactive viewing; DOT enables Graphviz interoperability; JSON exports structured scene data; GraphML provides standards-compliant interchange; GeoJSON exports geographic topologies; Snapshot captures timestamped state for temporal archiving.

32 Pluggable Node Renderer

The `NodeRenderer` trait defines the interface for custom node renderers. Built-in types (router, switch, server, firewall, cloud, `l3_switch`, and the full icon library from §10) are registered at startup via the `inventory` crate’s compile-time static registration—a zero-overhead plugin pattern where each type calls `inventory::submit!` and is discovered at runtime via `inventory::iter` with no central registry mutation. The same pattern is used for adapter registration (§34). The `render::registry` module provides lookup functions (`find_node_renderer`); unknown `node_type` values fall back to the default circle renderer.

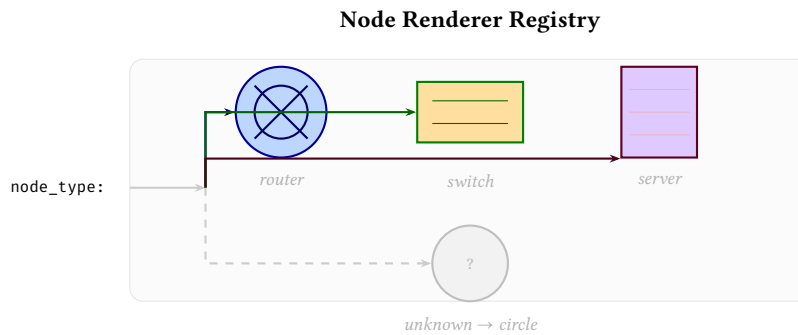


Figure 82. Pluggable node renderer dispatch. The `render::registry` module maps `node_type` strings to shape renderers. Built-in types (router, switch, server, firewall, cloud, l3_switch) are registered at startup. Unknown types fall back to the default circle renderer, ensuring graceful degradation for unrecognized device types.

33 A Grammar of Network Graphics

33.1 The What & Why: Moving Beyond Fixed Themes

Most network visualization systems rely on rigid styling APIs, auto-suggested layouts that function as opaque heuristics, or monolithic YAML configuration files. While NetVis provides robust default layout heuristics and predefined visual themes, these approaches fundamentally limit the practitioner's ability to precisely encode domain-specific telemetry into the structural and visual layout.

Building upon the internal relational mapping pipeline described in §3, we propose a formal Grammar of Graphics [6] tailored explicitly for network visualization. However, unlike standard data science where observations are plotted as independent geometric marks using rigid tools like `ggplot2`, network topologies are heavily interdependent. They require layered coordinate routing, domain-specific edge bundling, and cascading style rules.

To solve this, we introduce an expressive, declarative Python API inspired by the fluent builder pattern of dataframes (like Polars). The grammar directly mirrors NetVis's internal Rust pipeline, decoupling data from visuals through a chain of type-based bindings, rules, and topological filters.

33.2 The How to Use It: A Fluent Builder API

A grammar for networks must encapsulate node and edge aesthetics, routing behaviors, collapsing logic, and dynamic rule evaluation. The proposed Python API allows practitioners to pipe a topology object through a sequence of declarative transformations.

The grammar evaluates styling rules using a strict precedence hierarchy. This prevents "Feature Soup" by ensuring that explicit data mappings always override generic type definitions, and conditional state rules (e.g., active alarms) override base mappings.

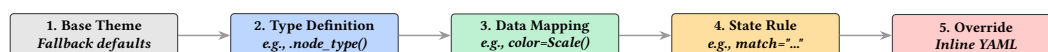


Figure 83. Visual Precedence Hierarchy. The pipeline resolves styling attributes from left to right, ensuring that dynamic telemetry and active state correctly override baseline topologies.

Fluent Grammar Concept: Polars-Inspired Builder

```
from netvis.grammar import NetVis, Scale, Style

# The declarative builder maps topological data onto a visual scene
spec = (
    NetVis(topology)

    # 1. Base coordinate system and refiners
    .layout("force-directed", cooling=0.9, barnes_hut=True)

    # 2. Type-based default geometries
    .node_type("router", shape="hexagon", size=15)
    .node_type("switch", shape="square", size=10)
    .edge_type("uplink", width=3, parallel_mode="bundled")

    # 3. Data-driven Visual Mappings (Grammar of Graphics)
    .node_mapping(fill=Scale.category("vendor", palette="datacenter"))
```

```

        .edge_mapping(color=Scale.sequential("utilization", palette="inferno"))

# 4. Conditional State Rules
        .node_rule(match="status:critical", fill="red", glow=True)

# 5. Smart collision-free labels via R*-tree
        .labels(text="hostname", avoid_collisions=True, size=10)
    )

# Execution: Compiles the grammar and executes the layout pipeline
svg = spec.render()

```

By adopting this chained syntax, power is shifted back to the user. Rather than relying on NetVis to automatically guess how to render a complex scene, the user explicitly dictates styling based on node types, metric scales, and logical predicates.

33.3 Walkthrough: One Topology, Many Perspectives

A frequent challenge in network engineering is that practitioners require fundamentally different perspectives of the *exact same data*. A YAML configuration assumes a single "best" representation. In contrast, the fluent builder pipeline leverages the NetVis internal Filter DSL (§30) directly within the grammar chain.

Consider a medium-sized enterprise datacenter spanning hundreds of nodes. Using the fluent Python API, a practitioner can transform the identical underlying topology object into three distinct operational views simply by chaining different rules.

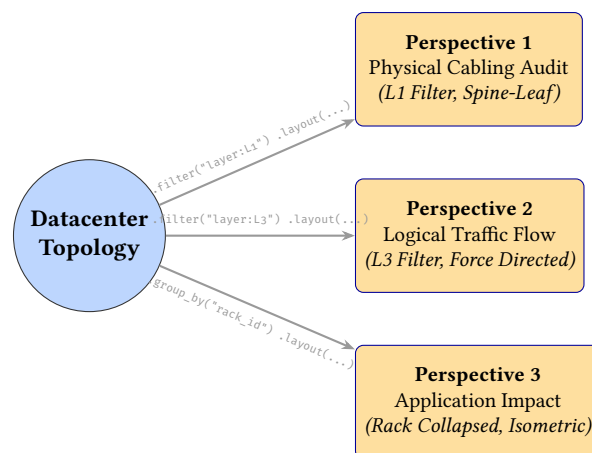


Figure 84. One Topology, Many Perspectives. The chained builder allows a single baseline topology to be dynamically mapped into completely distinct visual structures without altering the source data.

33.3.1 Perspective 1: The Physical Cabling Audit

Goal: Verify physical redundancy and cable types at the hardware layer.

```

# 1. Filter to Layer 1 (Physical) using NetVis Filter DSL
# 2. Enforce an orthogonal spine-leaf routing layout
# 3. Show every individual parallel cable (offset mode) with interface dots
physical_view = (
    NetVis(datacenter_topo)
        .filter("layer:L1")
        .layout("spine-leaf")
        .node_type("spine", shape="hexagon", size=20)
        .node_type("leaf", shape="square", interfaces=True)
        .edge_type("fiber", color="orange", width=2)
        .edge_type("copper", color="blue", width=1)
        .edges(parallel_mode="offset", directed=False)
        .labels(show=True, text="hostname")
        .theme("high_contrast")
)

```

33.3.2 Perspective 2: Logical Routing & Telemetry

Goal: Visualize OSPF costs, BGP peerings, and live traffic congestion.

```

# 1. Filter to Layer 3 (Logical)

```

```

# 2. Auto-optimize force-directed parameters to minimize edge crossings
# 3. Map throughput telemetry to edge colors via sequential scales
logical_view = (
    NetVis(datacenter_topo)
    .filter("layer:L3")
    .layout(
        "force-directed",
        optimize=["edge_crossings", "angular_resolution"]
    )
    .node_mapping(size=Scale.linear("total_routes", range=(10, 30)))
    .edge_mapping(color=Scale.sequential("utilization", palette="inferno"))
    .node_rule(match="status:critical", fill="red", glow=True)
    .edges(parallel_mode="bundled", directed=True)
    .theme("dark")
)

```

33.3.3 Perspective 3: Cross-Layer Application Impact

Goal: Abstract away individual switches to see how an alert impacts full physical racks over time.

```

# 1. Collapse hardware into aggregate 'rack' groupings
# 2. Use 3D isometric projection
# 3. Diff against yesterday's state to highlight new flows
impact_view = (
    NetVis(datacenter_topo)
    .layout("isometric")
    .group_by("rack_id", render_as="bounding_box")
    .node_rule(match="highest_alert:critical", fill="red", shape="hexagon")
    .edge_mapping(width=Scale.log("app_flows", range=(1, 8)))
    .diff(baseline=yesterday_topo, removed=Style(opacity=0.2, linetype="dashed"))
)

```

33.3.4 Perspective 4: Geographic Flow Tracing & Structural Constraints

Goal: Visualize a multi-hop application path across a Wide Area Network (WAN) while enforcing structural alignment for non-geographic edge locations.

```

# 1. Map core routers to their physical latitude/longitude
# 2. Render physical links as great circle arcs
# 3. Highlight a specific multi-hop BGP path with glowing emphasis
# 4. Enforce structural alignment for cloud transit gateways
wan_view = (
    NetVis(wan_topo)
    .layout("geographic", great_circle_arcs=True)
    .constraint(group="aws-transit", align="horizontal", keep_apart=50)
    .node_mapping(size=Scale.linear("bgp_peers", range=(5, 15)))
    .edge_type("fiber", color="gray", width=1)
    .path(
        waypoints=["ams-cr1", "fra-cr2", "par-cr1"],
        layer="BGP",
        color="#ff00ff",
        emphasis=True
    )
    .theme("light")
)

```

This multi-perspective capability highlights why a programmatic Python DSL is vastly superior to static YAML for analytical workflows: the visualization is dynamically compiled from domain telemetry, perfectly mapping to the internal Rust rendering pipeline.

33.4 Case Study: The Morphology of Parallel Links

A defining challenge of network cartography is the representation of multiple links between the same two devices (e.g., Link Aggregation Groups, redundant cabling). Rigid tools force a single visual interpretation. By applying the grammar to the *exact same topological subset*, we can radically alter the semantic interpretation of the connection.

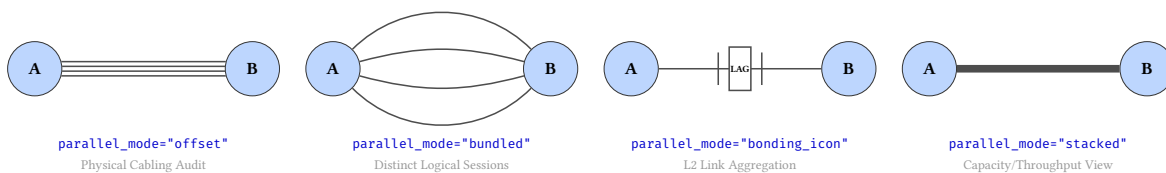


Figure 85. The Morphology of Parallel Links. A single topological pair (A, B) connected by four edges is rendered in four distinct modes by simply mutating the `parallel_mode` property in the grammar. This eliminates the need for manual coordinate hacking to achieve domain-specific semantic meaning.

In a single YAML file, the user would have to manually offset coordinates or write custom SVG filters. With the fluent builder, transitioning from a physical wiring audit (`offset`) to a logical capacity view (`stacked`) requires only a single method modification: `.edges(parallel_mode="stacked")`.

33.5 Case Study: Extreme Morphing of an Annotated Data Structure

To truly demonstrate the power of separating data from visual representation, we must start with an intentionally rich, annotated topological data structure. In an educational or operational setting, the baseline data should contain multiple dimensions of telemetry—geography, structural role, and real-time metrics—without implying any specific visual layout.

The Baseline Input Data Structure

```
{
  "nodes": [
    { "id": "NYC-CR1", "type": "core", "lat": 40.71, "lon": -74.00, "metrics": { "bgp": 50 } },
    { "id": "LON-CR1", "type": "core", "lat": 51.50, "lon": -0.12, "metrics": { "bgp": 42 } },
    { "id": "BOS-PE1", "type": "edge", "lat": 42.36, "lon": -71.05, "metrics": { "bgp": 12 } }
  ],
  "edges": [
    { "from": "NYC-CR1", "to": "LON-CR1", "attributes": { "latency": 75, "capacity": 100 } },
    { "from": "NYC-CR1", "to": "BOS-PE1", "attributes": { "latency": 10, "capacity": 10 } }
  ]
}
```

By applying the Grammar of Graphics to this single payload, we can completely morph its structural geometry to answer entirely different questions. We can scale nodes dynamically based on the `metrics.bgp` payload, restrict visibility based on type, or completely alter the spatial projection mapping.

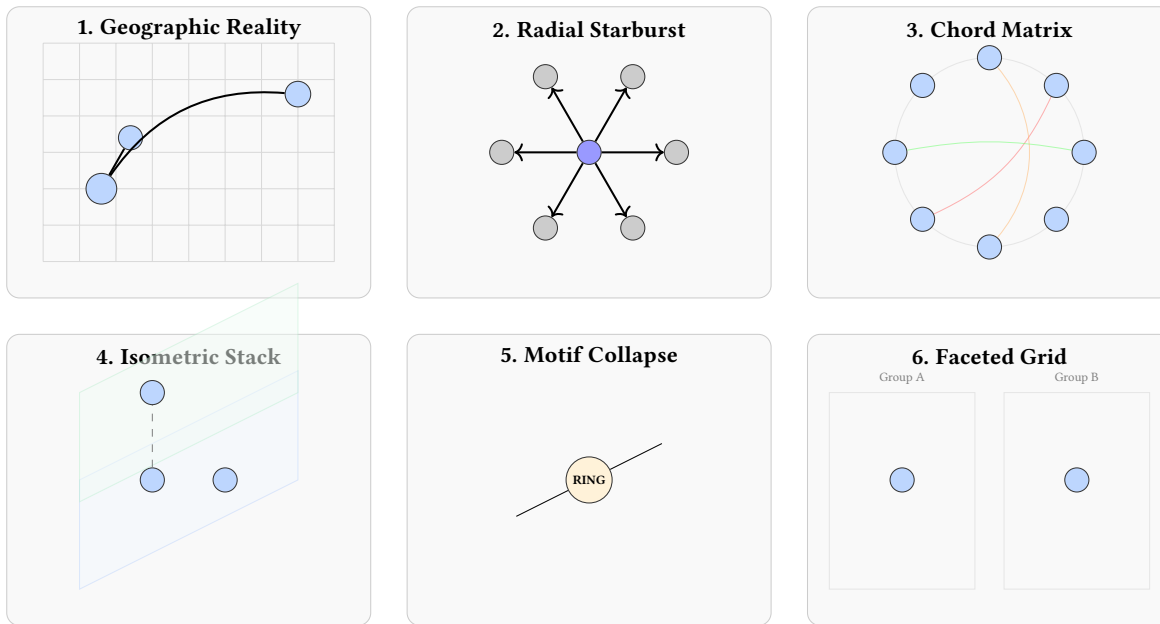


Figure 86. The extreme morphing capability of the NetVis grammar. A single topological dataset is projected through six distinct visual angles, each optimized for a specific analytical task: from geographic reality to abstract traffic matrices and semantic summaries.

33.5.1 Angle 1: The Geographic Reality

Maps nodes to their true lat/lon coordinates, rendering the transatlantic link as a curved great-circle arc. Node size is scaled proportionally by the size of their BGP routing tables.

```
geo_view = (
    NetVis(data)
    .layout("geographic", great_circle_arcs=True)
    .node_mapping(size=Scale.linear("metrics.bgp", range=(5, 20)))
    .edge_mapping(width=Scale.linear("attributes.capacity", range=(1, 5)))
)
```

33.5.2 Angle 2: The Starburst (Radial) Topology

Ignores geography entirely. Pins the type="core" nodes to the center and radiates type="edge" nodes outward in a starburst, revealing the underlying hub-and-spoke dependency model.

```
radial_view = (
    NetVis(data)
    .layout("radial", root_match="type:core")
    .node_mapping(fill=Scale.category("type", palette=["blue", "gray"]))
    # Taper edges to visually enforce directional hierarchy
    .edges(taper="target", opacity=0.8)
)
```

33.5.3 Angle 3: The Chord Diagram Matrix

Forces all nodes into a rigid circular layout, sorting them by their BGP metrics, and routing edges internally across the circle's chord. This completely abstracts the network into a pure traffic-flow matrix, making it trivial to spot asymmetric latency outliers via color gradients.

```
chord_view = (
    NetVis(data)
    .layout("circular", sort_by="metrics.bgp")
    .edges(routing="chord")
    .edge_mapping(color=Scale.sequential("attributes.latency", palette="RdYlGn_r"))
)
```

33.5.4 Angle 4: The 3D Isometric Stack

Exploits the multidimensional nature of network overlays by breaking the topology into vertical 3D layers based on functional domains (e.g., Physical vs. Logical routing). This separates control-plane telemetry from data-plane capacity without cluttering a 2D plane.

```
isometric_view = (
    NetVis(data)
```

```

.layout("isometric", tilt=30, rotation=45)
.group_by("type", render_as="layer_plane")
# Cast intra-layer links as solid, and inter-layer as dashed vertical beams
.edges(routing="curved", opacity=0.7)
.edge_rule(match="is_cross_layer:true", linetype="dashed")
)

```

33.5.5 Angle 5: The Motif Collapse (Semantic Zoom)

For massive topologies, raw node views become noisy "hairballs." This angle invokes the internal Motif Simplification engine. Rather than plotting every edge device, it collapses common topological motifs (like redundant HA pairs or dense access rings) into single intelligent glyphs, summarizing their capacity structurally.

```

collapsed_view = (
  NetVis(data)
  .layout("force-directed", cooling=0.9)
  # Automatically collapse well-known graph motifs (e.g., dual-homed rings) into glyphs
  .simplify(strategy="motif_collapse", target_motifs=["redundant_pair", "access_ring"])
  # Map the aggregated underlying capacity to the size of the new collapsed glyph
  .node_mapping(size=Scale.log("aggregated_capacity"))
)

```

33.5.6 Angle 6: The Faceted Small Multiples

A hallmark of traditional data science grammars (like ggplot2) is the ability to split complex datasets into matrices of smaller, independent charts. By applying a facet wrap on the group property, the topology is automatically partitioned into a grid of isolated subgraphs, making it trivial to compare the internal structure of different transit domains side-by-side.

```

facet_view = (
  NetVis(data)
  .layout("force-directed")
  .facet_wrap("group", columns=2)
  .node_mapping(color=Scale.category("type", palette="set1"))
  .theme("print_friendly")
)

```

This extreme morphing capability is essential for both pedagogical courses and advanced operations: the data dictates the *what*, while the declarative grammar seamlessly manages the *how*.

33.6 The Grammar of Time and Interaction

A network is not a static painting; it is a relational database of geometry, state, and time. To fully realize a Grammar of Graphics for networks, the declarative API must extend beyond spatial mappings and embrace temporal and interactive dimensions. Traditional tools force practitioners to abandon their declarative configuration and drop into imperative JavaScript (e.g., D3.js) the moment they require animation or user interaction.

By targeting our WASM rendering engine instead of static SVG, the Python builder natively supports *Statistical Overlays*, *Time-Series Transitions*, and *Declarative Interactivity* directly within the fluent grammar chain.

```

blast_radius_view = (
  NetVis(temporal_event_graph)
  .layout("force-directed")

  # 1. The Stat Layer: Render a continuous density heatmap behind the nodes
  .stat_density(metric="cpu_util", geom="polygon", palette="inferno", opacity=0.3)

  # 2. The Animation Layer: Play back the event history
  .transition(frame_by="timestamp", duration="10s", ease="linear")

  # 3. The Interaction Layer: Bind WASM behaviors without writing JavaScript
  .interact(on="click", action="highlight_neighbors", depth=2)
)

```

In this "Blast Radius" scenario, the practitioner creates a dynamic web component with three lines of declarative grammar. The underlying Rust engine interprets these commands as a multi-stage pipeline:

- **Statistical Overlays (stat_density):** *The Why:* In massive networks, individual node telemetry (like CPU spikes) becomes unreadable noise. *The How:* Rather than plotting discrete shapes, the engine uses a 2D Kernel Density Estimation (KDE) to project node metrics onto a continuous grid. It then draws marching-squares isocontours to render a "weather map" heatmap beneath the topology. This instantly

reveals regional hotspots of congestion or failure that are otherwise hidden in the noise of individual nodes.

- **Time-Series Transitions (transition):** *The Why:* Networks are fundamentally temporal; analyzing an outage requires understanding the *sequence* of cascading failures, not just the final broken state. *The How:* If the input data is an event-sourced graph containing chronological changes, this layer compiles a WASM playback controller. The engine automatically calculates smooth, D3-style vector interpolations for node positions, edge routing changes, and color shifts as the graph transitions between timestamps. For automation pipelines, this exact same transition layer can be exported headlessly to an MP4 video file via FFmpeg bindings, turning diagnostic graphs into shareable incident response videos.
- **Declarative Interactivity (interact):** *The Why:* Complex topologies often require user drill-down (semantic zoom) to explore localized areas without losing the global context, but forcing network engineers to write DOM event listeners creates too much friction. *The How:* The engine binds specific interaction rules directly to the WASM event loop via the AST. When the user clicks a compromised node in the browser, the grammar instructs the engine to recursively highlight its neighbors up to a `depth=2`, instantly visualizing the potential blast radius. No manual JavaScript DOM manipulation is required.

33.7 Cognitive Continuity: Cinematic Layout Morphing

The single most radically accretive feature enabled by this architecture solves a fundamental problem in network cartography: *cognitive dissonance*. When an engineer switches from a physical WAN map (to view a fiber cut) to a logical BGP tree (to view routing loops), they must mentally rebuild the entire network from scratch. Because the visual layout changes instantly, the brain loses object permanence. "Router A," which was located in London on the map, is suddenly in the top-left corner of a hierarchical tree.

By combining the Grammar of Graphics with the parallel execution speed of the Rust engine and the vector interpolation of the WASM target, we introduce **Cinematic Layout Morphing**.

```
storyboard = (  
  NetVis(telemetry_data)  
  # Define independent coordinate projections  
  .scene("Physical", layout="geographic", edge_routing="great_circle")  
  .scene("Logical", layout="radial", root="type:core")  
  .scene("Traffic", layout="circular", edge_routing="chord")  
  
  # Sequence the morphing transitions bound to a scroll event  
  .morph(sequence=["Physical", "Logical", "Traffic"], trigger="scroll")  
)
```

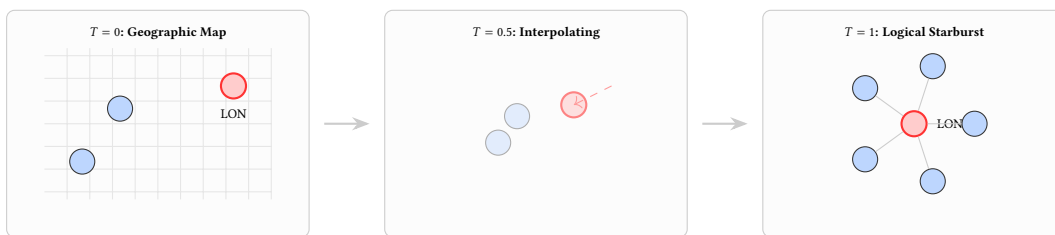


Figure 87. The Cinematic Layout Morphing storyboard. By smoothly interpolating node coordinates across different coordinate projections, NetVis preserves cognitive continuity, allowing the brain to track individual entities (e.g., the LON router) as they rearrange from geographic locations into logical hierarchies.

Rather than computing a single static layout, the engine simultaneously computes all three spatial projections (Geographic, Radial, Circular). The WASM frontend then smoothly interpolates the $\langle x, y \rangle$ coordinates of every node and edge. As the user scrolls down the page, the network physically untangles itself—London physically slides from the map and anchors itself into the center of the logical starburst.

This preserves **cognitive continuity**. The user never loses track of the entities. This capability is uniquely suited to NetVis's architecture: standard Python data science tools generate static images, while standard browser-based DOM frameworks freeze under the computational weight of calculating multiple simultaneous $O(N^2)$ force-directed layouts for 10,000 nodes. By offloading the math to Rust and the interpolation to WASM Canvas, NetVis delivers a profoundly innovative "Show, Don't Tell" storytelling mechanism.

33.8 Fractal Topology: Continuous Semantic Zoom

If cinematic morphing solves the problem of cognitive continuity across *perspectives*, Continuous Semantic Zoom solves the most notorious problem in all of network visualization: *The Hairball*.

When attempting to render an enterprise network with 50,000 nodes, displaying every entity simultaneously results in an unreadable mass of overlapping lines. Practitioners typically respond by creating disjointed diagrams: one for the global WAN, one for the regional datacenter, and one for the local rack. This fragments the mental model.

Inspired by modern interactive cartography (e.g., Google Maps), NetVis introduces a **Fractal Topology Grammar**. Instead of defining a single layout, the user defines a Level-of-Detail (LOD) hierarchy, instructing the engine to apply different layout algorithms and semantic collapse rules based on the camera's Z-axis depth in the WASM viewer.

```
fractal_map = (
    NetVis(massive_global_topology)

    # Zoom Level 0 (Global): Collapse everything into geographic continent nodes
    .zoom_level(0, layout="geographic", collapse_by="continent")

    # Zoom Level 1 (Regional): Unpack continents into radial datacenter clusters
    .zoom_level(1, layout="radial", collapse_by="datacenter")

    # Zoom Level 2 (Datacenter): Unpack datacenters into Spine-Leaf fabrics
    .zoom_level(2, layout="spine-leaf", collapse_by="rack")

    # Zoom Level 3 (Rack): Reveal raw physical port wiring
    .zoom_level(3, layout="port-map")
)
```

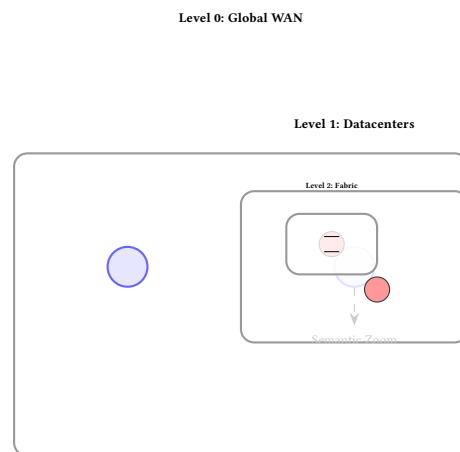


Figure 88. Continuous Semantic Zoom (Fractal Topology). The engine dynamically unpacks hierarchical abstractions based on camera depth. Zooming into a high-level "Region" node seamlessly reveals the underlying "Datacenter" radial cluster, which in turn reveals the "Spine-Leaf" fabric, all within a single unified coordinate space.

This fundamentally transforms the visualization from a static picture into a **Continuous Data Explorer**. As the user scrolls their mouse wheel, the Rust engine dynamically partitions the graph, expands the topological hierarchy, and applies localized layout algorithms in real time. The practitioner simply declares the abstraction rules for each tier, and the grammar elegantly manages the complex LOD mathematics.

By lifting interaction, time, cinematic spatial morphing, and continuous semantic zoom into the grammar itself, NetVis bridges the gap between static publication graphics and deeply interactive diagnostic applications.

33.9 The How We Built It: AST Compilation to Rust

Under the hood, this fluent Python API does not perform rendering directly. Instead, it constructs an Abstract Syntax Tree (AST) representing the visual specification.



Figure 89. The Grammar of Network Graphics architecture pipeline. Fluent Python builder commands accumulate state into an abstract visual specification, which is serialized across the PyO3 boundary to construct the internal Rust StyleSheet and filter pipeline.

Each method call (e.g., `.node_type()`) sequentially accumulates configuration state into a single `RenderSpecification` object. When `spec.render()` is invoked, the AST is passed across the PyO3 boundary. The core Rust engine natively interprets the bindings by populating the `StyleSheet` (filling `node_by_type`, `node_rules`, and `VisualMapping` collections), executing the filter evaluator matching the `.filter()` requests, and then computing the final scene layout. This architecture maintains the zero-cost abstraction benefits of Rust while exposing a deeply intuitive, domain-specific declarative interface.

34 API and Usage

34.1 Rust API

Function	Input	Returns
<code>contracts::v1::render</code>	Graph, Options	RenderEnvelope
<code>Layout::layout</code>	Graph	Scene
<code>SvgRenderer::render</code>	Scene	SVG string
<code>render_to_png</code>	SVG, PngOptions	PNG bytes
<code>render_to_pdf</code>	SVG, PdfOptions	PDF bytes
<code>to_dot</code>	NetVisGraph	DOT string
<code>ExportBuilder::to_png</code>	PngOptions	PNG bytes

Table 17. Core Rust API entry points.

34.2 WASM / JavaScript

JavaScript integration

```
import { init, renderToSVG, renderWithDiagnostics } from '@netvis/core';
await init();
const { output, diagnostics } = await renderWithDiagnostics(topologyJson);
```

A background Web Worker API offloads layout computation, streaming intermediate node positions via progress callbacks.

34.3 Temporal Player API

The WASM target exports a `TemporalPlayer` session for interactive temporal playback in the browser:

Function	Description
<code>createTemporalPlayer(canvasId, topology)</code>	Create a player from base topology + events
<code>temporalPlay(id)</code>	Start RAF-driven playback
<code>temporalPause(id)</code>	Pause playback
<code>temporalToggle(id)</code>	Toggle play/pause
<code>temporalStep(id, n)</code>	Step forward/backward by <i>n</i> snapshots
<code>temporalScrub(id, step)</code>	Jump to a specific step index
<code>temporalStatus(id)</code>	Returns {step, timestamp, isPlaying, filterHasNoMatches}
<code>setupTemporalKeyboardShortcuts(id)</code>	Register keyboard controls (Space, Arrows)

Table 18. Temporal player WASM/JavaScript API. See §22 for the underlying playback model.

35 Deployment

NetVis compiles from a single Rust codebase to three targets:

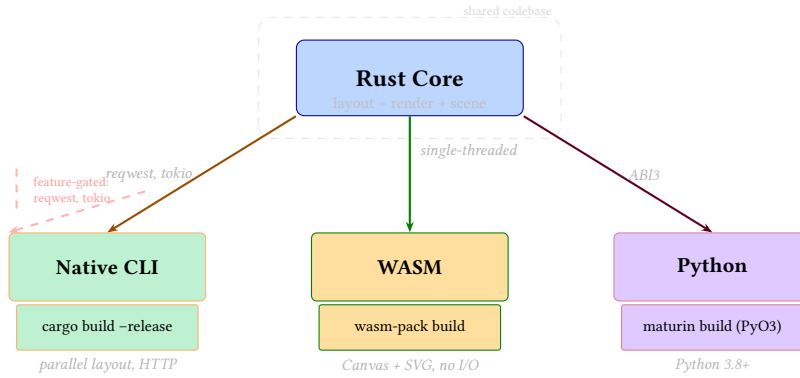


Figure 90. Deployment architecture. A single Rust codebase compiles to three targets via feature flags. WASM-incompatible crates (request, tokio) are gated to native-only. The Python binding uses PyO3 with ABI3 stability for broad version support.

Target	Build command	Notes
Native CLI	<code>cargo build --release --features cli</code>	Full feature set; HTTP adapters; async I/O
WASM [26]	<code>wasm-pack build --target web</code>	Single-threaded; Canvas + SVG rendering; no file I/O
Python	<code>maturin build --features python</code>	PyO3, ABI3 stability (Python 3.8+)

Table 19. Deployment targets and build commands.

WASM-incompatible crates (request, tokio) are gated to native-only via feature flags.

Deterministic mode. Reproducible output for CI/CD pipelines is enabled via CLI `--deterministic` or YAML `render.seed`. The mechanism is a thread-local RAI guard (`DeterministicMode`): when enabled, `HashMap` iteration is stabilized by key-sorting, random seeds are fixed, and ID generation is deterministic. The guard stores the previous state and restores it on `Drop`, supporting correct nesting. `with_deterministic(f)` provides a closure-scoped variant.

36 Performance

Benchmarks were measured on an Apple M4 Pro with 24 GB RAM, running macOS and Rust 1.93.

Operation	100	500	1K	2K	5K
Force-directed layout	20 ms	144 ms	309 ms	816 ms	1.75 s
Hierarchical layout	1.5 ms	8 ms	19 ms	—	—
SVG render	0.8 ms	4.5 ms	11 ms	76 ms	223 ms
Peak heap allocation	215 KB	952 KB	1.9 MB	3.8 MB	9.7 MB

Table 20. Performance baselines by topology size (spine-leaf topologies).

A 10,000-node spine-leaf topology (Enterprise engine, auto LOD) renders in 5.3 s end-to-end.

36.1 Multi-Core Parallel Simulation

The native CLI target uses a Rayon-based `ParallelSimulation` for force-directed layout. Force computation and position integration are parallelized across available cores, achieving up to 13× speedup at 10K scale on an 8-core machine. WASM remains single-threaded (multi-threading is available via the opt-in `wasm-rayon` feature flag when `SharedArrayBuffer` is supported).

36.2 Streaming Geometry Export

For large topologies (10K+ nodes), `render_to_writer` streams SVG output directly to an I/O writer, eliminating the $O(n)$ intermediate string allocation that would otherwise dominate memory at scale.

36.3 Interactive Layout Refinement

When a user drags a node in the interactive editor, a `reheat_neighborhood` pass applies distance-based force falloff (1.0 at the moved node, decaying to 0.4 at two-hop distance) followed by a 20-iteration cooling simulation.

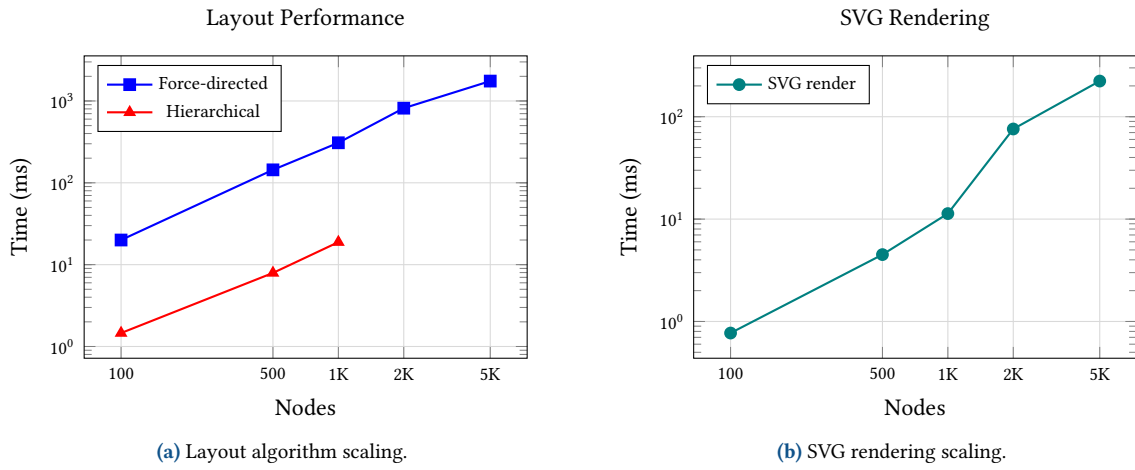


Figure 91. NetVis benchmark results on spine-leaf topologies. Left: layout computation time for force-directed (Barnes–Hut) and hierarchical (Sugiyama) engines. Right: SVG rendering time including label placement. Both axes are log-scaled; data points are median wall-clock times.

An `active_mask` optimization skips force computation for nodes outside the affected neighborhood, reducing per-drag cost by ~35%.

Tool	100 nodes	1K nodes	2K nodes
Graphviz sfdp	84 ms	250 ms	914 ms
Graphviz neato	88 ms	1,024 ms	4,434 ms
NetVis (full pipeline)	24 ms	330 ms	620 ms

Table 21. Comparative benchmark on identical spine-leaf topologies. NetVis times include layout, label placement, styling, and SVG rendering.

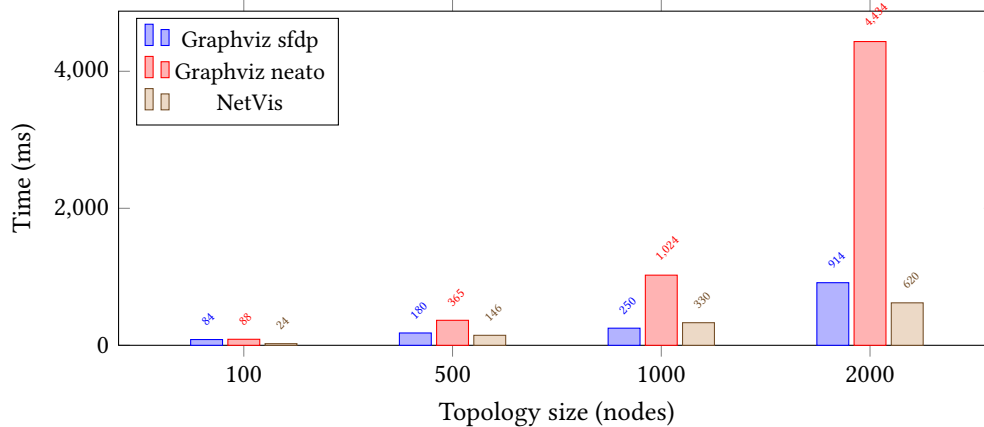


Figure 92. Comparative performance on identical spine-leaf topologies. NetVis includes the full pipeline (layout, labels, styling, SVG render); Graphviz includes layout and SVG output only.

37 Conclusion

This report has presented NetVis, a 108,000+ line Rust-based network topology visualization engine. Four contributions distinguish it from prior work:

1. **Dual-engine force-directed layout.** A Standard engine for topologies up to 2,000 nodes and a Barnes–Hut-accelerated Enterprise engine scaling to 10,000+ nodes (§7), complemented by six additional layout algorithms (hierarchical, geographic, isometric, orthogonal, subway, radial/tree).
2. **R*-tree-accelerated label placement.** A two-phase greedy algorithm using spatial indexing for $O(\log n)$ collision queries, placing labels for 5,000-node topologies in under 200 ms (§14).
3. **Composable refiner pipeline.** Ten post-layout refiners—label placement, edge bundling, smart routing, deoverlap, nudging, boundary bundling, and others—that compose in user-specified order to separate layout concerns from visual polish (§6, §16).

4. **Automated quality metrics.** Sixteen metric families (crossing count, angular resolution, overlap ratio, label clarity, and others) computed in-pipeline, enabling regression testing and objective comparison (§30).

Benchmarks on identical spine-leaf topologies show NetVis to be 1.5–7× faster than Graphviz `sfdp/neato` while producing the full rendering pipeline (layout, labels, styling, SVG output) rather than layout alone (§36). The system compiles from a single codebase to native CLI, WebAssembly, and Python targets (§35), and exposes domain-specific features—parallel edge rendering with automatic LAG aggregation (§9), interface rendering (§11), topology diffing (§21), traffic animation (§23), and metric encoding (§13)—that general-purpose graph tools lack.

Recent additions further distinguish NetVis from general-purpose tools: an event-sourced temporal model with interactive playback (§22), first-class annotations with spatial collision avoidance (§24), a browser-based editor with WebWorker layout and node pinning (§25), interactive filtering (§28), and WCAG 2.1 accessible SVG output (§31).

38 Limitations

Limitation	Description	Workaround / Plan
Scale (>10K nodes)	Enterprise engine handles 10K nodes in ~5 s; 50K+ would benefit from GPU acceleration	Use hierarchical/geographic/tree layout; GPU via WebGPU is under investigation
Greedy label placement	Sequential placement; pathological dense clusters may produce overlaps	Suppress labels via LOD; increase canvas size; simulated-annealing post-pass planned
Single-threaded WASM	No threading in browser; large layouts slower	WebWorker offloading implemented (§25); opt-in <code>wasm-rayon</code> feature for <code>SharedArrayBuffer</code> environments
Non-determinism	HashMap iteration order is non-deterministic by default	Use <code>--deterministic</code> or <code>render.seed</code>
Adapter coverage	Only YAML, NetBox, LLDP/CDP built in	Adapter trait is public; community adapters encouraged
No user study	Quality claims based on automated metrics only	Controlled user study planned

Table 22. Known limitations and workarounds.

39 Future Work

1. **GPU-accelerated layout.** WebGPU compute shaders for 50,000+ node topologies.
2. **Streaming topology ingestion.** Live gNMI/gRPC streaming feeding the event-sourced temporal model (§22) for real-time topology updates without full snapshot reload.
3. **Simulated-annealing label post-pass.** Global optimization resolving remaining overlaps in dense clusters.
4. **User study.** A controlled perceptual evaluation with practising network engineers, using a within-subjects design comparing NetVis output against Graphviz and draw.io baselines on topology comprehension tasks (path identification, failure domain detection, capacity bottleneck location). We plan to measure task completion time, error rate, and subjective quality via the System Usability Scale (SUS) [27].
5. **Collaborative editing.** CRDT-based multi-user conflict resolution extending the editor module (§25).
6. **SNMP and streaming telemetry adapters.** Community adapters for SNMP, Napalm, and gNMI.
7. **Parallel edge animation.** Per-link traffic flow animation within a LAG bundle.

References

- [1] NetBox Labs, *NetBox: The leading solution for network automation*, 2024. [Online]. Available: <https://netbox.dev>
- [2] E. R. Gansner and S. C. North, “An open graph visualization system and its applications to software engineering,” 11, vol. 30, 2000, pp. 1203–1233.
- [3] M. Chimani, C. Gutwenger, M. Jünger, G. W. Klau, K. Klein, and P. Mutzel, “The open graph drawing framework (OGDF),” *Handbook of Graph Drawing and Visualization*, pp. 543–569, 2013.
- [4] M. Bostock, V. Ogievetsky, and J. Heer, “D3: Data-driven documents,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 12, pp. 2301–2309, 2011.

- [5] yWorks GmbH, *yEd graph editor*, 2024. [Online]. Available: <https://www.yworks.com/products/yed>
- [6] L. Wilkinson, *The Grammar of Graphics*, 2nd. Springer, 2005.
- [7] T. M. J. Fruchterman and E. M. Reingold, “Graph drawing by force-directed placement,” *Software — Practice and Experience*, vol. 21, no. 11, pp. 1129–1164, 1991.
- [8] bluss, *Petgraph – graph data structure library for Rust*, 2024. [Online]. Available: <https://crates.io/crates/petgraph>
- [9] K. Sugiyama, S. Tagawa, and M. Toda, “Methods for visual understanding of hierarchical system structures,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 11, no. 2, pp. 109–125, 1981.
- [10] L. Verlet, “Computer “experiments” on classical fluids. I. thermodynamical properties of Lennard-Jones molecules,” *Physical Review*, vol. 159, pp. 98–103, 1967.
- [11] J. Barnes and P. Hut, “A hierarchical $O(N \log N)$ force-calculation algorithm,” *Nature*, vol. 324, pp. 446–449, 1986.
- [12] E. M. Reingold and J. S. Tilford, “Tidier drawings of trees,” *IEEE Transactions on Software Engineering*, vol. SE-7, no. 2, pp. 223–228, 1981.
- [13] J. Christensen, J. Marks, and S. Shieber, “An empirical study of algorithms for point-feature label placement,” in *ACM Transactions on Graphics*, vol. 14, 1995, pp. 203–232.
- [14] N. Beckmann, H.-P. Kriegel, R. Schneider, and B. Seeger, “The R*-tree: An efficient and robust access method for points and rectangles,” *ACM SIGMOD Record*, vol. 19, no. 2, pp. 322–331, 1990.
- [15] Stoeoef, *Rstar – R*-tree spatial index for Rust*, 2024. [Online]. Available: <https://crates.io/crates/rstar>
- [16] D. Holten and J. J. van Wijk, “Force-directed edge bundling for graph visualization,” *Computer Graphics Forum*, vol. 28, no. 3, pp. 983–990, 2009.
- [17] A. Riskus, “Approximation of a cubic Bézier curve by circular arcs and vice versa,” *Information Technology and Control*, vol. 35, no. 4, pp. 371–378, 2006.
- [18] C. Yuksel, S. Schaefer, and J. Keyser, “Parameterization and applications of Catmull–Rom curves,” *Computer-Aided Design*, vol. 43, no. 7, pp. 747–755, 2011. DOI: [10.1016/j.cad.2010.08.008](https://doi.org/10.1016/j.cad.2010.08.008)
- [19] C. de Boor, “On calculating with B-splines,” *Journal of Approximation Theory*, vol. 6, no. 1, pp. 50–62, 1972. DOI: [10.1016/0021-9045\(72\)90080-9](https://doi.org/10.1016/0021-9045(72)90080-9)
- [20] W. Boehm, “Inserting new knots into B-spline curves,” *Computer-Aided Design*, vol. 12, no. 4, pp. 199–201, 1980. DOI: [10.1016/0010-4485\(80\)90154-2](https://doi.org/10.1016/0010-4485(80)90154-2)
- [21] D. H. U. Kochanek and R. H. Bartels, “Interpolating splines with local tension, continuity, and bias control,” in *Proceedings of SIGGRAPH ’84*, 1984, pp. 33–41. DOI: [10.1145/800031.808575](https://doi.org/10.1145/800031.808575)
- [22] G. Taubin, “A signal processing approach to fair surface design,” in *Proceedings of SIGGRAPH ’95*, 1995, pp. 351–358. DOI: [10.1145/218380.218473](https://doi.org/10.1145/218380.218473)
- [23] H. C. Purchase, “Metrics for graph drawing aesthetics,” *Journal of Visual Languages and Computing*, vol. 13, no. 5, pp. 501–516, 2002.
- [24] C. Dunne and B. Shneiderman, “Motif simplification: Improving network visualization readability with fan, connector, and clique glyphs,” *Proc. ACM CHI*, pp. 3247–3256, 2015.
- [25] J. Abello, F. van Ham, and N. Krishnan, “ASK-GraphView: A large scale graph visualization system,” in *IEEE Transactions on Visualization and Computer Graphics*, vol. 12, 2006, pp. 669–676.
- [26] A. Haas et al., “Bringing the web up to speed with WebAssembly,” in *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.
- [27] J. Brooke, “SUS: A “quick and dirty” usability scale,” in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds., Taylor & Francis, 1996, pp. 189–194.

40 YAML Configuration Examples

This section consolidates the detailed YAML configuration snippets referenced throughout the document, keeping the core architectural discussion concise.

40.1 Basic Campus Network (§4)

The minimal topology YAML requires only name, nodes, and edges. The rendering section is optional; NetVis selects a suitable layout automatically.

Minimal topology (campus-edge.yaml)

```
name: Campus edge
render:
  theme: light
nodes:
  - { id: inet, label: Internet, node_type: cloud }
  - { id: fw1, label: Firewall, node_type: firewall }
```

```

- { id: r1,   label: Edge Router, node_type: router }
- { id: s1,   label: Access Switch, node_type: switch }
edges:
- { from: inet, to: fw1,  label: uplink }
- { from: fw1,  to: r1,   label: transit }
- { from: r1,   to: s1,   label: 10G }

```

```
netvis -input campus-edge.yaml -o campus.svg
```

40.2 Geographic CDN (§4)

Nodes with lat/lon attributes use the geographic layout with great-circle arc edges and a background map layer.

Geographic layout (cdn.yaml, excerpt)

```

render:
  preset: geographic
  theme: dark
geographic:
  show_map: true
  great_circle_arcs: true
nodes:
- { id: nyc, label: New York, node_type: router,
  lat: 40.7128, lon: -74.0060 }
- { id: lon, label: London, node_type: router,
  lat: 51.5074, lon: -0.1278 }

```

40.3 Isometric Multi-Layer (§4)

The isometric layout renders protocol stacks or layered architectures with a pseudo-3D perspective.

Isometric layout (multilayer.yaml, excerpt)

```

render:
  layout: isometric
layers:
- { id: physical, label: Physical, order: 0 }
- { id: network,  label: Network,  order: 1 }
- { id: routing,  label: Routing,  order: 2 }
nodes:
- { id: p-core1, label: Core-1, node_type: router,
  attributes: { layer: physical } }

```

40.4 Parallel Edges and Bond Groups (§4)

Multiple physical links between the same pair of nodes are grouped into an EdgeGroup.

Parallel edge modes

```

edges:
  # LAG: 4x 10G auto-aggregated to 40G
- { from: core1, to: core2, label: "10G",
  bond_group: core-lag, parallel_mode: bundled }
- { from: core1, to: core2, label: "10G",
  bond_group: core-lag }
  # Redundant uplinks shown as offset lines
- { from: core1, to: dist1, label: "10G",
  parallel_mode: offset }
- { from: core1, to: dist1, label: "10G" }

```

40.5 Subway Layout (§4)

Subway layout

```
render:
  layout: subway
  theme: blueprint
```

40.6 Pipeline Composition (§6)

Pipeline composition in YAML

```
render:
  layout: force_directed
  refiners: [labels, bundling, nudging, constraints]
```

40.7 Interface Configuration (§11)

Interface configuration

```
nodes:
  - id: core-sw
    node_type: switch
    interfaces:
      - { id: eth0, status: up, mode: port, marker: dot }
      - { id: eth1, status: warning, mode: detailed, speed: 10G }
      - { id: eth2, status: critical, mode: port }
```

40.8 Edge Gradients (§12)

Edge gradient

```
edges:
  - from: core1
    to: edge1
    gradient:
      start_color: "#3b82f6" # blue at source
      end_color: "#ef4444" # red at destination
```

40.9 Metric Encoding (§13)

Metric encoding

```
render:
  metric_encoding:
    - { attribute: cpu_utilization, channel: fill_color, palette: reds }
    - { attribute: packet_loss, channel: size,
        min_size: 15, max_size: 40 }
```

40.10 Traffic Animation (§23)

Traffic overlay

```
traffic:
  edges:
    - { id: core-to-agg, utilization: 0.75,
        direction: bidirectional, style: dot }
```


40.11 Annotations (§24)

Annotation configuration

```
annotations:
- anchor: router-fra-01
  type: callout
  text: "Primary BGP peer"
  shape: box
  layer: bgp-overlay
- anchor: rack-a-2
  type: group
  text: "Storage Cluster"
  style: dashed
```

40.12 Assigning Effects (§26)

Assigning effects

```
nodes:
- id: core-router
  highlight: true # default glow
- id: alert-node
  highlight: "emphasis" # glow-emphasis
```

References

- [1] NetBox Labs, *NetBox: The leading solution for network automation*, 2024. [Online]. Available: <https://netbox.dev>
- [2] E. R. Gansner and S. C. North, “An open graph visualization system and its applications to software engineering,” 11, vol. 30, 2000, pp. 1203–1233.
- [3] M. Chimani, C. Gutwenger, M. Jünger, G. W. Klau, K. Klein, and P. Mutzel, “The open graph drawing framework (OGDF),” *Handbook of Graph Drawing and Visualization*, pp. 543–569, 2013.
- [4] M. Bostock, V. Ogievetsky, and J. Heer, “D3: Data-driven documents,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 12, pp. 2301–2309, 2011.
- [5] yWorks GmbH, *yEd graph editor*, 2024. [Online]. Available: <https://www.yworks.com/products/yed>
- [6] L. Wilkinson, *The Grammar of Graphics*, 2nd. Springer, 2005.
- [7] T. M. J. Fruchterman and E. M. Reingold, “Graph drawing by force-directed placement,” *Software — Practice and Experience*, vol. 21, no. 11, pp. 1129–1164, 1991.
- [8] bluss, *Petgraph – graph data structure library for Rust*, 2024. [Online]. Available: <https://crates.io/crates/petgraph>
- [9] K. Sugiyama, S. Tagawa, and M. Toda, “Methods for visual understanding of hierarchical system structures,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 11, no. 2, pp. 109–125, 1981.
- [10] L. Verlet, “Computer “experiments” on classical fluids. I. thermodynamical properties of Lennard-Jones molecules,” *Physical Review*, vol. 159, pp. 98–103, 1967.
- [11] J. Barnes and P. Hut, “A hierarchical $O(N \log N)$ force-calculation algorithm,” *Nature*, vol. 324, pp. 446–449, 1986.
- [12] E. M. Reingold and J. S. Tilford, “Tidier drawings of trees,” *IEEE Transactions on Software Engineering*, vol. SE-7, no. 2, pp. 223–228, 1981.
- [13] J. Christensen, J. Marks, and S. Shieber, “An empirical study of algorithms for point-feature label placement,” in *ACM Transactions on Graphics*, vol. 14, 1995, pp. 203–232.
- [14] N. Beckmann, H.-P. Kriegel, R. Schneider, and B. Seeger, “The R*-tree: An efficient and robust access method for points and rectangles,” *ACM SIGMOD Record*, vol. 19, no. 2, pp. 322–331, 1990.
- [15] Stoeoef, *Rstar – R*-tree spatial index for Rust*, 2024. [Online]. Available: <https://crates.io/crates/rstar>
- [16] D. Holten and J. J. van Wijk, “Force-directed edge bundling for graph visualization,” *Computer Graphics Forum*, vol. 28, no. 3, pp. 983–990, 2009.
- [17] A. Riskus, “Approximation of a cubic Bézier curve by circular arcs and vice versa,” *Information Technology and Control*, vol. 35, no. 4, pp. 371–378, 2006.
- [18] C. Yuksel, S. Schaefer, and J. Keyser, “Parameterization and applications of Catmull–Rom curves,” *Computer-Aided Design*, vol. 43, no. 7, pp. 747–755, 2011. DOI: [10.1016/j.cad.2010.08.008](https://doi.org/10.1016/j.cad.2010.08.008)

- [19] C. de Boor, "On calculating with B-splines," *Journal of Approximation Theory*, vol. 6, no. 1, pp. 50–62, 1972. DOI: [10.1016/0021-9045\(72\)90080-9](https://doi.org/10.1016/0021-9045(72)90080-9)
- [20] W. Boehm, "Inserting new knots into B-spline curves," *Computer-Aided Design*, vol. 12, no. 4, pp. 199–201, 1980. DOI: [10.1016/0010-4485\(80\)90154-2](https://doi.org/10.1016/0010-4485(80)90154-2)
- [21] D. H. U. Kochanek and R. H. Bartels, "Interpolating splines with local tension, continuity, and bias control," in *Proceedings of SIGGRAPH '84*, 1984, pp. 33–41. DOI: [10.1145/800031.808575](https://doi.org/10.1145/800031.808575)
- [22] G. Taubin, "A signal processing approach to fair surface design," in *Proceedings of SIGGRAPH '95*, 1995, pp. 351–358. DOI: [10.1145/218380.218473](https://doi.org/10.1145/218380.218473)
- [23] H. C. Purchase, "Metrics for graph drawing aesthetics," *Journal of Visual Languages and Computing*, vol. 13, no. 5, pp. 501–516, 2002.
- [24] C. Dunne and B. Shneiderman, "Motif simplification: Improving network visualization readability with fan, connector, and clique glyphs," *Proc. ACM CHI*, pp. 3247–3256, 2015.
- [25] J. Abello, F. van Ham, and N. Krishnan, "ASK-GraphView: A large scale graph visualization system," in *IEEE Transactions on Visualization and Computer Graphics*, vol. 12, 2006, pp. 669–676.
- [26] A. Haas et al., "Bringing the web up to speed with WebAssembly," in *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.
- [27] J. Brooke, "SUS: A "quick and dirty" usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds., Taylor & Francis, 1996, pp. 189–194.

Index

- accessibility, 53
 - contrast, 53
- adapters, 10
- AestheticScore, 50
- annotation, 45
- AnnotationData, 9, 45
- AnnotationTarget, 45
- API, 63
- architecture, 10
- ARIA, 53
- auto-layout, 47

- B-spline, 38
- Bézier, 38
- bandwidth aggregation, 21
- Barnes-Hut, 11
- benchmarks, 64
- betweenness centrality, 28
- bond group, 21
- build_scene_from_snapshot, 44

- callout, 45
- cancellation, 37
- Canvas2D, 52
- CanvasHitIndex, 53
- Catmull-Rom, 38
- chord diagram, 19
- CLI, 63
- click-to-navigate, 53
- collision avoidance, 27
- color palettes, 25
- community detection, 48
- compositing, 52
- conclusion, 65
- constraint solver, 35
- constraints, 35
- contrast ratio, 53
- corner labels, 28
- crossing minimizer, 37
- crossings, 50
- cursor-relative zoom, 53
- curves, 38

- data model, 8
- deployment, 63
- deterministic mode, 64
- DeterministicMode, 64
- device icons, 22
- diff, 43
- dirty region, 52
- drop shadow, 47

- edge, 8
- edge bundling, 28
- edge crossings, 37
- edge gradients, 24
- editor, 46

- FDEB, 28

- figure sizing, 53
- filter DSL, 49
- flow direction, 24
- flow field, 45
- force-directed edge bundling, 28
- future work, 66
- fuzzy matching, 49

- geographic scale, 46
- Girvan-Newman, 48
- glow, 47
- GPU acceleration, 66
- Grammar of Graphics, 55
- graph analytics, 48

- heatmap, 45
- hit detection, 53
- HTML export, 51

- IEEE figure mode, 53
- import/export, 47
- inset map, 53
- InteractiveFilter, 49
- interface status, 23
- interfaces, 23

- label placement, 27
- label propagation, 48
- LAG, 21
- layout
 - circular, 19
 - composite, 11
 - force-directed, 11
 - geographic, 11
 - geographic starburst, 14
 - hierarchical, 11
 - isometric, 11
 - orthogonal, 11, 16
 - radial, 11
 - subway, 11, 16
 - tree, 11
- layout worker, 47
- limitations, 66
- link aggregation, 21
- link utilization, 26
- Louvain, 48

- MassData, 11
- metric encoding, 25
- mini-map, 53

- neighborhood reheating, 64
- netvis-editor, 47
- node, 8
- node pinning, 47
- node renderer, 54
- node shapes, 22
- node_type, 22
- nudging, 33

- octilinear, [16](#)
- OffscreenCanvas, [52](#)
- orthogonal elbow, [28](#)
- overlap, [50](#)
- parallel edges, [21](#)
- parallel simulation, [64](#)
- ParallelSimulation, [12](#)
- path emphasis, [24](#), [25](#)
- PDF export, [54](#)
- performance, [64](#)
- pipeline, [10](#)
- playback, [44](#)
- PlaybackController, [44](#)
- POP layout, [14](#)
- ports, [23](#)
- progressive layout, [37](#)
- progressive rendering, [53](#)
- publication-quality, [53](#)
- Python API, [55](#)
- Python binding, [63](#)
- QR code, [54](#)
- quadtree, [11](#)
- quality metrics, [50](#)
- R*-tree, [27](#)
- Rayon, [12](#), [64](#)
- refiners, [33](#)
- RegionKind
 - Annotation, [46](#)
- Relational Mapping, [55](#)
- renderer, [10](#)
- rendering pipeline, [51](#)
- routing
 - orthogonal, [33](#)
 - port-aware, [33](#)
 - smart, [33](#)
- scalability, [66](#)
- scale bar, [45](#), [46](#)
- scene graph, [8](#)
- Share URL, [47](#)
- snapshot, [44](#)
- soft constraints, [35](#)
- splines, [38](#)
- streaming export, [64](#)
- Sugiyama, [11](#)
- SVG, [51](#)
- tapered edges, [25](#)
- temporal analysis, [44](#)
- TemporalGraph, [10](#), [44](#)
- TemporalPlayer, [44](#)
- themes, [20](#)
- Timeline, [44](#)
- timeline, [44](#)
- topology diffing, [43](#)
- topology graph, [8](#)
- Track, [44](#)
- traffic animation, [45](#)
- undo/redo, [46](#)
- utilization, [45](#)
- validation, [49](#)
- variable-width edges, [25](#)
- ViewportTransform, [53](#)
- visual effects, [47](#)
- WASM, [46](#), [63](#)
 - temporal, [44](#)
 - TemporalPlayer API, [63](#)
- WASM API, [63](#)
- WCAG, [53](#)
 - contrast ratio, [53](#)
- weathermap, [26](#)
- Web Component, [47](#)
- WebAssembly, [63](#)
- WebWorker, [47](#)

Revision History

Version	Date	Changes
1.9.1	Mar 2026	Expanded IEEE/ACM figure export documentation (full preset table, YAML config, font/stroke scaling); WCAG contrast module description; constraint solver rendered example; circular HEB comparison figure; spine-leaf bundling figure
1.9.0	Mar 2026	WASM canvas inset overlay (mini-map) with click-to-navigate; PDF QR code embedding for physical-to-digital bridge; IEEE single/double-column figure export modes; inset maps, scale bars, and callouts for geographic visualizations; WCAG 2.1 contrast checking and high-contrast theme validation
1.8.0	Mar 2026	Event-sourced temporal model; PlaybackController/Track/Timeline primitives; WASM TemporalPlayer with RAF-driven playback; first-class annotation data model with three anchor modes; annotation spatial collision via R*-tree; interactive canvas filtering
1.5.0	Mar 2026	Subway, tree, radial layouts; interface rendering; node shapes; path emphasis; metric encoding; edge gradients; 6 new refiners; terminal renderer
1.4.0	Mar 2026	Parallel edges (5 modes); LAG aggregation; orthogonal layout; clarity scoring; graph analytics; filter DSL; fuzzy validation
1.3.0	Mar 2026	Margin insights; cover diagrams; design rule primitives
1.2.0	Mar 2026	Architectural strategy; formalized type logic
1.1.0	Mar 2026	Routing refiners; topology diffing; timeline; traffic animation; annotations; editor; effects; 16 quality metrics
1.0.0	Jan 2025	Initial pipeline specification